

Code Assistant Agent 项目全源码精讲

Code Assistant Agent 项目全源码精讲

一个基于 RAG 的代码问答与 Agent 系统，以 TinyDB 开源项目为知识库。本教程按「由地基到屋顶」的顺序，对项目中每一个代码文件进行详细讲解。

阅读指南

本教程是什么

这份文档把 code-assistant-agent 项目（FastAPI + RAG + ReAct Agent + Streamlit + Docker）的每一个代码文件都讲一遍。每一章的结构统一为：

1. **完整代码**：先把文件原样贴出来，你可以对着看。
2. **这段代码解决什么问题**：先讲设计意图，再讲实现。
3. **关键代码逐句拆解**：挑出值得细品的行，讲清楚“为什么这么写”。
4. **面试题要点**：面试官可能怎么问，你该怎么答。

推荐的阅读方法

- **顺序阅读**：第 1 层到第 8 层是按依赖关系排的，先讲地基、再讲业务。跳过前面的文件，后面的会出现看不懂的引用。
- **对照源码**：建议同时打开项目目录，找到对应的文件，边看边读。
- **动手验证**：每章结尾的「动手做」建议，跑一遍比只看代码理解深得多。
- **随时查**：这份文档里保留了完整代码块，可脱离项目单独阅读。

项目文件总览

先给你一张“地图”，后面每一层都会展开：

```
code-assistant-agent/
├── backend/
│   └── app/
│       ├── config.py           # 【第 1 层】总配置
│       ├── logger.py          # 【第 1 层】统一日志
│       ├── database.py        # 【第 1 层】异步数据库引擎 + 依赖注入
│       ├── auth.py            # 【第 3 层】JWT 鉴权 + bcrypt + Redis 黑名单
│       ├── middleware.py      # 【第 3 层】Redis 滑动窗口限流
│       └── models/            # 【第 2 层】9 张表的 ORM 模型
│           ├── user.py
│           ├── conversation.py
│           ├── agent_log.py
│           ├── retrieval_log.py
│           └── query_rewrite.py
```

		└─ evaluation_run.py	
		└─ index_version.py	
		└─ feedback.py	
		└─ __init__.py	
		└─ rag/	# 【第 4 层】RAG 检索管线
		└─ chunker.py	
		└─ dense_retriever.py	
		└─ sparse_retriever.py	
		└─ fusion.py	
		└─ query_rewriter.py	
		└─ reranker.py	
		└─ code_indexer.py	
		└─ user_upload.py	
		└─ test_set.py	
		└─ evaluation.py	
		└─ agent/	# 【第 5 层】ReAct Agent
		└─ llm.py	
		└─ tool_base.py	
		└─ prompt.py	
		└─ tools.py	
		└─ harness.py	
		└─ eval_tasks.py	# Agent 评测任务集（预留）
		└─ services/	# 【第 6 层】业务服务层
		└─ conversation_service.py	
		└─ routers/	# 【第 7 层】API 路由层
		└─ auth_router.py	
		└─ agent_router.py	
		└─ search_router.py	
		└─ upload_router.py	
		└─ main.py	# 【第 8 层】应用入口
	└─ tests/		# 【第 8.6 层】单元测试与集成测试
	└─ conftest.py		# 测试环境配置（连测试库、跳过密钥校验）
	└─ test_auth.py		# 密码哈希单元测试
	└─ test_auth_api.py		# 鉴权 API 集成测试
	└─ test_fusion.py		# RRF 融合单元测试
	└─ test_metrics.py		# 评估指标单元测试
	└─ Dockerfile		# 【第 8 层】后端镜像
└─ frontend/			
└─ streamlit_app.py			# 【第 8 层】Streamlit 前端
└─ Dockerfile			# 【第 8 层】前端镜像
└─ docker-compose.yml			# 【第 8 层】一键部署
└─ .github/workflows/ci.yml			# 【第 8 层】CI
└─ .env.example			# 环境变量模板
└─ README.md			# 项目说明

目录

- [第 0 章 项目整体架构](#)
- [第 1 层 地基：配置 / 日志 / 数据库](#)
 - [1.1 config.py](#)
 - [1.2 logger.py](#)
 - [1.3 database.py](#)
- [第 2 层 数据模型：9 张表的仓库](#)
 - [2.1 models/init.py](#)
 - [2.2 user.py](#)
 - [2.3 conversation.py](#)
 - [2.4 agent_log.py](#)
 - [2.5 retrieval_log.py](#)
 - [2.6 query_rewrite.py](#)

- [2.7 evaluation_run.py](#)
 - [2.8 index_version.py](#)
 - [2.9 feedback.py](#)
 - [第 3 层 安全：鉴权与限流](#)
 - [3.1 auth.py](#)
 - [3.2 middleware.py](#)
 - [第 4 层 RAG 检索管线](#)
 - [4.1 chunker.py](#)
 - [4.2 dense_retriever.py](#)
 - [4.3 sparse_retriever.py](#)
 - [4.4 fusion.py](#)
 - [4.5 query_rewriter.py](#)
 - [4.6 reranker.py](#)
 - [4.7 code_indexer.py](#)
 - [4.8 user_upload.py](#)
 - [4.9 test_set.py](#)
 - [4.10 evaluation.py](#)
 - [第 5 层 Agent：让模型自己决策](#)
 - [5.1 llm.py](#)
 - [5.2 tool_base.py](#)
 - [5.3 prompt.py](#)
 - [5.4 tools.py](#)
 - [5.5 harness.py](#)
 - [5.6 eval_tasks.py](#)
 - [第 6 层 服务层](#)
 - [6.1 conversation_service.py](#)
 - [第 7 层 API 路由层](#)
 - [7.1 auth_router.py](#)
 - [7.2 agent_router.py](#)
 - [7.3 search_router.py](#)
 - [7.4 upload_router.py](#)
 - [第 8 层 入口与部署](#)
 - [8.1 main.py](#)
 - [8.2 streamlit_app.py](#)
 - [8.3 docker-compose.yml](#)
 - [8.4 Dockerfile](#)
 - [8.5 ci.yml](#)
 - [8.6 tests/ — 单元测试与集成测试](#)
 - [第 9 章 全链路数据流总结](#)
 - [第 10 章 潜在改进建议](#)
 - [附录 A 环境变量参考](#)
-

第 0 章 项目整体架构

0.1 这个项目是什么

一句话定位：这是一个“你问问题，AI 去代码库里翻代码、然后回答你”的系统。

它把 TinyDB 这个开源项目（一个纯 Python 写的 JSON 文档数据库）的源码当成知识库，提供了三种能力：

能力	说明	对应的入口
代码检索	输入一个问题，返回最相关的代码片段	/api/search
AI 问答	Agent 自主决定搜什么、怎么搜，最终给出中文回答	/api/agent/chat
测试生成 / 代码解释	Agent 检索到代码后，让 LLM 解释它或给它写单元测试	Agent 的 explain / testgen 工具

前端是一个 Streamlit 页面（对话 / 检索 / 上传三个 Tab），后端是 FastAPI。

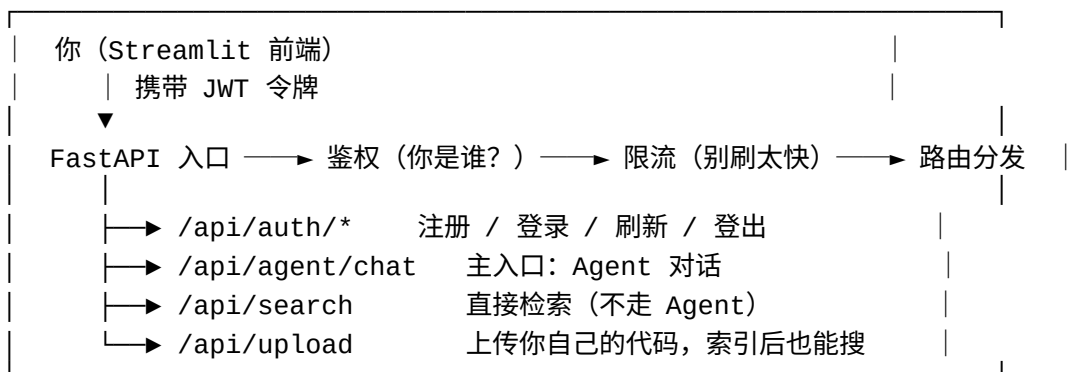
0.2 用一句话形容它的核心价值

它不是一个“玩具 demo”，而是一套**组件完整、工程化齐备、带评估数据**的 AI 应用：

- **RAG 管线完整**：HyDE 查询改写 → 双路召回（稠密 + 稀疏）→ RRF 融合 → cross-encoder 重排，每一步都有对应代码。
- **Agent 是手写的 ReAct**，不是调 LangGraph 一把梭——能讲清楚“思考-行动-观察”的原理。
- **工程化到位**：JWT 鉴权、Redis 滑动窗口限流、数据隔离、CI、Docker、可观测日志。
- **有评估**：20 条测试集 + 5 组消融实验，结论是“组件要按领域验证，不能无脑叠加”。

0.3 架构总览（用比喻理解）

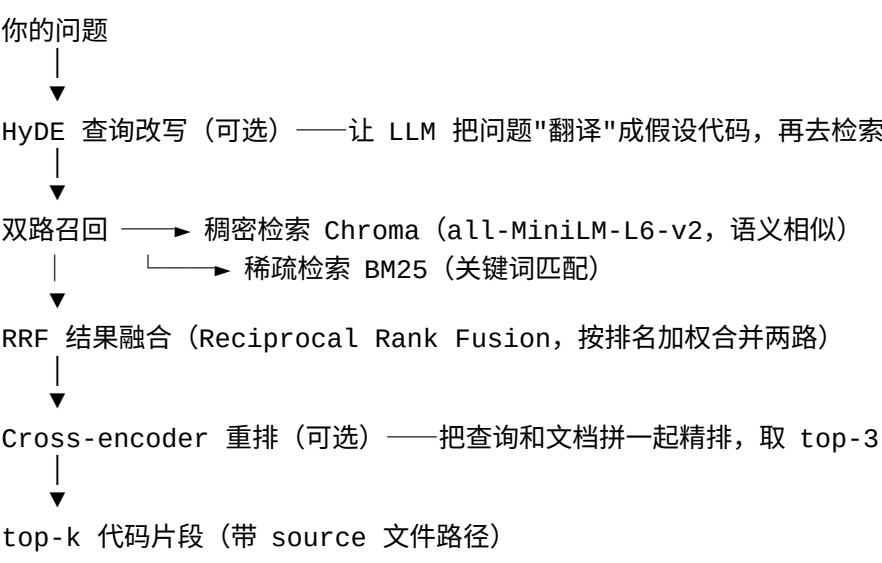
把整个系统想象成一座「图书馆 + 咨询台」：



图书馆有**藏书区**（离线建好的索引）、**检索台**（RAG 管线）、**咨询员**（Agent）。你既可以自己去检索台查（/api/search），也可以让咨询员帮你查并总结（/api/agent/chat）。咨询员自己也会去检索台查——所以 **Agent 和 RAG 是上下层关系**：RAG 是能力，Agent 是决策者。

0.4 核心业务链路：RAG 检索管线

这是项目技术含量最高的部分，也是后面第 4 层要展开的：



为什么需要双路？ 稠密检索懂语义（“存数据”能匹配到 `storages.py`），但可能漏掉关键词（对 `JSONStorage` 这种专有名词不敏感）；BM25 精确匹配关键词，但不懂同义词。两路互补，融合后更稳。

为什么融合用 RRF 而不是加权平均？ 因为两路分数的“度量衡”完全不同（一个是向量余弦距离，一个是词频统计），直接相加没有意义。RRF 只看**排名**不看分数，天然规避了分数不可比的问题。

0.5 Agent 的 ReAct 循环

Agent 层是项目另一个亮点。它让 LLM 自己决定“要不要查、查什么、查完怎么答”，循环最多 6 步：

思考：分析当前情况，决定下一步
行动：{"name": "search", "args": {"query": "..."}} ← LLM 输出 JSON
观察：工具返回的代码片段 ← 系统注入
思考：信息够了，组织答案
答案：<最终回答>

这个“思考-行动-观察”的循环就是 ReAct（Reasoning + Acting）。后面第 5 层会看到手写的括号深度解析器怎么从 LLM 输出里安全地抽出 Action JSON。

0.6 数据存哪、各司其职

存储	存什么	谁在写 / 谁在读
MySQL 8.0	用户、会话、消息、Agent 决策日志、检索日志、评估记录	ORM 模型（第 2 层）+ 服务层

存储	存什么	谁在写 / 谁在读
Redis 7	检索结果缓存、BM25 索引备份、token 黑名单、限流 ZSET	检索器 / 鉴权 / 中间件
Chroma（本地文件）	稠密向量索引（代码块 + 向量）	dense_retriever.py
文件系统	.env 配置、data/target_repo（TinyDB 源码）、data/chroma	config / code_indexer

0.7 技术栈总表

层	技术	关键点
后端框架	FastAPI（异步）	asyncio 并发、依赖注入、中间件
ORM / 数据库	SQLAlchemy 2.0 + aiomysql、MySQL 8.0	异步会话、声明式模型
缓存 / 限流	Redis 7	检索缓存、滑动窗口限流、token 黑名单
向量库	Chroma	本地持久化
Embedding	sentence-transformers/all-MiniLM-L6-v2	稠密检索向量化
检索	混合检索（Dense + BM25）	双路召回
融合	RRF	$1/(k+rank)$ 无权重融合
改写	HyDE	LLM 生成假设文档再检索
重排	cross-encoder（ms-marco）	对 top-N 精排
Agent	手写 ReAct	Thought-Action-Observation
LLM	DeepSeek API / Ollama	配置可切换
前端	Streamlit	对话 / 检索 / 上传三 Tab
部署	Docker Compose、GitHub Actions	一键启动 + CI

0.8 层与层的依赖关系

第 1 层 → 第 2 层 → 第 3 层 → 第 4 层 → 第 5 层 → 第 6 层 → 第 7 层 → 第 8 层

依赖方向（谁引用谁）：

- 第 1 层（**config / logger / database**）：被所有其他层引用，是地基。
- 第 2 层（**models**）：被服务层、路由层、RAG 层引用，是“货架”。
- 第 3 层（**auth / middleware**）：被路由层引用，保护整个 API。
- 第 4 层（**RAG**）：被 Agent 工具和检索路由引用。
- 第 5 层（**Agent**）：调用第 4 层的检索能力 + 第 1 层的 LLM。
- 第 6 层（**service**）：被第 7 层路由引用。
- 第 7 层（**routers**）：被 main.py 挂载，暴露 API。
- 第 8 层（**main / 前端 / 部署 / CI**）：组装一切。

所以阅读顺序是“由地基到屋顶”：先懂配置和数据库，再懂数据长什么样，再懂安全，再进入业务核心（RAG 和 Agent），最后看 API 如何暴露、系统如何部署。

第 1 层地基：配置 / 日志 / 数据库

学一个项目就像盖一栋楼。第 1 层是**地基**：

- `config.py` —— 决定这栋楼连哪些水电（数据库、Redis、LLM 配置）
- `logger.py` —— 楼里的监控探头（统一日志）
- `database.py` —— 楼里最主要的管道系统（异步数据库连接 + 依赖注入）

不先看懂它们，后面每个文件都会出现你看不懂的引用（`from app.config import settings`、`from app.logger import log`、`Depends(get_db)`）。

1.1 config.py — 总控制台

完整代码

```
import os
from pathlib import Path

from pydantic_settings import BaseSettings

PROJECT_ROOT = Path(__file__).resolve().parents[2]

class Settings(BaseSettings):
    # App
    APP_NAME: str = "Code Assistant Agent"
    DEBUG: bool = True

    # MySQL
    MYSQL_HOST: str = "localhost"
    MYSQL_PORT: int = 3306
    MYSQL_USER: str = "root"
    MYSQL_PASSWORD: str = "123456"
    MYSQL_DATABASE: str = "code_assistant"
```

```

# Redis
REDIS_HOST: str = "localhost"
REDIS_PORT: int = 6379
REDIS_CACHE_TTL: int = 300 # 检索缓存过期时间（秒），默认 5 分钟

# Chroma
CHROMA_PERSIST_DIR: str = "./data/chroma"

# 目标代码仓库路径
REPO_PATH: str = "./data/target_repo"

# Embedding 模型
# EMBEDDING_MODEL: str = "BAAI/bge-small-zh-v1.5"
EMBEDDING_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"
EMBEDDING_DEVICE: str = "cpu"

# 分块策略
CHUNK_STRATEGY: str = "recursive" # recursive / semantic / token
CHUNK_SIZE: int = 500
CHUNK_OVERLAP: int = 50

# LLM
# LLM_API_ENDPOINT: str = "http://localhost:11434/api/chat" # Ollama
LLM_API_ENDPOINT: str = "https://api.deepseek.com/v1/chat/completions"
LLM_MODEL: str = "deepseek-v4-flash"
LLM_API_KEY: str = ""
# LLM_MODEL: str = "gemma:7b"
# JWT
JWT_SECRET: str = "" # 从 .env 读取，禁止硬编码

class Config:
    env_file = PROJECT_ROOT / ".env"
    env_file_encoding = "utf-8"

    def _validate_secrets(self):
        """启动时校验密钥，缺失则拒绝启动，避免用空密钥签发 JWT"""
        if not self.LLM_API_KEY:
            raise ValueError(
                "LLM_API_KEY 未配置。请在 .env 中设置，或通过环境变量传入。"
                "禁止用空密钥运行。"
            )
        if len(self.JWT_SECRET) < 32:
            raise ValueError(
                "JWT_SECRET 未配置或太短（至少 32 字符）。请在 .env 中设置，"
                "可用: python -c \"import secrets; print(secrets.token_hex(32))\" 生
成。"
            )

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        # 测试场景可通过环境变量跳过校验（比如 CI 用固定测试密钥，或纯逻辑单测）
        if not os.environ.get("SKIP_SECRET_VALIDATION"):
            self._validate_secrets()

settings = Settings()

```

这段代码解决什么问题

整个项目有几十个“要变的东西”：数据库地址、Redis 端口、用哪个 LLM、密钥、分块大小……如果散落在各个文件里，改配置要满世界找，还容易把密码硬编码进代码。config.py

把所有配置集中到一个类里，启动时自动从 `.env` 读取。任何文件想用配置，只需 `from app.config import settings`。

关键代码逐句拆解

① `PROJECT_ROOT = Path(__file__).resolve().parents[2]`

找“项目根目录”：`__file__` 是当前文件路径

(`.../backend/app/config.py`)，`.resolve()` 解析成绝对路径，`.parents[2]` 往上走 2 层 (`parents[0]=app`，`parents[1]=backend`，`parents[2]=项目根目录`)。这样无论从哪个目录启动程序，`.env` 都能被找到。

② `class Settings(BaseSettings)` — Pydantic 的三大超能力

- **类型校验**：`MYSQL_PORT: int` 如果 `.env` 里写了 `abc`，启动即报错，而不是运行到一半才炸。
- **自动读环境变量**：字段名 `MYSQL_HOST` 自动匹配同名环境变量。
- **自动读 `.env` 文件**：`class Config` 里 `env_file` 指定路径。

三者合一：代码里只写默认值，真正的配置从 `.env` 注入，类型错了立刻报错。

③ 字段带默认值的类型注解

```
MYSQL_HOST: str = "localhost"
REDIS_CACHE_TTL: int = 300
DEBUG: bool = True
```

Pydantic 会在运行时做**强制类型转换**：`.env` 里的 `"300"` 字符串会被转成 `300` 整数。

④ LLM 配置那几行注释 = 切换开关

```
# LLM_API_ENDPOINT: str = "http://localhost:11434/api/chat" # Ollama
LLM_API_ENDPOINT: str = "https://api.deepseek.com/v1/chat/completions"
LLM_MODEL: str = "deepseek-v4-flash"
# LLM_MODEL: str = "gemma:7b"
```

想从 DeepSeek 切到本地 Ollama，改两行配置即可，业务代码零改动。这是“配置驱动”：把变化的东西从代码里剥离出来。

⑤ `class Config` 的 `env_file_encoding = "utf-8"`

如果不指定，Windows 上默认 GBK 编码，含中文注释的 `.env` 会解析失败。这是真实踩过的坑。

⑥ `_validate_secrets()` — 快速失败 (fail fast)

```
if not self.LLM_API_KEY:
    raise ValueError("LLM_API_KEY 未配置。...")
if len(self.JWT_SECRET) < 32:
    raise ValueError("JWT_SECRET 未配置或太短（至少 32 字符）。...")
```

防止新手忘了配密钥，系统“成功启动”却用空密钥签发 JWT——那等于任何人拿任意 token 都

能伪造身份。启动时就检查，缺了直接拒绝启动。

⑦ `__init__` 里的测试后门

```
if not os.environ.get("SKIP_SECRET_VALIDATION"):
    self._validate_secrets()
```

正常启动校验密钥，但跑单元测试时没有真密钥。所以提供 `SKIP_SECRET_VALIDATION=1` 跳过校验。你在 `tests/test_fusion.py` 开头看到的两行：

```
os.environ.setdefault("SKIP_SECRET_VALIDATION", "1")
os.environ.setdefault("JWT_SECRET", "test-secret-only-for-ci-0123456789abcdef")
```

这就是“测试隔离”：测试环境和生产环境的配置分开。

⑧ 模块级单例 `settings = Settings()`

文件底部直接创建全局实例，全项目 `from app.config import settings` 拿到的都是同一个对象，配置只加载一次。

面试题要点

Q：为什么用 `pydantic-settings` 而不是 `os.getenv()`？ 答：类型校验 + 自动读 `.env` + 字段带默认值。`os.getenv()` 拿到的都是字符串，类型转换得自己写，还容易忘。

Q：密钥怎么管理？ 答：密钥只放 `.env`，`.env` 在 `.gitignore` 里（不提交到仓库），用 `secrets.token_hex(32)` 生成。启动时 `_validate_secrets()` 校验缺失即报错，杜绝空密钥运行。

Q：测试和开发环境配置怎么隔离？ 答：CI 里通过环境变量 `SKIP_SECRET_VALIDATION=1` 跳过密钥校验，测试文件自己设置固定测试密钥；数据库/Redis 地址由 CI 服务容器提供，互不污染。

动手做

```
# 1. 复制环境变量模板
cp .env.example .env
# 2. 生成一个安全的 JWT 密钥
python -c "import secrets; print(secrets.token_hex(32))"
# 3. 填入 .env 后，试试不填 LLM_API_KEY 启动，观察报错
cd backend && python -c "from app.config import settings; print(settings.LLM_MODEL)"
```

1.2 `logger.py` — 监控探头

完整代码

```
import logging
import sys
```

```
def setup_logger(name: str = "code_assistant") -> logging.Logger:
    """创建统一格式的 logger"""
    logger = logging.getLogger(name)

    if not logger.handlers:
        logger.setLevel(logging.INFO)

        handler = logging.StreamHandler(sys.stdout)
        handler.setLevel(logging.INFO)

        formatter = logging.Formatter(
            "[%s] %(levelname)s [%s.%(funcName)s] %(message)s",
        )
        handler.setFormatter(formatter)
        logger.addHandler(handler)

    return logger

# 全局默认 logger
log = setup_logger()
```

这段代码解决什么问题

代码里到处是 `log.info(...)`、`log.warning(...)`。这个文件定义“log”从哪来、长什么样。用 `logging` 而不是 `print`，好处是**分级**（`debug/info/warning/error`）、**格式化**、**统一出口**。

关键代码逐句拆解

① `logging.getLogger(name)` — 单例工厂

同名 `logger` 只创建一次。第二次调用返回同一个对象。

② `if not logger.handlers:` — 防重复添加

因为 `setup_logger` 可能被多次调用（模块重复 `import`），如果每次都 `addHandler`，日志会被打印多遍。先检查已有 `handler` 就不再添加。

③ `StreamHandler(sys.stdout)` — 输出到标准输出

Docker 容器里 `docker logs` 能抓到 `stdout`。生产部署日志不写文件，走 `stdout` 是容器规范。

④ `Formatter` 格式设计

```
"[%s] %(levelname)s [%s.%(funcName)s] %(message)s"
```

输出示例：

```
[code_assistant] INFO [fusion.hybrid_search] Dense: 10 results, Sparse: 10 results
```

`module + funcName` 自动带上“哪条日志来自哪个文件的哪个函数”，排查问题直接定位代码位置。

⑤ 模块级 `log = setup_logger()`

文件底部创建全局单例，任何文件 `from app.logger import log` 直接用。

面试题要点

Q: 为什么不直接 `print`? 答: `print` 无法分级、无法统一格式、容器环境难管理。
`logging` 自带级别控制 (INFO/WARNING/ERROR)，且每条日志能自动带上模块和函数名，配合 Docker 的 `stdout` 收集方案，可观测性更好。

Q: 日志格式为什么带模块名? 答: 一个系统几十个模块，报错时能立刻知道日志来自哪个文件哪个函数，不用全局搜。这也是“可观测性”的基础——项目后面还有数据库级的日志 (`agent_logs / retrieval_logs`)，是日志的“持久化版”。

1.3 database.py — 管道系统

完整代码

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession,
async_sessionmaker
from sqlalchemy.orm import DeclarativeBase

from app.config import settings

# 异步 MySQL 连接 URL: mysql+aiomysql://user:pass@host:port/db
DATABASE_URL = (
    f"mysql+aiomysql://{settings.MYSQL_USER}:{settings.MYSQL_PASSWORD}"
    f"@{settings.MYSQL_HOST}:{settings.MYSQL_PORT}/{settings.MYSQL_DATABASE}"
)

engine = create_async_engine(
    DATABASE_URL,
    echo=settings.DEBUG
)

async_session_factory = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False
)

class Base(DeclarativeBase):
    """所有 ORM 类型的基类"""
    pass

async def get_db():
    """FASTAPI 依赖注入：每次请求都创建一个数据库会话"""
    async with async_session_factory() as session:
        try:
            yield session
            await session.commit()

        except Exception:
            await session.rollback()
            raise

    finally:
        await session.close()
```

这段代码解决什么问题

搭好连接 MySQL 的“管道”，并提供一个 FastAPI 依赖注入函数 `get_db`，让每个请求都有独立、安全、自动提交/回滚的数据库会话。

关键代码逐句拆解

① DATABASE_URL — 连接串

拼出来是 `mysql+aiomysql://root:123456@localhost:3306/code_assistant`。协议 `mysql+aiomysql` 里的 **aiomysql** 是异步驱动，和后面的 `create_async_engine` 配套——整条链路是异步的。

为什么强调异步？FastAPI 的卖点是高并发。同步数据库调用会阻塞事件循环——一个请求在等数据库，其他几百个请求全部卡住。异步驱动让等待不阻塞其他请求。

② engine — 连接池

```
engine = create_async_engine(DATABASE_URL, echo=settings.DEBUG)
```

`engine` 是 SQLAlchemy 的连接池：预先维护一批到 MySQL 的连接，需要时取、用完还，避免每个请求新建 TCP 连接（很贵）。

`echo=settings.DEBUG`：DEBUG 模式下打印每一条执行的 SQL。开发时你能直接看到 ORM 翻译成了什么 SQL——理解 ORM 行为最好的方式。生产关掉 DEBUG 就不刷屏。

③ async_session_factory — 会话工厂

`engine` 管“连接”，`session` 管“一次业务操作”。`async_sessionmaker` 是会话工厂，每次 `async_session_factory()` 生成一个新会话。

`expire_on_commit=False` 是性能/坑位细节：默认 SQLAlchemy 在 `commit()` 后会让已加载对象“过期”，下次访问属性要重新查库；异步场景下过期对象在事务外访问会报错。设成 `False`，`commit` 后属性仍可用。

④ class Base(DeclarativeBase) — 所有表的祖宗

后面每个模型文件都 `from app.database import Base`，然后 `class User(Base): ...`。SQLAlchemy 收集所有继承 `Base` 的类，最终在 `main.py` 的 `Base.metadata.create_all` 统一建表。

⑤ get_db() — 依赖注入的核心

```
async def get_db():
    async with async_session_factory() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

```
finally:
    await session.close()
```

yield 是 FastAPI 依赖注入的关键，执行顺序：

请求到达 → 创建 session → yield 交给路由使用
→ 路由处理完 → commit() 保存修改
→ 路由抛异常 → rollback() 回滚并重新抛出
→ finally 里 close() 关闭会话

这个函数替路由干了三件脏活：**开会话、自动提交、出错回滚**。路由只管 session.add(...)，不用写 commit。异常路径保证数据库不会处于“改了一半”的状态——这就是事务的原子性。

面试题答题要点

Q：项目里怎么管理数据库连接？ 答：SQLAlchemy 异步引擎（create_async_engine + aiomysql）维护连接池；每次请求通过 FastAPI 依赖注入 get_db() 获取独立 AsyncSession，依赖注入自动 commit / rollback / close，路由代码不关心事务生命周期。

Q：为什么用异步数据库驱动？ 答：FastAPI 是异步框架，如果数据库调用是同步的，会阻塞事件循环，高并发下整个服务卡住。aiomysql 让数据库等待期间可以处理其他请求。

Q：expire_on_commit=False 是干嘛的？ 答：默认提交后对象会过期，异步环境下在事务外访问过期属性会报错；设为 False 后提交后仍能安全访问对象属性。

Q：Base.metadata.create_all 在哪个文件触发？ 答：main.py 的 lifespan 启动钩子里，async with engine.begin() as conn: await conn.run_sync(Base.metadata.create_all)，应用启动时自动建表。

动手做

```
# 启动依赖（仅 MySQL + Redis，不启后端）
docker compose up mysql redis

# 在后端目录里触发一次建表，观察 echo 打印的 SQL
cd backend
python -c "
import asyncio
from app.database import engine, Base
async def main():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    await engine.dispose()
asyncio.run(main())
"
```

第 1 层小结

三兄弟互相咬合：config.py 提供 settings → database.py 用它拼连接串；logger.py 提供 log；database.py 提供 engine / async_session_factory / Base / get_db。后面每一个文件都会引用它们。

第 2 层 数据模型：9 张表的仓库

第 1 层搭好了地基，这一层来建“货架”——数据要存成什么样子。这 9 张表是整个系统所有信息的载体：

表	存什么	谁在读
users	注册用户	鉴权、所有接口
conversations / messages	Agent 会话和消息	对话接口
agent_logs	Agent 每一步决策	可观测 / 排查
retrieval_logs	每次检索的完整链路	可观测
query_rewrites	查询改写记录	评估 / 调优
evaluation_runs	消融实验的指标	评估报告
index_versions	索引构建历史	索引管理
feedbacks	用户对回答的点赞/点踩	效果反馈

共通的“表设计语言”（读懂了这些，8 个文件基本是套路）：

1. 继承 **Base**: `class User(Base), from app.database import Base`。
2. `__tablename__`：显式指定表名。
3. 字段用 **Column(类型, 约束, comment=...)** 定义。
4. 每个表都有 **id** 主键: `Column(Integer, primary_key=True, autoincrement=True)`。
5. 几乎每个表都有 **created_at**: `default=lambda: datetime.datetime.now(UTC)`——注意是传函数引用而不是直接调 `now()`，这样每条记录插入时才取最新时间，而不是模块加载时的时间。这个细节面试常考。

开始逐个看。

2.1 models/init.py — 模型的汇总清单

完整代码

```
from app.models.conversation import Conversation, Message
from app.models.feedback import Feedback
```

```

from app.models.index_version import IndexVersion
from app.models.retrieval_log import RetrievalLog
from app.models.query_rewrite import QueryRewrite
from app.models.evaluation_run import EvaluationRun
from app.models.agent_log import AgentLog
from app.models.user import User

__all__ = ["Conversation", "Message", "Feedback", "IndexVersion", "RetrievalLog",
"QueryRewrite", "EvaluationRun", "AgentLog", "User"]

```

这段代码解决什么问题

把 8 个模型文件里的类汇总到一个入口。这样别处只需要 `from app.models import User`，不用记每个类在哪个文件。

它还有个隐藏但关键的作用：`main.py` 里 `Base.metadata.create_all` 建表时，SQLAlchemy 必须先知道有哪些模型。如果某个模型类从未被 `import`，它就不会注册到 `Base.metadata` 里，建表就会漏掉那张表。`__init__.py` 把 8 个模型全 `import` 一遍，确保建表时一个不漏。

`__all__` 是 Python 的“白名单”：`from app.models import *` 时只导入列出的名字；也给 IDE 提示了公开 API。

2.2 user.py — 用户表

完整代码

```

"""用户模型"""

import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Boolean, DateTime
from app.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(String(32), unique=True, nullable=False, index=True,
comment="用户名")
    hashed_password = Column(String(128), nullable=False, comment="bcrypt 哈希密码")
    role = Column(String(16), default="user", comment="角色: user / admin")
    is_active = Column(Boolean, default=True, comment="是否激活")
    created_at = Column(DateTime, default=lambda: datetime.datetime.now(UTC),
comment="创建时间")

```

关键字段逐个讲

字段	设计	为什么
<code>username</code>	<code>String(32) +</code>	用户名唯一；注册时查重靠它；加索引让按用

字段	设计	为什么
	<code>unique=True + index=True</code>	户名查登录飞快
<code>hashed_password</code>	<code>String(128)</code>	只存 bcrypt 哈希，绝不存明文。bcrypt 输出约 60 字符，给 128 留足空间
<code>role</code>	<code>String(16)</code> 默认 <code>"user"</code>	预留角色系统（user/admin），当前鉴权只用 <code>is_active</code> ，但字段留着为扩展
<code>is_active</code>	<code>Boolean</code> 默认 <code>True</code>	账号禁用开关， <code>get_current_user</code> 里检查 <code>not user.is_active</code> 就拒绝
<code>created_at</code>	<code>default=lambda: ... now(UTC)</code>	见第 2 层开头的“共通语言”

为什么用户名加索引？登录流程是 `select(User).where(User.username == req.username)`——没有索引就是全表扫描；有 `index=True` 走 B+ 树索引， $O(\log n)$ 。

2.3 conversation.py — 会话与消息表

完整代码

```
import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, DateTime, JSON, Text,
UniqueConstraint
from app.database import Base

class Conversation(Base):
    """一次对话 session"""
    __tablename__ = "conversations"
    __table_args__ = (
        # 同一用户 + 同一 session_id 唯一，防止并发创建重复会话
        UniqueConstraint("user_id", "session_id",
name="uq_conversation_user_session"),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, nullable=False, index=True, comment="所属用户 ID")
    session_id = Column(String(64), nullable=False, index=True, comment="会话 ID")
    created_at = Column(DateTime, default=datetime.datetime.utcnow, comment="创建时
间")
    updated_at = Column(
        DateTime,
        default=lambda: datetime.datetime.now(UTC),
        onupdate=lambda: datetime.datetime.now(UTC),
        comment="最后更新时间",
    )
```

```

class Message(Base):
    """对话中的单条消息"""
    __tablename__ = "messages"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, nullable=False, index=True, comment="所属用户 ID")
    session_id = Column(String(64), nullable=False, index=True, comment="所属会话 ID")
    role = Column(String(16), nullable=False, comment="角色: user / assistant")
    content = Column(Text, nullable=False, comment="消息内容")
    tool_calls = Column(JSON, nullable=True, comment="Agent 调用的工具信息 (JSON) ")
    created_at = Column(DateTime, default=lambda: datetime.datetime.now(UTC),
comment="发送时间")

```

关键设计逐个讲

① 为什么 **conversations** 和 **messages** 两张表?

一个会话 (Conversation) 包含多条消息 (Message)，这是一对多关系。拆两张表的好处：会话只存元信息（创建时间、更新时间），消息存内容；查“有哪些会话”不用扫描所有消息；一个会话上千条消息也能轻松加载最近 N 条。

② **UniqueConstraint("user_id", "session_id")** — 数据库层兜底

```

__table_args__ = (
    UniqueConstraint("user_id", "session_id", name="uq_conversation_user_session"),
)

```

同一用户 + 同一 session 只允许一条会话记录。即使应用层并发创建重复会话（get_or_create_conversation 里先查后建存在竞态），数据库也会拒绝第二条，从根上防重复。

③ **user_id** 和 **session_id** 都加索引

每次查历史 where(user_id=?, session_id=?), 两个索引让查询高效。

④ **updated_at** 用 **onupdate**

```
onupdate=lambda: datetime.datetime.now(UTC)
```

onupdate 在记录被 UPDATE 时自动刷新时间，配合 default（插入时）。很多系统喜欢把“最后活跃时间”放会话表，方便展示/清理。

⑤ **Message.tool_calls** 是 JSON 字段

Column(JSON) 存 Agent 调用了什么工具。虽然当前 conversation_service 还没往里写数据，但字段预留了，说明设计者想到“消息里可能带工具调用详情”。

⑥ **Message.role** 只有 **user / assistant**

注意：这个系统把 Agent 内部的工具调用过程放在 agent_logs 表，而 messages 只存最终问答。所以 role 两个值够用。

2.4 agent_log.py — Agent 决策日志

完整代码

```
"""Agent 决策日志：记录每次 Agent 运行的每步决策链路"""

import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Text, DateTime
from app.database import Base

class AgentLog(Base):
    __tablename__ = "agent_logs"

    id = Column(Integer, primary_key=True, autoincrement=True)
    session_id = Column(String(64), nullable=False, index=True, comment="会话 ID")
    step_number = Column(Integer, nullable=False, comment="第几步")
    thought = Column(Text, nullable=True, comment="LLM 的思考过程")
    action_name = Column(String(32), nullable=True, comment="调用的工具名")
    action_args = Column(Text, nullable=True, comment="工具参数 (JSON)")
    observation = Column(Text, nullable=True, comment="工具返回结果摘要")
    created_at = Column(
        DateTime, default=lambda: datetime.datetime.now(UTC), comment="创建时间"
    )
```

这段代码解决什么问题

这是“**Agent 的黑匣子**”。ReAct 循环每走一步，harness.py 的 `_log_step` 就往这张表插一条：LLM 想了什么（`thought`）、调了哪个工具（`action_name`）、参数是什么（`action_args`）、工具返回了什么（`observation`）。

对应 README 里的“三层可观测性”之一。出问题，你能把一次 Agent 运行**逐步回放**：它在第 2 步搜了 `JSONStorage`，观察结果如何，为什么第 4 步决定给答案。这种能力在面试讲“可观测性”时非常好用。

`step_number` 记录第几步；`thought` 存 LLM 的完整思考文本（harness.py 里截断到 500 字符）；`observation` 截断到 200 字符——防止日志表无限膨胀。

2.5 retrieval_log.py — 检索日志

完整代码

```
import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Text, DateTime, JSON, Float
from app.database import Base
```

```
class RetrievalLog(Base):
    """检索日志：记录每次 hybrid_search 的完整链路"""
    __tablename__ = "retrieval_logs"

    id = Column(Integer, primary_key=True, autoincrement=True)
    query_text = Column(Text, nullable=False, comment="用户原始查询")
    dense_top_k = Column(JSON, nullable=True, comment="Dense 路 top-k 结果")
    sparse_top_k = Column(JSON, nullable=True, comment="Sparse 路 top-k 结果")
    fused_top_n = Column(JSON, nullable=True, comment="RRF 融合后 top-n 结果")
    strategy = Column(String(32), default="hybrid", comment="检索策略")
    total_latency_ms = Column(Float, nullable=True, comment="总耗时（毫秒）")
    created_at = Column(
        DateTime, default=lambda: datetime.datetime.now(UTC), comment="创建时间"
    )
```

这段代码解决什么问题

RAG 的“黑匣子”。fusion.py 的 async_hybrid_search 每次跑完混合检索，就把三路中间结果原样 JSON 化存进来：

- query_text: 用户原始查询
- dense_top_k: 稠密路返回的前几条
- sparse_top_k: 稀疏路返回的前几条
- fused_top_n: RRF 融合后的最终结果
- strategy: 用的什么策略（hybrid 等）
- total_latency_ms: 链路总耗时

价值：你可以回看任何一次检索——稠密路和稀疏路各返回了什么、融合后剩下什么、花了多少毫秒。分析“为什么某次检索结果不好”时，这表就是现场录像。JSON 列能直接存结构化列表，查询时原样取回。

2.6 query_rewrite.py — 查询改写记录

完整代码

```
import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Text, DateTime
from app.database import Base

class QueryRewrite(Base):
    """查询改写记录：存原始查询、改写后查询、改写策略"""
    __tablename__ = "query_rewrites"

    id = Column(Integer, primary_key=True, autoincrement=True)
    original_query = Column(Text, nullable=False, comment="原始查询")
    rewritten_query = Column(Text, nullable=False, comment="改写后查询")
    strategy = Column(String(16), nullable=False, comment="改写策略: hyde / expand")
    created_at = Column(
        DateTime, default=lambda: datetime.datetime.now(UTC), comment="创建时间"
    )
```

这段代码解决什么问题

query_rewriter.py 用 HyDE / expand 改写查询时，把“原查询 → 改写后查询 → 策略”落库。
目的：积累改写的真实样本。以后你可以分析：哪些查询被改写了、改成了什么、HyDE 生成的假设文档质量如何。这也是评估 HyDE 组件是否有用的数据基础。

注意 strategy 字段没有默认值、nullable=False——强制要求写记录时必须标明策略，否则数据不完整。

2.7 evaluation_run.py — 评估运行记录

完整代码

```
import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Text, Float, JSON, DateTime
from app.database import Base

class EvaluationRun(Base):
    __tablename__ = "evaluation_runs"

    id = Column(Integer, primary_key=True, autoincrement=True)
    run_name = Column(String(128), nullable=False, comment="评估名称，如 'baseline_no_rerank'")
    description = Column(Text, nullable=True, comment="评估描述")
    config = Column(JSON, nullable=False, comment="评估配置：哪些组件开启")
    test_set_size = Column(Integer, nullable=False, comment="测试集查询数量")
    hit_rate = Column(Float, nullable=True, comment="Hit Rate")
    mrr = Column(Float, nullable=True, comment="Mean Reciprocal Rank")
    ndcg = Column(Float, nullable=True, comment="NDCG@k")
    avg_latency_ms = Column(Float, nullable=True, comment="平均检索延迟")
    created_at = Column(
        DateTime, default=lambda: datetime.datetime.now(UTC), comment="创建时间"
    )
```

这段代码解决什么问题

消融实验（evaluation.py 的 run_ablation）跑完后，把每组实验的指标写进这张表。字段设计很有讲究：

- run_name: ablation_v1_dense_only 这种唯一标识。
- config: JSON 字段存“这组实验开了哪些组件”（{"dense": true, "sparse": false, ...}）——这样表格不用为每个组件建一列，新组件加进 config 即可，schema 不变。这是 JSON 字段的典型用法。
- hit_rate / mrr / ndcg / avg_latency_ms: 三个质量指标 + 一个性能指标。
- test_set_size: 测试集条数，便于横向比较不同规模的结果。

价值：README 里那张消融表就是从这里导出的。面试时说“我的每个组件都有评估数据支撑”，指的就是这条链路。

2.8 index_version.py — 索引版本记录

完整代码

```
import datetime
from datetime import UTC
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from app.database import Base

class IndexVersion(Base):
    """索引版本记录：每次重建索引时插入一条"""
    __tablename__ = "index_versions"

    id = Column(Integer, primary_key=True, autoincrement=True)
    strategy = Column(String(32), comment="分块策略")
    chunk_size = Column(Integer, comment="块大小")
    chunk_overlap = Column(Integer, comment="块重叠")
    file_count = Column(Integer, comment="文件数")
    chunk_count = Column(Integer, comment="块数")
    build_duration_ms = Column(Integer, nullable=True, comment="构建耗时（毫秒）")
    created_at = Column(
        DateTime, default=lambda: datetime.datetime.now(UTC), comment="创建时间"
    )
```

这段代码解决什么问题

每次重建索引（code_indexer.py 的 save_version_record）记录一条：用了什么分块策略、多大、多少个文件、多少个 chunk、耗时多少。

为什么有用？ 当你换了分块策略（比如从 recursive 换到 semantic），想对比“哪个策略检索效果更好”，得有据可查：当时的索引是用什么参数建的。这是索引的可追溯性——配合 evaluation_runs 就能做“索引配置 × 检索指标”的交叉分析。

注意这个文件顶部 import 了 ForeignKey 但没用——属于遗留 import，不影响功能，但面试时被问到可以大方指出这是可以清理的小瑕疵（诚实 + 细心）。

2.9 feedback.py — 用户反馈

完整代码

```
import datetime
from datetime import UTC

from sqlalchemy import Column, Integer, String, Text, DateTime
from app.database import Base

class Feedback(Base):
    """用户对 Agent 回答的反馈(点赞/点踩)"""
    __tablename__ = "feedbacks"

    id = Column(Integer, primary_key=True, autoincrement=True)
    message_id = Column(Integer, nullable=False, index=True, comment="对应消息 ID")
    session_id = Column(String(64), nullable=False, comment="所属会话 ID")
```

```
rating = Column(Integer, nullable=False, comment="评分: 1=点赞, -1=点踩")
comment = Column(Text, nullable=True, comment="用户可选评价文本")
created_at = Column(DateTime, default=lambda: datetime.datetime.now(UTC),
comment="创建时间")
```

这段代码解决什么问题

收集用户对 Agent 回答质量的反馈（点赞/点踩）。`rating` 用整数：1 点赞、-1 点踩。`message_id` 关联到具体那条消息（虽然当前没建外键约束，但字段语义明确）。`comment` 是可选补充说明。

注意 `rating` 用整数而不是布尔：为未来扩展留了余地（比如支持 1-5 星）。这个“用更宽的字段类型为扩展留空间”的思路值得学习。

第 2 层小结

9 张表（`conversation.py` 里定义了两张：Conversation 和 Message）可以分成三组记忆：

- **业务数据：**users、conversations、messages、feedbacks——系统的“正业”。
- **可观测数据：**agent_logs、retrieval_logs、query_rewrites——Agent 和检索的“黑匣子”。
- **工程数据：**evaluation_runs、index_versions——评估和索引的“台账”。

共同点：都有 `id` 主键、`created_at` 时间戳，都继承 `Base`。理解“每个表为什么存在”比背字段更重要——面试官问“你项目里有哪些表”，你要能说出**每张表服务的业务或可观测性目的**。

第 3 层 安全：鉴权与限流

地基和数据模型都齐了。但整栋楼现在是“裸奔”的——谁都能进来。第 3 层负责**装门禁**：

- `auth.py` —— 大门：你是谁？（JWT 鉴权 + bcrypt 密码哈希 + Redis 黑名单登出）
- `middleware.py` —— 闸机：你刷太快了？（Redis 滑动窗口限流）

这一层是面试含金量最高的部分之一，因为“安全”是后端岗位最看重的工程能力。

3.1 auth.py — JWT 鉴权全家桶

完整代码

```
"""JWT 鉴权：创建/验证 token、密码哈希、依赖注入"""

import datetime
from datetime import UTC, timedelta
from typing import Optional

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
```

```

from jose import JWTError, jwt
import bcrypt
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

from app.config import settings
from app.database import get_db
from app.models.user import User

security = HTTPBearer()

# JWT 配置直接从 settings 读取, 或硬编码默认值
SECRET_KEY = settings.JWT_SECRET
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 小时
REFRESH_TOKEN_EXPIRE_DAYS = 7

def hash_password(password: str) -> str:
    """bcrypt 哈希密码 (原生 API) """
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """验证密码"""
    try:
        return bcrypt.checkpw(
            plain_password.encode("utf-8"),
            hashed_password.encode("utf-8"),
        )
    except ValueError:
        return False

def create_access_token(data: dict) -> str:
    """创建访问 token (短期) """

    to_encode = data.copy()
    expire = datetime.datetime.now(UTC) +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire, "type": "access"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(data: dict) -> str:
    """创建刷新 token (长期) """

    to_encode = data.copy()
    expire = datetime.datetime.now(UTC) + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode.update({"exp": expire, "type": "refresh"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> dict:
    """解码 token, 失败抛 401"""

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expire token",
        )

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db:AsyncSession = Depends(get_db),
) -> User:

```



```

"""FastAPI 依赖注入：从请求头获取当前登录用户"""
if is_token_blacklisted(credentials.credentials):
    raise HTTPException(status_code=401, detail="Token has been revoked")
payload = decode_token(credentials.credentials)
username = payload.get("sub")
if not username:
    raise HTTPException(status_code=401, detail="Invalid token")

result = await db.execute(select(User).where(User.username == username))
user = result.scalar_one_or_none()
if not user or not user.is_active:
    raise HTTPException(status_code=401, detail="User not found or inactive")
return user

import redis as redis_lib

def _get_redis():
    return redis_lib.Redis(
        host=settings.REDIS_HOST,
        port=settings.REDIS_PORT,
        db=0,
        decode_responses=True,
        protocol=2,
    )

def blacklist_token(token: str, expire_seconds: int):
    """把 token 加入 Redis 黑名单"""
    try:
        r = _get_redis()
        r.setex(f"blacklist:{token}", expire_seconds, "1")
    except Exception:
        pass

def is_token_blacklisted(token: str) -> bool:
    """检查 token 是否在黑名单中"""
    try:
        r = _get_redis()
        return bool(r.exists(f"blacklist:{token}"))
    except Exception:
        return False

```

这段代码解决什么问题

“怎么证明你是你？”——这是鉴权的核心问题。方案是 **JWT**：用户登录成功后，服务端签发一个签名过的 **token**，客户端后续每次请求带上它，服务端验签即信。关键点：**JWT 是无状态的**——服务端不存 session，只靠签名判断真伪。

整体流程：

注册： 明文密码 ——bcrypt 哈希——> 存 hashed_password
 登录： 输入密码 ——bcrypt 校验——> 通过则签发 access_token + refresh_token
 请求： 每次带 Bearer token ——> get_current_user 解析出用户
 登出： 把 token 塞进 Redis 黑名单，剩余有效期自动过期

关键代码逐句拆解

① **security = HTTPBearer()** — 声明“我要从请求头拿 token”

```
security = HTTPBearer()
```

FastAPI 的 HTTPBearer 是一个安全方案：自动从请求头 `Authorization: Bearer <token>` 里解析出 token。配合 `get_current_user` 的参数 `credentials: HTTPAuthorizationCredentials = Depends(security)`，FastAPI 会在路由依赖里自动解析。请求没带 token 或格式不对，直接 401。

② bcrypt 哈希 (`hash_password / verify_password`)

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    try:
        return bcrypt.checkpw(plain_password.encode("utf-8"),
                               hashed_password.encode("utf-8"))
    except ValueError:
        return False
```

- **密码绝不存明文。** bcrypt 自带随机盐 (`bcrypt.gensalt()`)，同一个密码每次哈希结果都不同。
- `verify_password` 用 `try/except ValueError` 兜底：如果数据库里存的是 malformed 哈希（比如被截断），`checkpw` 会抛 `ValueError`，捕获后返回 `False`，而不是让服务崩溃。
- 返回 `.decode("utf-8")`：`hashpw` 返回 bytes，存数据库要转成 str。

③ 双 token 设计

```
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 小时
REFRESH_TOKEN_EXPIRE_DAYS = 7
```

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.datetime.now(UTC) +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire, "type": "access"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.datetime.now(UTC) + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode.update({"exp": expire, "type": "refresh"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

- **access token** 短期（24h），用于正常请求鉴权。泄露了损失有限。
- **refresh token** 长期（7 天），只在 `/refresh` 接口用，用于换取新的 access token。
- 两个 token 都往 payload 里塞了 **type 字段**：`access / refresh`。这样 `/refresh` 接口能校验“你给我的真的是 refresh token，不是拿 access token 来冒充”（见 `auth_router.py`）。
- **`to_encode = data.copy()`**：防止修改调用方传入的 dict（`jwt.encode` 内部可能修改传入对象）。
- 用 `datetime.now(UTC)`（带时区）而不是 `datetime.utcnow()`（已废弃），避免时区歧义。

④ decode_token — 验签 + 失败抛 401

```
def decode_token(token: str) -> dict:
    try:
```

```

        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid
or expire token")

```

jose (python-jose) 的 `jwt.decode` 会同时验证签名和过期时间。任何失败（签名不符、过期、格式错误）统一抛 `JWTError`，这里转成 401。注意 `algorithms=[ALGORITHM]` 显式指定算法——防止算法混淆攻击（不指定时某些库会接受 `none` 算法）。

⑤ `get_current_user` — 依赖注入的鉴权枢纽

```

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: AsyncSession = Depends(get_db),
) -> User:
    if is_token_blacklisted(credentials.credentials):
        raise HTTPException(status_code=401, detail="Token has been revoked")
    payload = decode_token(credentials.credentials)
    username = payload.get("sub")
    if not username:
        raise HTTPException(status_code=401, detail="Invalid token")

    result = await db.execute(select(User).where(User.username == username))
    user = result.scalar_one_or_none()
    if not user or not user.is_active:
        raise HTTPException(status_code=401, detail="User not found or inactive")
    return user

```

四道检查，层层过滤：

1. **Redis 黑名单**：先查 token 是否被登出过。
2. **验签 + 过期**：`decode_token`。
3. **payload 里有没有 `sub`**（用户名）——`sub` 是 JWT 标准声明，这里是用户名。
4. **用户存在且激活**：查库确认，同时检查 `is_active`。

这是 FastAPI 依赖注入的典型用法：任何路由只要声明 `current_user: User = Depends(get_current_user)`，就自动完成全部鉴权，路由内部拿到的 `current_user` 就是数据库里那个 `User` 对象。

⑥ Redis 黑名单（登出能力）

```

def blacklist_token(token: str, expire_seconds: int):
    try:
        r = _get_redis()
        r.setex(f"blacklist:{token}", expire_seconds, "1")
    except Exception:
        pass

def is_token_blacklisted(token: str) -> bool:
    try:
        r = _get_redis()
        return bool(r.exists(f"blacklist:{token}"))
    except Exception:
        return False

```

JWT 无状态 = 签发后服务端不知道它“死没死”。要支持登出，必须主动记住“哪些 token 作废了”。

- `blacklist_token`: 把 token 写进 Redis, key 是 `blacklist:<token>`, value "1", **TTL = token 剩余有效期**。到点自动删除, 不占空间。
- `is_token_blacklisted`: 查 Redis 里有没有这个 key。
- `try/except Exception: pass`: Redis 不可用时**降级**——黑名单检查失败就当作“没被拉黑”, 保证核心功能不因缓存故障而瘫。这是“可用性优先于强一致”的权衡。

`_get_redis` 里注意 `protocol=2`: 指定 Redis 协议版本为 2, 兼容更老的 Redis 服务端。

面试题要点

Q: JWT 和传统 session 有什么区别? 答: session 是服务端存状态、客户端存 `session_id`; JWT 是无状态的, 服务端只靠签名验真伪。JWT 适合分布式/微服务(每个节点验签即可, 不用共享 session 存储), 代价是无法主动吊销(所以要配黑名单)。

Q: 为什么用双 token? 答: access token 短命(24h), 即使泄露影响有限; refresh token 长命(7 天)只用于换新 access。避免用户频繁登录, 又控制泄露风险。refresh token 必须校验 `type == "refresh"`, 防止 access token 冒充。

Q: JWT 怎么登出? 答: JWT 无状态, 登出时把 token 加入 Redis 黑名单, TTL 设为 token 剩余有效期; 每次鉴权先查黑名单。Redis 自动过期, 不用手动清理。

Q: 密码为什么用 bcrypt 而不是 MD5/SHA? 答: bcrypt 自带盐 + 计算慢(抗暴力破解)。MD5/SHA 是快速哈希, 撞库成本低, 且无盐时相同密码哈希相同。bcrypt 每次加随机盐, 相同密码哈希也不同。

Q: Redis 挂了怎么办? 答: 黑名单检查失败就降级为“未拉黑”, 保证核心功能可用。这是可用性优先的取舍——牺牲登出的强一致, 换取服务不因缓存故障而瘫。

动手做

```
# 用 python 直接体验 JWT 生成与校验
cd backend
python -c "
from app.auth import create_access_token, decode_token
tok = create_access_token({'sub': 'hong'})
print('token:', tok[:40], '...')
print('payload:', decode_token(tok))
"
```

3.2 middleware.py — Redis 滑动窗口限流

完整代码

```
"""Redis 滑动窗口限流中间件 (基于 ZSET)"""

import time
import uuid

from fastapi import Request
from starlette.responses import JSONResponse
```

```
from starlette.middleware.base import BaseHTTPMiddleware
from app.config import settings
from app.logger import log
```

```
class RateLimitMiddleware(BaseHTTPMiddleware):
```

```
    """滑动窗口限流：每客户端在 window_seconds 内最多 rate_limit 次请求
```

```
    用 Redis ZSET 实现真正的滑动窗口：每个请求时间戳作为一个 member，
    每次检查时删除窗口外的旧记录，统计窗口内的数量。
```

```
    相比固定窗口 (time()//window)，滑动窗口不会在窗口边界产生突刺。
```

```
    """
```

```
    def __init__(self, app, rate_limit: int = 60, window_seconds: int = 60):
```

```
        super().__init__(app)
```

```
        self.rate_limit = rate_limit
```

```
        self.window_seconds = window_seconds
```

```
        self.redis = None
```

```
        try:
```

```
            import redis as redis_lib
```

```
            self.redis = redis_lib.Redis(
```

```
                host=settings.REDIS_HOST,
```

```
                port=settings.REDIS_PORT,
```

```
                db=0,
```

```
                decode_responses=True,
```

```
                protocol=2,
```

```
            )
```

```
            self.redis.ping()
```

```
        except Exception as e:
```

```
            log.warning("Redis unavailable, rate limiting disabled: %s", e)
```

```
    def _client_key(self, request: Request) -> str:
```

```
        """识别客户端：优先从 JWT 解析用户，否则退回 IP。
```

```
        注意：中间件在鉴权依赖 (get_current_user) 之前运行，拿不到
```

```
        request.state.user，所以这里自己解析 Authorization 头。
```

```
        """
```

```
        auth = request.headers.get("Authorization", "")
```

```
        if auth.startswith("Bearer "):
```

```
            token = auth[len("Bearer "):]
```

```
            try:
```

```
                from app.auth import decode_token
```

```
                payload = decode_token(token)
```

```
                if payload.get("type") == "access":
```

```
                    return f"user:{payload.get('sub', 'unknown')}}"
```

```
            except Exception:
```

```
                # token 无效或过期，退回 IP 限流
```

```
                pass
```

```
        return f"ip:{request.client.host}"
```

```
    async def dispatch(self, request: Request, call_next):
```

```
        if self.redis is None:
```

```
            return await call_next(request)
```

```
        # 只对 API 路径限流，跳过静态资源和文档
```

```
        if not request.url.path.startswith("/api/"):
            return await call_next(request)
```

```
        client = self._client_key(request)
```

```
        key = f"ratelimit:{client}"
```

```
        try:
```

```
            now = time.time()
```

```
            window_start = now - self.window_seconds
```

```
            pipeline = self.redis.pipeline()
```

```
            pipeline.zremrangebyscore(key, 0, window_start) # 删窗口外旧记录
```

```

        pipeline.zadd(key, {uuid.uuid4().hex: now})          # 加当前请求
        pipeline.zcard(key)                                  # 统计窗口内数量
        pipeline.expire(key, self.window_seconds + 1)
        _, _, count, _ = pipeline.execute()

        if count > self.rate_limit:
            return JSONResponse(
                status_code=429,
                content={"detail": "Rate limit exceeded"},
            )
    except Exception as e:
        log.warning("Rate limit check failed: %s", e)

    return await call_next(request)

```

这段代码解决什么问题

防止有人**疯狂刷接口**（暴力破解密码、爬虫、恶意打爆服务）。限制：每个客户端在 60 秒窗口内最多 60 次请求，超过就返回 429。

为什么要“滑动窗口”而不是“固定窗口”？固定窗口实现是 `time() // 60` 取分钟桶。问题：59.9 秒的请求和 60.1 秒的请求只差 0.2 秒，却被分到两个不同窗口，各能发 60 次——**实际 0.2 秒内可以发 120 次**，窗口边界产生“突刺”。滑动窗口没有这个漏洞。

关键代码逐句拆解

① BaseHTTPMiddleware + dispatch — Starlette 中间件接口

```

class RateLimitMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, rate_limit=60, window_seconds=60):
        super().__init__(app)
        ...
    async def dispatch(self, request: Request, call_next):
        ...

```

`dispatch` 是中间件的入口：它在每个请求进入路由**之前**被调用。你可以在 `call_next(request)` 前后做文章——`call_next` 才是真正把请求交给路由。这里就是“先限流，超了就拦住，不超才放行”。

② 客户端识别 `_client_key` — 关键设计

```

def _client_key(self, request: Request) -> str:
    auth = request.headers.get("Authorization", "")
    if auth.startswith("Bearer "):
        token = auth[len("Bearer "):]
        try:
            from app.auth import decode_token
            payload = decode_token(token)
            if payload.get("type") == "access":
                return f"user:{payload.get('sub', 'unknown')}"
        except Exception:
            pass
    return f"ip:{request.client.host}"

```

限流按“人”还是按“IP”？按 IP 有个问题：公司里很多人共享一个出口 IP，一个被限、全公司都 429。这个实现**优先按登录用户限流**（`user:<username>`），没登录才退回 IP。

注意中间件执行时机：它在鉴权依赖 `get_current_user` 之前运行，所以 `request.state.user` 还没有。作者自己解析 `Authorization` 头 + `decode_token`。这是个容易踩的坑，代码注释里也写明了。

还有一个细节：只对 `type == "access"` 的 token 走用户限流——refresh token 不在请求头上当访问凭证，不参与。

③ Redis ZSET 滑动窗口（核心算法）

```
now = time.time()
window_start = now - self.window_seconds
pipeline = self.redis.pipeline()
pipeline.zremrangebyscore(key, 0, window_start) # ① 删掉窗口外的旧记录
pipeline.zadd(key, {uuid.uuid4().hex: now}) # ② 当前请求时间戳入 ZSET
pipeline.zcard(key) # ③ 数窗口内有几个请求
pipeline.expire(key, self.window_seconds + 1)
_, _, count, _ = pipeline.execute()

if count > self.rate_limit:
    return JSONResponse(status_code=429, content={"detail": "Rate limit exceeded"})
```

ZSET（有序集合） 每个元素是 (member, score)，按 score 排序。这里：- **score** = 请求到达的时间戳 (`time.time()`) - **member** = 随机 **UUID**（保证每个请求唯一）

四步操作：1. `zremrangebyscore(key, 0, window_start)`：删除 score 在 `[0, window_start]` 的记录——也就是窗口外过期的旧请求。2. `zadd(key, {uuid: now})`：把当前请求加进集合。3. `zcard(key)`：数集合里有多少个——就是当前窗口内活着的请求数。4. `expire(key, window_seconds + 1)`：给 key 设过期，防止长期不活跃的 key 占用 Redis。

`count > rate_limit` 就返回 429。

为什么要用 pipeline？ 这 4 个操作如果逐个发 Redis，要 4 次网络往返；pipeline 把 4 条命令打包一次发送，Redis 依次执行，一次返回结果。**显著降低网络开销**——限流逻辑要加在每个请求上，性能不能差。

④ Redis 不可用时优雅降级

```
self.redis = None
try:
    ...
    self.redis.ping()
except Exception as e:
    log.warning("Redis unavailable, rate limiting disabled: %s", e)

async def dispatch(self, request: Request, call_next):
    if self.redis is None:
        return await call_next(request)
```

Redis 连不上就**禁用限流**而不是拒绝所有请求。这与 `auth.py` 里的 `except Exception: pass` 是同一个哲学：**限流是保护措施，不能让保护措施本身把服务搞挂。**

⑤ 只对 /api/ 限流

```
if not request.url.path.startswith("/api/"):
    ...
```

```
return await call_next(request)
```

静态资源、/docs 文档不参与限流，避免误伤开发者调试。

面试题要点

Q：固定窗口和滑动窗口的区别？ 答：固定窗口按 `time()//window` 切桶，窗口边界会有突刺（59.9s 和 60.1s 的请求被算进两个窗口，各能满额）。滑动窗口基于 Redis ZSET 记录每个请求时间戳，每次动态删除窗口外记录再计数，边界连续、无突刺。

Q：滑动窗口用 ZSET 怎么实现？ 答：每个请求时间戳作为 score、UUID 作为 member 存入 ZSET；每次请求先 `zremrangebyscore` 删窗口外旧记录，再 `zadd` 当前请求，`zcard` 统计数量，超过阈值返回 429。用 pipeline 一次发送多条命令降低延迟。

Q：按用户限流还是按 IP 限流？ 答：两者结合。优先按登录用户（`user:<username>`），未登录退回 IP。因为中间件运行在鉴权之前，拿不到 `request.state.user`，所以自己解析 Authorization 头。

Q：Redis 挂了怎么办？ 答：限流中间件初始化失败时置 `redis=None`，dispatch 里直接放行，保证服务可用。限流是保护措施，不能成为单点故障。

Q：为什么用 Redis 而不是本地内存？ 答：多实例部署时本地内存各自独立，限流会失效；Redis 是共享存储，所有实例看到同一份计数。且 ZSET 的滑动窗口逻辑天然适合 Redis 实现。

动手做

```
# 手动体验 ZSET 滑动窗口
redis-cli
# 然后依次执行：
# ZADD ratelimit:demo 100 a 200 b 300 c
# ZCARD ratelimit:demo
# ZREMRANGEBYSCORE ratelimit:demo 0 150
# ZCARD ratelimit:demo    # 看到 2 (b、c 还在)
```

第 3 层小结

`auth.py` 管“你是谁”（身份），`middleware.py` 管“你刷多快”（频率）。两个文件都体现了降级优先的设计：Redis 挂了，鉴权黑名单降级为不检查、限流降级为不禁用。这个“保护措施不能成为单点故障”的思路，是生产级代码的标志，面试官会喜欢。

第 4 层 RAG 检索管线

第 4 层是项目的技术核心，也是面试里最能展示深度的地方。RAG（Retrieval-Augmented Generation，检索增强生成）的完整链路是：

离线阶段：建索引

源码 —> chunker 分块 —> 稠密向量存 Chroma + BM25 索引存 Redis
在线阶段：查索引
问题 —> 改写 (HyDE, 可选) —> 双路召回 —> RRF 融合 —> 重排 (可选) —> 结果
评估阶段：验证
20 条测试集 —> 5 组消融实验 —> Hit Rate / MRR / NDCG 指标

这一层有 10 个文件，我按“在线链路 → 离线编排 → 评估”的顺序讲：

- 在线链路（用户查询时跑）：chunker → dense_retriever → sparse_retriever → fusion → query_rewriter → reranker
- 离线编排：code_indexer、user_upload
- 评估：test_set、evaluation

4.1 chunker.py — 代码分块器

完整代码

```
from typing import Any

from langchain_text_splitters import RecursiveCharacterTextSplitter,
TokenTextSplitter

from app.config import settings

class CodeChunker:
    """代码分块器，根据 strategy 配置选择不同分块策略"""

    def __init__(
        self,
        strategy: str = "recursive",
        chunk_size: int = 500,
        chunk_overlap: int = 50,
    ):
        self.strategy = strategy
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

    def chunk(self, documents: list[dict[str, Any]]) -> list[dict[str, Any]]:
        if self.strategy == "recursive":
            return self._recursive_chunk(documents)
        elif self.strategy == "token":
            return self._token_chunk(documents)
        elif self.strategy == "semantic":
            return self._semantic_chunk(documents)
        else:
            raise ValueError(f"Unknown chunk strategy: {self.strategy}")

    def _recursive_chunk(self, documents: list[dict]) -> list[dict]:
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=self.chunk_size,
            chunk_overlap=self.chunk_overlap,
            separators=["\n\n", "\n", "def ", "class ", " ", " ", " "],
        )
        return self._apply_splitter(splitter, documents)

    def _token_chunk(self, documents: list[dict]) -> list[dict]:
        splitter = TokenTextSplitter(
            chunk_size=self.chunk_size // 4,
```

```

        chunk_overlap=self.chunk_overlap // 4,
    )
    return self._apply_splitter(splitter, documents)

def _semantic_chunk(self, documents: list[dict]) -> list[dict]:
    chunks = []
    for doc in documents:
        lines = doc["content"].split("\n")
        current_chunk_lines: list[str] = []
        for line in lines:
            if (line.startswith("def ") or line.startswith("class ")) and
current_chunk_lines:
                indent = len(line) - len(line.lstrip())
                if indent == 0:
                    text = "\n".join(current_chunk_lines)
                    if text.strip():
                        chunks.append({
                            "text": text,
                            "metadata": {
                                "source": doc["path"],
                                "chunk_index": len(chunks),
                            },
                        })
                    current_chunk_lines = []
                current_chunk_lines.append(line)
        if current_chunk_lines:
            text = "\n".join(current_chunk_lines)
            if text.strip():
                chunks.append({
                    "text": text,
                    "metadata": {
                        "source": doc["path"],
                        "chunk_index": len(chunks),
                    },
                })
    return chunks

def _apply_splitter(self, splitter, documents: list[dict]) -> list[dict]:
    chunks = []
    for doc in documents:
        texts = splitter.split_text(doc["content"])
        for i, text in enumerate(texts):
            chunks.append({
                "text": text,
                "metadata": {
                    "source": doc["path"],
                    "chunk_index": i,
                },
            })
    return chunks

def info(self) -> dict:
    return {
        "strategy": self.strategy,
        "chunk_size": self.chunk_size,
        "chunk_overlap": self.chunk_overlap,
    }

```

这段代码解决什么问题

Embedding 模型有输入长度限制，而且“一个函数”比“整个文件”更适合精确检索。所以**索引前必须把大文件切成小块**。这个类根据配置 `CHUNK_STRATEGY` 切换三种分块策略。

关键代码逐句拆解

① 入口 `chunk()` — 策略路由

```
def chunk(self, documents):
    if self.strategy == "recursive":
        return self._recursive_chunk(documents)
    elif self.strategy == "token":
        return self._token_chunk(documents)
    elif self.strategy == "semantic":
        return self._semantic_chunk(documents)
    else:
        raise ValueError(f"Unknown chunk strategy: {self.strategy}")
```

`documents` 是 `[{path, content}]` 结构。未知策略直接抛 `ValueError` (fail fast)。

② 统一输出格式: `{text, metadata}`

所有策略最后都输出统一结构:

```
{
    "text": "代码片段内容",
    "metadata": {
        "source": "tinydb/storages.py",    # 来源文件
        "chunk_index": 0,                  # 第几个块
    },
}
```

metadata 是检索结果的“身份证”——后面融合、去重、定位文件全靠 `source + chunk_index` 这一对 key。

③ **recursive** 策略 — 通用默认

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=self.chunk_size,
    chunk_overlap=self.chunk_overlap,
    separators=["\n\n", "\n", "def ", "class ", " ", " ", " "],
)
```

`RecursiveCharacterTextSplitter` 按优先级依次尝试分隔符: 段落 → 换行 → **def/class** → 缩进 → 空格。如果一段太长, 就再往下拆。`chunk_overlap=50` 让相邻块有 50 字符重叠——避免在边界处切断一个函数名/语句。

④ **token** 策略

```
splitter = TokenTextSplitter(
    chunk_size=self.chunk_size // 4,
    chunk_overlap=self.chunk_overlap // 4,
)
```

按 token 数切。`//4` 是估算: 英文大概 4 个字符 $\approx$ 1 个 token, 所以 500 字符 $\approx$ 125 token。

⑤ **semantic** 策略 — 手写按函数边界切 (亮点)

```
for line in lines:
    if (line.startswith("def ") or line.startswith("class ")) and
```

```
current_chunk_lines:
    indent = len(line) - len(line.lstrip())
    if indent == 0:    # 只有顶格定义才切，类里的方法不切
        ...把 current_chunk_lines 存成一块...
current_chunk_lines.append(line)
```

逻辑：逐行扫描，遇到**顶格**（`indent == 0`）的 `def/class` 就认为上一个函数/类结束，把累积的行存成一块。

为什么只切顶格？ 因为类里的方法（有缩进的 `def`）应该留在类里，不单独成块——不然一个类被拆得七零八落，检索到方法却丢了类的上下文。`indent == 0` 这个判断很精准。

⑥ `info()`

返回分块配置（`strategy / size / overlap`），用于记录索引版本。

面试题要点

Q：为什么要分块？分块多大合适？ 答：Embedding 模型有最大输入长度；且块越小检索越精确（“一个函数”比“整个文件”好匹配）。但块太小会丢失上下文。项目用 500 字符 + 50 重叠，代码场景比通用文本更关注函数边界。

Q：三种分块策略的区别？ 答：`recursive` 按分隔符优先级递归切，通用；`token` 按 `token` 数切，对模型上下文窗口友好；`semantic`（代码版）按顶格 `def/class` 切，每个 `chunk` 大概率是一个完整函数或类。代码场景下 `semantic` 更合适。

Q：overlap 是干嘛的？ 答：相邻块重叠一部分文本，避免在边界切断关键代码（如函数签名、语句）。切断了检索匹配就失败。

Q：metadata 里 source 和 chunk_index 有什么用？ 答：检索结果靠这对 key 定位原始文件和块位置；RRF 融合也用它做去重键。没有它，结果无法追溯到源码。

4.2 `dense_retriever.py` — 稠密向量检索

完整代码

```
"""Dense 检索器：封装 Chroma 的检索、添加、删除操作"""

import os
import json
from pathlib import Path
from typing import Any

from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

from app.config import settings
from app.logger import log

PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent

# 检索范围常量：系统语料 / 用户上传
```

```
SYSTEM_CORPUS = {"source_type": "system"}
USER_CORPUS = {"source_type": "user_upload"}
```

```
class DenseRetriever:
    """基于 Chroma 的稠密向量检索器"""

    def __init__(self):
        chroma_path = settings.CHROMA_PERSIST_DIR
        if not os.path.exists(chroma_path):
            chroma_path = str(PROJECT_ROOT / chroma_path)
        self.persist_dir = chroma_path

        os.makedirs(self.persist_dir, exist_ok=True)

        try:
            self.embeddings = HuggingFaceEmbeddings(
                model_name=settings.EMBEDDING_MODEL,
                model_kwargs={"device": settings.EMBEDDING_DEVICE},
            )
        except Exception as e:
            raise RuntimeError(
                f"Failed to load embedding model '{settings.EMBEDDING_MODEL}': {e}"
            )

        self.redis_client = None
        try:
            import redis as redis_lib
            self.redis_client = redis_lib.Redis(
                host=settings.REDIS_HOST,
                port=settings.REDIS_PORT,
                db=0,
                decode_responses=True,
                protocol=2,
            )
            self.redis_client.ping()
        except Exception as e:
            log.warning("Redis not available, caching disabled: %s", e)
            self.redis_client = None

        try:
            self.db = Chroma(
                persist_directory=self.persist_dir,
                embedding_function=self.embeddings,
            )
        except Exception as e:
            raise RuntimeError(f"Failed to initialize Chroma at {self.persist_dir}: {e}")

    def search(self, query: str, k: int = 5,
               where: dict | None = None) -> list[dict[str, Any]]:
        """稠密检索。where 是 Chroma metadata filter, 用于隔离检索范围。

        示例:
            search("query") # 搜全部 (默认)
            search("query", where={"source_type": "system"})
            search("query", where={"owner_id": 1, "source_type": "user_upload"})
        """
        if not query or not query.strip():
            return []

        where_str = json.dumps(where, sort_keys=True) if where else "all"
        cache_key = f"dense_search:{query.strip()}:{k}:{where_str}"
        if self.redis_client:
            try:
                cached = self.redis_client.get(cache_key)
                if cached:
                    return json.loads(cached)
            except:
```

```

        except Exception:
            pass

    try:
        if where:
            docs = self.db.similarity_search(query, k=k, filter=where)
        else:
            docs = self.db.similarity_search(query, k=k)
    except Exception as e:
        log.error("Search failed: %s", e)
        return []

    results = [
        {
            "text": doc.page_content,
            "source": doc.metadata.get("source", "unknown"),
            "chunk_index": doc.metadata.get("chunk_index", -1),
            "owner_id": doc.metadata.get("owner_id"),
            "score": None,
        }
        for doc in docs
    ]

    if self.redis_client and results:
        try:
            self.redis_client.setex(
                cache_key, settings.REDIS_CACHE_TTL,
                json.dumps(results, ensure_ascii=False),
            )
        except Exception:
            pass

    return results

def search_with_score(self, query: str, k: int = 5,
                      where: dict | None = None) -> list[dict[str, Any]]:
    if not query or not query.strip():
        return []

    where_str = json.dumps(where, sort_keys=True) if where else "all"
    cache_key = f"dense_search_score:{query.strip()}:{k}:{where_str}"
    if self.redis_client:
        try:
            cached = self.redis_client.get(cache_key)
            if cached:
                return json.loads(cached)
        except Exception:
            pass

    try:
        if where:
            docs_with_scores = self.db.similarity_search_with_score(query, k=k,
filter=where)
        else:
            docs_with_scores = self.db.similarity_search_with_score(query, k=k)
    except Exception as e:
        log.error("Search with score failed: %s", e)
        return []

    results = [
        {
            "text": doc.page_content,
            "source": doc.metadata.get("source", "unknown"),
            "chunk_index": doc.metadata.get("chunk_index", -1),
            "owner_id": doc.metadata.get("owner_id"),
            "score": score,
        }
        for doc, score in docs_with_scores
    ]

```

```

        if self.redis_client and results:
            try:
                self.redis_client.setex(
                    cache_key, settings.REDIS_CACHE_TTL,
                    json.dumps(results, ensure_ascii=False),
                )
            except Exception:
                pass

        return results

def add_chunks(self, chunks: list[dict[str, Any]]) -> list[str]:
    if not chunks:
        log.warning("No chunks to add")
        return []

    texts = [c["text"] for c in chunks]
    metadatas = [c["metadata"] for c in chunks]

    try:
        ids = self.db.add_texts(texts=texts, metadatas=metadatas)
        return ids
    except Exception as e:
        log.error("Failed to add chunks: %s", e)
        return []

def count(self) -> int:
    try:
        return self.db._collection.count()
    except Exception as e:
        log.error("Failed to get count: %s", e)
        return 0

def delete_collection(self):
    try:
        self.db.delete_collection()
        log.info("Collection deleted")
    except Exception as e:
        log.warning("Delete failed (may not exist): %s", e)

```

这段代码解决什么问题

稠密检索 = 把文本和查询都转成向量，算语义相似度。这是 RAG 的“语义路”。实现上封装 Chroma（向量数据库）+ all-MiniLM-L6-v2（嵌入模型）。

它和 BM25 的本质区别：BM25 是“字面匹配”（搜“store”必须出现“store”），稠密检索是“语义匹配”（搜“怎么保存数据”能匹配到 storages.py，因为语义相近）。

关键代码逐句拆解

① 检索范围常量 — 数据隔离的基石

```

SYSTEM_CORPUS = {"source_type": "system"}
USER_CORPUS = {"source_type": "user_upload"}

```

系统语料（TinyDB 源码）和用户上传内容在检索端就区分开。这就是 Chroma 的 metadata filter 条件。

② PROJECT_ROOT 计算

```
PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent.parent
```

`__file__` 在 `backend/app/rag/dense_retriever.py`, 往上推 4 层 = 项目根目录。比 `config.py` 的 `parents[2]` 多 2 层, 因为目录层级更深。用于定位相对路径的 `CHROMA_PERSIST_DIR`。

③ 缓存 key 的构造 — 缓存命中的关键

```
where_str = json.dumps(where, sort_keys=True) if where else "all"
cache_key = f"dense_search:{query.strip()}:{k}:{where_str}"
```

`sort_keys=True` 保证 `{"a":1, "b":2}` 和 `{"b":2, "a":1}` 生成同一个 key——dict 顺序不影响缓存命中。这是一个容易忽略但很重要的细节。

④ 缓存读写的降级

```
if self.redis_client:
    try:
        cached = self.redis_client.get(cache_key)
        if cached:
            return json.loads(cached)
    except Exception:
        pass
```

- 查缓存命中就直接返回, 省掉一次昂贵的向量检索。
- **Redis 不可用时静默跳过** (`except: pass`), 检索照常走, 只是没有缓存。
- `json.loads(cached)` 把缓存 JSON 还原成结果列表。

写缓存同理: `setex(cache_key, settings.REDIS_CACHE_TTL, json.dumps(results, ensure_ascii=False))`。`ensure_ascii=False` 防止中文被转义成 `\uXXXX`, 省空间、可读。

⑤ 统一结果结构

```
results = [
    {
        "text": doc.page_content,
        "source": doc.metadata.get("source", "unknown"),
        "chunk_index": doc.metadata.get("chunk_index", -1),
        "owner_id": doc.metadata.get("owner_id"),
        "score": None,
    }
    for doc in docs
]
```

注意 **"score": None**——稠密检索本身不返回相关性分数 (Chroma 的 `similarity_search` 不附带分数)。这恰恰是 RRF 融合的设计前提: RRF 只看排名不看分数。如果需要分数, 用 `search_with_score` (底层 `similarity_search_with_score`)。

`metadata.get(...)` 用 `.get` + 默认值, 保证即使某个 chunk 缺字段也不崩。

⑥ add_chunks — 写索引

```
texts = [c["text"] for c in chunks]
metadatas = [c["metadata"] for c in chunks]
```



```
ids = self.db.add_texts(texts=texts, metadatas=metadatas)
```

批量写入。metadatas 里带 source / chunk_index / source_type / owner_id——这些就是后续 filter 检索的条件。

面试题要点

Q: 稠密检索和稀疏检索的区别? 答: 稠密检索把文本/查询都编码成向量, 用余弦相似度算语义相似——懂同义词、能匹配语义相近但字面不同的内容; 稀疏检索 (BM25) 是词频统计, 精确匹配关键词但对同义词无能为力。两者互补。

Q: 为什么稠密检索结果不带分数也能融合? 答: 因为融合用的是 RRF——只看排名不看分数。稠密路的向量距离和 BM25 的词频分数度量衡不同, 直接比较无意义; RRF 按排名给分规避了这个问题。

Q: 检索结果缓存怎么做? 答: 以 query + k + where 序列化做 key 存 Redis, TTL 5 分钟。json.dumps(where, sort_keys=True) 保证 dict 顺序不影响 key。Redis 挂了自动降级跳过缓存。

Q: 数据隔离怎么做的? 答: 索引时给 chunk 打 source_type (system / user_upload) 和 owner_id 标签; 检索时通过 Chroma metadata filter (where 参数) 过滤。用户上传内容还叠加应用层兜底过滤 (见 user_upload.py), 双重保险。

Q: 嵌入模型选型? 答: all-MiniLM-L6-v2——轻量 (~23MB)、快、效果均衡, 适合代码检索的英文场景。代码里也留了中文模型的注释备选 (bge-small-zh-v1.5)。

4.3 sparse_retriever.py — BM25 稀疏检索

完整代码

```
"""Sparse 检索器: 基于 BM25 的关键词检索"""

import re
import json

from rank_bm25 import BM25Okapi

from app.config import settings
from app.logger import log

class SparseRetriever:
    """基于 BM25 的稀疏检索器"""

    def __init__(self):
        self.bm25 = None
        self.chunks: list[dict] = []
        self._tokenized: list[list[str]] = []

        self.redis_client = None
        try:
            import redis as redis_lib
            self.redis_client = redis_lib.Redis(
```

```

        host=settings.REDIS_HOST,
        port=settings.REDIS_PORT,
        db=0,
        decode_responses=True,
        protocol=2,
    )
    self.redis_client.ping()
except Exception as e:
    log.warning("Redis not available: %s", e)
    self.redis_client = None

def _tokenize(self, text: str) -> list[str]:
    tokens = re.findall(r"[a-zA-Z_]\w*", text.lower())
    return [t for t in tokens if len(t) > 1]

def build_index(self, chunks: list[dict]):
    self.chunks = chunks
    self._tokenized = [self._tokenize(c["text"]) for c in chunks]
    self.bm25 = BM25Okapi(self._tokenized)
    self._save_to_redis()
    log.info("Index built with %d chunks", len(chunks))

def search(self, query: str, k: int = 5) -> list[dict]:
    if not query or not query.strip() or not self.bm25:
        return []

    tokenized_query = self._tokenize(query)
    if not tokenized_query:
        return []

    scores = self.bm25.get_scores(tokenized_query)
    top_n_indices = sorted(
        range(len(scores)), key=lambda i: scores[i], reverse=True
   )[:k]

    results = []
    for idx in top_n_indices:
        if scores[idx] > 0:
            results.append({
                "text": self.chunks[idx]["text"],
                "source": self.chunks[idx]["metadata"]["source"],
                "chunk_index": self.chunks[idx]["metadata"]["chunk_index"],
                "score": float(scores[idx]),
            })
    return results

def count(self) -> int:
    return len(self.chunks) if self.bm25 else 0

def redis_key(self) -> str:
    return f"bm25_index:{settings.CHUNK_STRATEGY}"

def _save_to_redis(self):
    if not self.redis_client or not self.bm25:
        return
    try:
        data = {
            "chunks": self.chunks,
            "corpus": self._tokenized,
        }
        self.redis_client.setex(
            self.redis_key(), 3600 * 24,
            json.dumps(data, ensure_ascii=False, default=str),
        )
    except Exception as e:
        log.warning("Failed to cache to Redis: %s", e)

@classmethod
def from_redis(cls) -> "SparseRetriever":

```

```

retriever = cls()
if not retriever.redis_client:
    return retriever
try:
    data = retriever.redis_client.get(retriever.redis_key())
    if data:
        parsed = json.loads(data)
        retriever.chunks = parsed["chunks"]
        retriever._tokenized = parsed["corpus"]
        retriever.bm25 = BM25Okapi(retriever._tokenized)
        log.info("Restored from Redis (%d chunks)", len(retriever.chunks))
except Exception as e:
    log.warning("Redis restore failed: %s", e)
return retriever

@classmethod
def from_chunks(cls, chunks: list[dict]) -> "SparseRetriever":
    retriever = cls()
    retriever.build_index(chunks)
    return retriever

```

这段代码解决什么问题

BM25 是信息检索的经典算法（Okapi BM25），属于**稀疏检索**：基于词频统计，算“查询词在文档里有多重要”。和稠密检索互补——它对**专有名词、函数名、标识符**（如 JSONStorage、insert）非常精准，而这些恰恰是代码检索最常见的查询。

关键代码逐句拆解

① 分词 — 代码检索的关键

```

def _tokenize(self, text: str) -> list[str]:
    tokens = re.findall(r"[a-zA-Z_]\w*", text.lower())
    return [t for t in tokens if len(t) > 1]

```

正则 `[a-zA-Z_]\w*` 匹配 Python 标识符（变量名、函数名、类名），`lower()` 统一小写，过滤单字符。这样 `insert`、`table`、`query` 等 token 能精确匹配代码里的词。

② build_index — 建 BM25 索引

```

def build_index(self, chunks: list[dict]):
    self.chunks = chunks
    self._tokenized = [self._tokenize(c["text"]) for c in chunks]
    self.bm25 = BM25Okapi(self._tokenized)
    self._save_to_redis()

```

把所有 chunk 分词，BM25Okapi 根据语料构建倒排统计。rank_bm25 库一行搞定。

③ search — 检索

```

scores = self.bm25.get_scores(tokenized_query)
top_n_indices = sorted(
    range(len(scores)), key=lambda i: scores[i], reverse=True
)[:k]

```

`get_scores` 返回所有 chunk 的 BM25 分数，按分数排序取前 k 个索引，然后**过滤掉分数为 0 的**（`if scores[idx] > 0`——完全没匹配到任何查询词的结果没意义）。

注意返回结构里 "score": float(scores[idx])——**BM25 路是有真实分数的**，与稠密路的 score: None 形成对比，更凸显 RRF“只看排名”的价值。

④ 索引持久化到 Redis — 亮点

```
def redis_key(self) -> str:
    return f"bm25_index:{settings.CHUNK_STRATEGY}"

def _save_to_redis(self):
    data = {"chunks": self.chunks, "corpus": self._tokenized}
    self.redis_client.setex(self.redis_key(), 3600 * 24, json.dumps(data,
ensure_ascii=False, default=str))

@classmethod
def from_redis(cls) -> "SparseRetriever":
    retriever = cls()
    data = retriever.redis_client.get(retriever.redis_key())
    if data:
        parsed = json.loads(data)
        retriever.chunks = parsed["chunks"]
        retriever._tokenized = parsed["corpus"]
        retriever.bm25 = BM25Okapi(retriever._tokenized)
    return retriever
```

为什么 BM25 索引要缓存到 Redis? BM25 索引在内存里，服务重启就没了。重建要重新读几百个 chunk 并分词（耗时）。缓存到 Redis 后，服务重启直接 from_redis() 恢复，秒级启动。

redis_key 带 CHUNK_STRATEGY: 分块策略变了，key 就变，绝不会用错索引。
from_redis() / from_chunks() 是类方法工厂，两种构建方式：从 Redis 恢复、或从 chunks 现建。

default=str: JSON 序列化时兜底处理不可序列化的对象（比如某些 datetime）。

面试题要点

Q: BM25 是什么？和 TF-IDF 什么区别？ 答：BM25 是 TF-IDF 的改进版。它加了文档长度归一化（短文档中的词匹配更值钱）和饱和函数（词频过高时得分非线性增长）。rank_bm25 的 BM25Okapi 实现了这个算法。

Q: 为什么 BM25 索引要存 Redis? 答：BM25 是内存索引，进程重启就丢。存 Redis 让新进程秒级恢复，不用重新分词几百个 chunk。key 带分块策略名防止不同配置的索引互相污染。

Q: 分词怎么针对代码设计？ 答：用 [a-zA-Z_]\w* 正则只提取标识符（函数名、类名、变量名），过滤单字符。这样 insert、JSONStorage 等代码元素能精确参与 BM25 词频统计，比通用英文分词更适合代码检索。

Q: 稀疏检索的局限？ 答：不懂同义词和语义（搜“保存数据”匹配不到 storages.py 里的 write）；对大小写、拼写敏感。所以需要稠密检索互补。

4.4 fusion.py — RRF 结果融合

完整代码

```
"""结果融合: RRF (Reciprocal Rank Fusion) 合并多路检索结果"""

import asyncio
import time
from typing import Any

from app.logger import log

def rrf(
    results_list: list[list[dict[str, Any]]],
    k: int = 60,
    top_n: int = 5,
) -> list[dict[str, Any]]:
    fusion_scores: dict[tuple[str, int], dict] = {}

    for results in results_list:
        for rank, item in enumerate(results):
            key = (item["source"], item["chunk_index"])
            if key not in fusion_scores:
                fusion_scores[key] = {
                    "text": item["text"],
                    "source": item["source"],
                    "chunk_index": item["chunk_index"],
                    "score": 0.0,
                }
            fusion_scores[key]["score"] += 1.0 / (k + rank + 1)

    sorted_items = sorted(
        fusion_scores.values(),
        key=lambda x: x["score"],
        reverse=True,
    )
    return sorted_items[:top_n]

def hybrid_search(
    query: str,
    dense_retriever,
    sparse_retriever,
    k_dense: int = 10,
    k_sparse: int = 10,
    k_rrf: int = 60,
    top_n: int = 5,
    dense_where: dict | None = None,
) -> dict:
    start = time.time()

    dense_results = dense_retriever.search(query, k=k_dense, where=dense_where)
    sparse_results = sparse_retriever.search(query, k=k_sparse)

    log.info("Dense: %d results, Sparse: %d results", len(dense_results),
    len(sparse_results))

    fused = rrf([dense_results, sparse_results], k=k_rrf, top_n=top_n)

    total_ms = round((time.time() - start) * 1000, 2)
    log.info("Fused: %d, latency: %.2fms", len(fused), total_ms)

    return {
        "results": fused,
        "dense_top_k": dense_results[:3],
        "sparse_top_k": sparse_results[:3],
    }
```

```

        "latency_ms": total_ms,
    }

async def async_hybrid_search(
    query: str,
    dense_retriever,
    sparse_retriever,
    k_dense: int = 10,
    k_sparse: int = 10,
    k_rrf: int = 60,
    top_n: int = 5,
    dense_where: dict | None = None,
) -> list[dict[str, Any]]:
    """异步混合检索：用 to_thread 避免阻塞事件循环"""
    result = await asyncio.to_thread(
        hybrid_search,
        query, dense_retriever, sparse_retriever,
        k_dense=k_dense, k_sparse=k_sparse,
        k_rrf=k_rrf, top_n=top_n,
        dense_where=dense_where,
    )

    try:
        from app.database import async_session_factory
        from app.models.retrieval_log import RetrievalLog
        import datetime
        from datetime import UTC

        async with async_session_factory() as session:
            session.add(RetrievalLog(
                query_text=query,
                dense_top_k=result["dense_top_k"],
                sparse_top_k=result["sparse_top_k"],
                fused_top_n=result["results"],
                strategy="hybrid",
                total_latency_ms=result["latency_ms"],
            ))
            await session.commit()
    except Exception as e:
        log.warning("Failed to log retrieval: %s", e)

    return result["results"]

```

这段代码解决什么问题

两路检索各自返回 top-k，**怎么合并成一路？** 直接加权平均不行——两路分数度量衡完全不同。

RRF (Reciprocal Rank Fusion) 是标准解法：只看排名，不碰分数。

关键代码逐句拆解

① RRF 核心公式

```

def rrf(results_list, k=60, top_n=5):
    fusion_scores = {}
    for results in results_list:
        for rank, item in enumerate(results):
            key = (item["source"], item["chunk_index"])
            if key not in fusion_scores:
                fusion_scores[key] = {...}
            fusion_scores[key]["score"] += 1.0 / (k + rank + 1)

```

公式是 $\text{score} = \sum 1/(k + \text{rank})$:

- 一个文档在**每一路**里都有一个排名（rank 从 0 开始）。
- 每路的贡献是 $1/(60 + \text{rank} + 1)$ 。
- 排名越靠前，贡献越大；跨路累加。

关键：它只用 rank（排名），从不看原分数。稠密路返回 0.9 分和 BM25 返回 1000 分，都只按“排第几”算贡献——这就是为什么前面说“分数不可比也能融合”。

去重键 (source, chunk_index)：同一个 chunk 被两路都召回时，fusion_scores 里只有一个 key，分数累加。**被两路同时召回的 chunk 会获得双倍贡献，自然排到前面**——这是 RRF 的另一个妙处：两路都认为重要的，大概率真的重要。

② hybrid_search — 同步编排

```
dense_results = dense_retriever.search(query, k=k_dense, where=dense_where)
sparse_results = sparse_retriever.search(query, k=k_sparse)
fused = rrf([dense_results, sparse_results], k=k_rrf, top_n=top_n)
```

双路各取 top-10 → RRF 融合 top-5 → 记录延迟。

返回值是**结构化字典**：

```
return {
    "results": fused,                # 融合后的最终结果
    "dense_top_k": dense_results[:3], # 稠密路 top-3 (用于日志)
    "sparse_top_k": sparse_results[:3], # 稀疏路 top-3 (用于日志)
    "latency_ms": total_ms,          # 总耗时
}
```

dense_top_k / sparse_top_k 是**给日志用的**——你可以对比两路各自搜到什么，融合后剩下什么。

③ async_hybrid_search — 异步版 + 链路日志

```
result = await asyncio.to_thread(
    hybrid_search, query, dense_retriever, sparse_retriever, ...
)
```

hybrid_search 是同步函数（里面 Chroma / BM25 都是同步库）。**直接 await 会阻塞事件循环**，所以用 asyncio.to_thread 把它扔到线程池跑，不阻塞其他请求。这是“同步阻塞库嵌进异步框架”的标准解法。

然后异步写 RetrievalLog（第 2 层的 retrieval_logs 表）——把三路中间结果 + 延迟落库。try/except 包住：日志失败不影响检索主流程。

面试题答题要点

Q：为什么用 RRF 而不是加权平均？ 答：稠密检索的分数（向量距离）和 BM25 的分数（词频统计）度量衡完全不同，直接加权没有意义。RRF 只看排名不看分数，规避了分数不可比的问题，是信息检索的标准做法。

Q：RRF 公式里的 k 有什么用？ 答：k 是平滑常数（常用 60）。防止排名第 1 的文档

分数过高，让排名差异不要被过分放大。 $1/(60+1)$ vs $1/(60+2)$ 差距很小，而 $1/1$ vs $1/2$ 差距巨大——k 让融合更“温和”。

Q：同一个文档在两路都被召回会怎样？ 答：(source, chunk_index) 作为去重键，两路贡献累加，它会被推到更靠前的位置。这是 RRF 的隐性优势：两路都认为重要的文档优先。

Q：为什么用 asyncio.to_thread？ 答：Chroma 和 BM25 是同步库，直接 await 会阻塞 FastAPI 的事件循环。to_thread 把它们扔到线程池，让出事件循环处理其他请求。

Q：检索日志为什么要存三路结果？ 答：可观测性。回看任何一次检索，能对比稠密路/稀疏路各自召回什么、融合后剩什么、耗时多少。排查“某次检索为什么不好”时有现场录像。

4.5 query_rewriter.py — HyDE 查询改写

完整代码

```
"""查询改写：HyDE (Hypothetical Document Embedding) 策略"""

import asyncio

from app.agent.llm import call_llm
from app.logger import log

HYDE_SYSTEM_PROMPT = """你是一个 Python 代码助手。用户会问一个关于代码库的问题。
请生成一段你认为最能回答这个问题的 Python 代码片段。
只需要输出代码，不要解释。如果问题不涉及代码，用伪代码表示。"""

EXPAND_SYSTEM_PROMPT = """你是一个搜索优化助手。用户输入一个简短的搜索查询。
请将其扩展为 2-3 个更具体、更可能匹配到相关代码的搜索词。
直接用逗号分隔输出，不要多余解释。"""

async def _save_rewrite_record(original: str, rewritten: str, strategy: str):
    """异步写入 query_rewrites 表（即发即忘，不阻塞调用方）"""
    try:
        from app.database import async_session_factory
        from app.models.query_rewrite import QueryRewrite

        async with async_session_factory() as session:
            session.add(QueryRewrite(
                original_query=original,
                rewritten_query=rewritten,
                strategy=strategy,
            ))
            await session.commit()
    except Exception as e:
        log.warning("Failed to save rewrite record: %s", e)

def hyde_rewrite(query: str) -> str:
    log.info("HyDE rewriting: '%s'", query)
    hypothetical_code = call_llm(query, system_prompt=HYDE_SYSTEM_PROMPT)
    if hypothetical_code:
        log.info("HyDE result: %s...", hypothetical_code[:80])
```



```

        return hypothetical_code
    return query

def expand_query(query: str) -> str:
    log.info("Expanding query: '%s'", query)
    expanded = call_llm(query, system_prompt=EXPAND_SYSTEM_PROMPT)
    if expanded:
        log.info("Expanded: %s", expanded)
        return expanded
    return query

async def rewrite(query: str, strategy: str = "hyde") -> str:
    """
    统一的查询改写入口（异步，通过 to_thread 避免阻塞事件循环）
    """
    # ① 将同步的 LLM 调用扔到线程池，不阻塞事件循环
    if strategy == "hyde":
        result = await asyncio.to_thread(hyde_rewrite, query)
    elif strategy == "expand":
        result = await asyncio.to_thread(expand_query, query)
    else:
        result = query

    # ② 粗细粒度的"是否改写"检测：去掉首尾空白后比较
    if result.strip() != query.strip() and result != query:
        asyncio.create_task(_save_rewrite_record(query, result, strategy))

    return result

```

这段代码解决什么问题

用户问的自然语言问题和代码库里的真实代码，语义空间有差距。比如问“TinyDB 怎么把数据存到磁盘的？”，检索时拿这句话去匹配代码，效果一般。**HyDE (Hypothetical Document Embedding)** 的思路：先让 LLM 把问题“翻译”成一段假设性的代码，再拿这段代码去检索。

用户问题：TinyDB 怎么把数据存到磁盘的？

```

|   LLM 生成假设代码
▼
with TinyDB('db.json') as db:
    db.insert({...})
|   这段代码的向量和 storages.py 里的真实代码更接近
▼
检索命中 storages.py ✓

```

为什么有效？嵌入模型在“代码→代码”的空间里比对，比“自然语言→代码”准得多。这是业界常用的 RAG 优化手段。

关键代码逐句拆解

① 两个系统提示词

```

HYDE_SYSTEM_PROMPT = """你是一个 Python 代码助手。... 请生成一段你认为最能回答这个问题的 Python 代码片段。
只需要输出代码，不要解释。"""

```

```

EXPAND_SYSTEM_PROMPT = """你是一个搜索优化助手。... 请将其扩展为 2-3 个更具体、更可能匹配到相关代码的搜索词。"""

```

两个策略：- **hyde**：生成“假设代码”。- **expand**：扩展成多个搜索词（更宽召回）。

② hyde_rewrite / expand_query — 同步 LLM 调用

```
def hyde_rewrite(query: str) -> str:
    hypothetical_code = call_llm(query, system_prompt=HYDE_SYSTEM_PROMPT)
    if hypothetical_code:
        return hypothetical_code
    return query
```

`call_llm` 是第 5 层会讲到的 LLM 封装。**LLM 调用失败或返回空时，降级返回原查询——改写失败不能让检索也失败。**

③ rewrite — 统一异步入口

```
if strategy == "hyde":
    result = await asyncio.to_thread(hyde_rewrite, query)
```

LLM 调用是同步阻塞的，用 `asyncio.to_thread` 扔线程池，不阻塞事件循环。

④ 改写记录落库（即发即忘）

```
if result.strip() != query.strip() and result != query:
    asyncio.create_task(_save_rewrite_record(query, result, strategy))
```

两个条件判断“确实改写了”（去掉首尾空白后文本不同），然后 `asyncio.create_task` 异步写 `query_rewrites` 表——**不 `await`，即发即忘**，不拖慢检索响应。这与 `fusion.py` 里“日志失败不影响主流程”是同一哲学。

面试题要点

Q: HyDE 的原理？ 答：检索前先让 LLM 根据问题生成一段假设文档（这里是假设代码），再用这段代码去检索。因为嵌入模型在“代码→代码”空间的相似度比对，比“自然语言→代码”更准，能提升召回率。

Q: HyDE 的代价？ 答：多一次 LLM 调用，延迟从几毫秒涨到几秒（README 里显示 3600ms）。所以它要作为“可选开关”（`search` 接口的 `use_hyde` 参数），而不是默认开启。

Q: 改写失败怎么办？ 答：LLM 调用失败或返回空，降级返回原查询——改写是锦上添花，不能让检索主干受影响。改写记录用 `create_task` 异步落库，不阻塞响应。

Q: 消融实验里 HyDE 有没有提升？ 答：这个项目里没有（Hit Rate 都是 0.95）。原因：基础混合检索已经够好，且测试集问题偏简单。这正好说明“组件要按领域验证”——HyDE 在复杂、口语化查询下收益更大，这个项目的问题比较直白。

4.6 reranker.py — Cross-encoder 重排

完整代码

```
"""Cross-encoder 重排序器：对初检结果精排（带 Redis 缓存）"""

import hashlib
import json

from sentence_transformers import CrossEncoder

from app.config import settings
from app.logger import log

class Reranker:
    """基于 cross-encoder 的精排重排序"""

    def __init__(self):
        model_name = "cross-encoder/ms-marco-MiniLM-L-6-v2"
        log.info("Loading reranker: %s", model_name)
        try:
            self.model = CrossEncoder(model_name)
            log.info("Reranker loaded")
        except Exception as e:
            log.error("Failed to load reranker: %s", e)
            self.model = None

        self.redis_client = None
        try:
            import redis as redis_lib
            self.redis_client = redis_lib.Redis(
                host=settings.REDIS_HOST,
                port=settings.REDIS_PORT,
                db=0,
                decode_responses=True,
                protocol=2,
            )
            self.redis_client.ping()
        except Exception:
            self.redis_client = None

    def rerank(self, query: str, candidates: list[dict], top_n: int = 3) -> list[dict]:
        if not self.model or not candidates:
            for c in candidates:
                c["rerank_score"] = 0.0
            return candidates[:top_n]

        # 尝试从缓存读取（使用确定性哈希，跨进程一致）
        cache_key = f"rerank:{hashlib.md5(query.encode()).hexdigest()}:{top_n}"
        if self.redis_client:
            try:
                cached = self.redis_client.get(cache_key)
                if cached:
                    return json.loads(cached)
            except Exception:
                pass

        pairs = [(query, c["text"]) for c in candidates]
        scores = self.model.predict(pairs)

        for i, c in enumerate(candidates):
            c["rerank_score"] = float(scores[i])

        ranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
```

```

        result = ranked[:top_n]

        log.info("Reranked %d candidates, top score: %.4f",
                  len(candidates), result[0]["rerank_score"] if result else 0)

    # 写入缓存
    if self.redis_client and result:
        try:
            self.redis_client.setex(
                cache_key, 3600, json.dumps(result, ensure_ascii=False)
            )
        except Exception:
            pass

    return result

def hybrid_search_with_rerank(
    self,
    query: str,
    dense_retriever,
    sparse_retriever,
    k_dense: int = 10,
    k_sparse: int = 10,
    k_rrf: int = 60,
    top_n: int = 3,
    dense_where: dict | None = None,
) -> list[dict]:
    from app.rag.fusion import hybrid_search

    result = hybrid_search(
        query, dense_retriever, sparse_retriever,
        k_dense=k_dense, k_sparse=k_sparse,
        k_rrf=k_rrf, top_n=top_n * 2,
        dense_where=dense_where,
    )

    return self.rerank(query, result["results"], top_n=top_n)

```

这段代码解决什么问题

Cross-encoder 是“终极裁判”：它把查询和文档**拼在一起** ((query, doc)) 送进模型，一次只判断一对，输出匹配分数。这比稠密检索的 bi-encoder（各自编码、算余弦）**更精确**，但慢——所以只对初检的 top-N 精排，不做全库排序。

这是标准的“召回粗排 + 精排”两阶段架构：

全库 —— (稠密/BM25 双路召回, 快) → top-10 —— (cross-encoder 精排, 慢但准) → top-3

关键代码逐句拆解

① 模型加载失败降级

```

try:
    self.model = CrossEncoder(model_name)
except Exception as e:
    log.error("Failed to load reranker: %s", e)
    self.model = None

```

模型加载失败（比如没下载、没显存）就置 None，后续 rerank 里 if not self.model 直接跳过精排，返回原结果——**重排是增强，不是主干，不能让它拖垮检索。**

② 精排主体

```
pairs = [(query, c["text"]) for c in candidates]
scores = self.model.predict(pairs)

for i, c in enumerate(candidates):
    c["rerank_score"] = float(scores[i])

ranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
result = ranked[:top_n]
```

构造 (query, doc) 对 → 模型预测分数 → 写回每个候选的 rerank_score → 按分数降序排序取 top_n。注意它就地修改了 **candidates** 的 dict（给每个加 rerank_score 字段），返回的是排序后的子集。

③ 缓存 key — 确定性哈希

```
cache_key = f"rerank:{hashlib.md5(query.encode()).hexdigest()}:{top_n}"
```

用 md5(query) 做 key。为什么用 md5 而不是直接拼 query？因为 query 可能很长、含特殊字符，直接拼进 Redis key 又长又丑。md5 保证：相同 query → 相同哈希（确定性，跨进程一致），key 长度固定 32 字符。top_n 也拼进去，因为不同 top_n 结果不同。

④ hybrid_search_with_rerank — 一键组合

```
result = hybrid_search(
    query, dense_retriever, sparse_retriever,
    top_n=top_n * 2,          # 先召回 2 倍数量的候选
    ...
)
return self.rerank(query, result["results"], top_n=top_n)
```

先混合检索召回 top_n * 2 个候选，再精排到 top_n。留出“余量”给精排挑——如果初检只召回 3 个，精排没有选择空间。

面试答题要点

Q: bi-encoder 和 cross-encoder 的区别？ 答：bi-encoder 把查询和文档各自编码成向量，算相似度——快，可对全库预先索引，但精度有限；cross-encoder 把查询和文档拼接一起送进模型，直接输出匹配分——准，但慢（无法预索引）。所以架构上“bi-encoder/BM25 粗召回 + cross-encoder 精排”是标准组合。

Q: 这个项目里 reranker 效果如何？ 答：消融实验显示反而降分（Hit Rate 从 0.95 → 0.85）。原因是 ms-marco 模型在网页搜索领域训练，对代码不敏感。这证明了“组件要按领域验证，不能无脑叠加”——是很诚实的工程结论。

Q: 重排缓存怎么做？ 答：md5(query) + top_n 做 key 存 Redis 1 小时。md5 保证确定性哈希，相同查询跨进程一致，命中缓存省掉一次模型推理。

4.7 code_indexer.py — 离线索引编排器

完整代码

```
"""代码索引编排器：读取文件 → chunker 分块 → dense_retriever 存储"""

import os
import glob
from pathlib import Path

from app.config import settings
from app.rag.chunker import CodeChunker
from app.rag.dense_retriever import DenseRetriever
from app.rag.sparse_retriever import SparseRetriever
from app.database import async_session_factory
from app.models.index_version import IndexVersion
from app.logger import log

PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent.parent

def load_code_files(repo_path: str) -> list[dict]:
    """读取 repo 下所有 .py 文件，返回 [{path, content}]"""
    documents = []
    full_path = repo_path
    if not os.path.isabs(repo_path):
        full_path = str(PROJECT_ROOT / repo_path)

    log.info("Looking for code in: %s", full_path)

    if not os.path.exists(full_path):
        log.error("Path does not exist: %s", full_path)
        return documents

    pattern = os.path.join(full_path, "**", "*.py")
    try:
        matched_files = glob.glob(pattern, recursive=True)
    except Exception as e:
        log.error("Failed to search files: %s", e)
        return documents

    for file_path in matched_files:
        if "__pycache__" in file_path or "/tests/" in file_path or "\\tests\\" in file_path:
            continue
        try:
            with open(file_path, "r", encoding="utf-8") as f:
                content = f.read()
        except PermissionError:
            log.warning("Permission denied: %s", file_path)
            continue
        except UnicodeDecodeError:
            log.warning("Encoding error, trying latin-1: %s", file_path)
            try:
                with open(file_path, "r", encoding="latin-1") as f:
                    content = f.read()
            except Exception:
                continue
        except Exception as e:
            log.warning("Failed to read %s: %s", file_path, e)
            continue

        if content.strip():
            documents.append({
                "path": os.path.relpath(file_path, full_path),
                "content": content,
            })
```

```
    })
    return documents
```

```
async def save_version_record(strategy: str, chunk_size: int, chunk_overlap: int,
                             file_count: int, chunk_count: int):
```

```
    """异步写入索引版本记录"""
```

```
    try:
```

```
        async with async_session_factory() as session:
```

```
            session.add(IndexVersion(
```

```
                strategy=strategy,
```

```
                chunk_size=chunk_size,
```

```
                chunk_overlap=chunk_overlap,
```

```
                file_count=file_count,
```

```
                chunk_count=chunk_count,
```

```
            ))
```

```
            await session.commit()
```

```
            log.info("Version record saved")
```

```
    except Exception as e:
```

```
        log.warning("Failed to save version record: %s", e)
```

```
async def create_index():
```

```
    """主流程：加载代码 → chunker 分块 → retriever 存储"""
```

```
    log.info("Loading code from %s", settings.REPO_PATH)
```

```
    documents = load_code_files(settings.REPO_PATH)
```

```
    log.info("Loaded %d files", len(documents))
```

```
    if not documents:
```

```
        log.error("No .py files found!")
```

```
        return
```

```
    try:
```

```
        chunker = CodeChunker(
```

```
            strategy=settings.CHUNK_STRATEGY,
```

```
            chunk_size=settings.CHUNK_SIZE,
```

```
            chunk_overlap=settings.CHUNK_OVERLAP,
```

```
        )
```

```
    except Exception as e:
```

```
        log.error("Failed to create chunker: %s", e)
```

```
        return
```

```
    log.info("Using chunk strategy: %s", settings.CHUNK_STRATEGY)
```

```
    try:
```

```
        chunks = chunker.chunk(documents)
```

```
    except Exception as e:
```

```
        log.error("Chunking failed: %s", e)
```

```
        return
```

```
    log.info("Created %d chunks", len(chunks))
```

```
    # 给系统语料打标，便于检索时与用户上传内容隔离
```

```
    for c in chunks:
```

```
        c["metadata"]["source_type"] = "system"
```

```
    try:
```

```
        retriever = DenseRetriever()
```

```
        retriever.delete_collection()
```

```
        retriever = DenseRetriever()
```

```
        retriever.add_chunks(chunks)
```

```
    # 从同一批 chunks 构建 BM25 稀疏索引并缓存到 Redis
```

```
    sparse = SparseRetriever.from_chunks(chunks)
```

```
    log.info("BM25 index built with %d chunks", sparse.count())
```

```
    await save_version_record(
```

```
        strategy=settings.CHUNK_STRATEGY,
```

```
        chunk_size=settings.CHUNK_SIZE,
```

```
        chunk_overlap=settings.CHUNK_OVERLAP,
```

```

        file_count=len(documents),
        chunk_count=len(chunks),
    )

    log.info("Index built successfully with %d chunks", len(chunks))
except Exception as e:
    log.error("Failed to build index: %s", e)
    return

return {"dense": retriever, "sparse": sparse}

async def rebuild_index():
    """对外暴露的"重建索引"接口，后续 API 路由会调用"""
    return await create_index()

if __name__ == "__main__":
    import asyncio

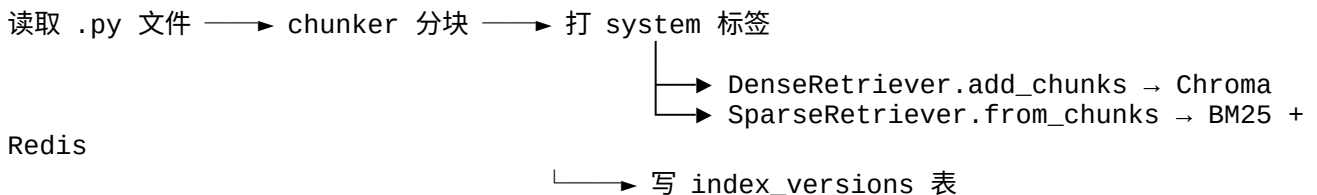
    async def main():
        try:
            await create_index()
        finally:
            from app.database import engine
            await engine.dispose()
            log.info("Database connections closed")

    asyncio.run(main())

```

这段代码解决什么问题

这是**离线建索引**的主流程（运行 `python -m app.rag.code_indexer` 触发）。把 TinyDB 源码变成可检索的稠密 + 稀疏双索引：



为什么“离线”？ 因为建索引要加载嵌入模型、对全部代码做向量化，耗时以分钟计。不能每次请求都做——**一次建好，反复查询**。这和“查询时在线检索”是两回事。

关键代码逐句拆解

① `load_code_files` — 递归读 .py 文件

```

pattern = os.path.join(full_path, "**", "*.py")
matched_files = glob.glob(pattern, recursive=True)

```

`glob` 递归匹配所有 .py。跳过清单：

```

if "__pycache__" in file_path or "/tests/" in file_path or "\\tests\\" in file_path:
    continue

```

跳过缓存目录和测试目录（`/tests/` 兼容 Linux，`\\tests\\` 兼容 Windows——跨平台细节）。测试代码不索引，因为测试不是“知识库”的一部分。

编码健壮性：优先 UTF-8；UnicodeDecodeError 时降级 latin-1 再试——保证个别编码奇怪的文件不拖垮整个索引。

输出路径用相对路径：

```
"path": os.path.relpath(file_path, full_path)
```

存 tinydb/storages.py 而不是绝对路径——可移植，换机器索引路径不变。

② 打 source_type 标签 — 隔离基础

```
for c in chunks:
    c["metadata"]["source_type"] = "system"
```

每个系统 chunk 打上 source_type: "system"。这样在线检索时 where={"source_type": "system"} 就只搜系统语料；用户上传的 chunk 是 "user_upload"，天然分开。同一向量库，两种语料，靠 metadata 隔离。

③ 重建索引 = 先删后建

```
retriever = DenseRetriever()
retriever.delete_collection()
retriever = DenseRetriever()
retriever.add_chunks(chunks)
```

先 delete_collection() 清空旧索引，再 new 一个实例重建。为什么 new 两次？因为删除后原实例的内部状态可能失效，重建一个干净实例。保证索引始终一致，不会新旧混合。

④ BM25 同步建

```
sparse = SparseRetriever.from_chunks(chunks)
```

用同一批 chunks 建 BM25 索引，并自动缓存到 Redis (build_index 里调用 _save_to_redis)。稠密和稀疏用同一批块，保证融合时 (source, chunk_index) 能对上。

⑤ 写版本记录

```
await save_version_record(strategy=..., chunk_size=..., file_count=...,
chunk_count=...)
```

写 index_versions 表——这次索引用什么配置建的、多少个文件多少个块，可追溯（配合 evaluation 做交叉分析）。

⑥ if __name__ == "__main__" 入口

```
if __name__ == "__main__":
    asyncio.run(main())
```

作为脚本运行 (python -m app.rag.code_indexer) 时执行；被 import 时不执行。finally 里 engine.dispose() 确保脚本退出前关掉数据库连接——防止进程挂住。

面试题要点

Q: 为什么要离线建索引? 答: 建索引要加载嵌入模型、对全部代码向量化, 耗时以分钟计, 不可能每次请求都做。所以一次离线建好(脚本/启动任务), 在线查询只做检索。稠密索引存 Chroma, BM25 索引存内存 + Redis 备份。

Q: 索引怎么隔离系统语料和用户语料? 答: 建索引时给每个 chunk 打 `source_type: "system"`; 用户上传打 `"user_upload" + owner_id`。检索时用 Chroma metadata filter 按标签过滤。同一个 Chroma 集合里物理共存、逻辑隔离。

Q: 重建索引为什么先删后建? 答: 保证索引一致性, 不会出现新旧 chunk 混合。`delete_collection` 清空后重新 `add`。

Q: 为什么跳过硬编码绝对路径、用 `relpath`? 答: 索引里存相对路径(如 `tinydb/storages.py`) 可移植, 换机器、换目录都能定位, 也保证 (`source`, `chunk_index`) 融合键稳定。

4.8 user_upload.py — 用户上传隔离检索

完整代码

```
"""用户文件上传索引: 按 owner_id 隔离 + 检索"""

from app.rag.chunker import CodeChunker
from app.rag.dense_retriever import DenseRetriever, USER_CORPUS
from app.logger import log

class UserUploadIndex:
    """管理用户上传文件的索引(按 owner_id 隔离)"""

    def __init__(self):
        # 复用 DenseRetriever (Chroma 持久化)
        self.retriever = DenseRetriever()

    def add_file(self, filename: str, content: str, owner_id: int) -> int:
        """索引一个上传的文件, 返回生成的 chunk 数

        给每个 chunk 写入 owner_id + source_type 元数据, 用于检索时隔离。
        """
        documents = [{"path": f"upload/{filename}", "content": content}]
        chunker = CodeChunker()
        chunks = chunker.chunk(documents)

        for c in chunks:
            c["metadata"]["source_type"] = "user_upload"
            c["metadata"]["owner_id"] = owner_id

        self.retriever.add_chunks(chunks)
        log.info("Indexed upload %s (owner=%d) -> %d chunks", filename, owner_id,
len(chunks))
        return len(chunks)

    def search(self, query: str, owner_id: int, k: int = 5) -> list[dict]:
        """只检索指定用户上传的文档
```

```

通过 Chroma metadata filter 在检索端隔离：
    {"source_type": "user_upload", "owner_id": owner_id}
    """
    if not query or not query.strip():
        return []

    where = {**USER_CORPUS, "owner_id": owner_id}
    results = self.retriever.search(query, k=k, where=where)

    # 兜底过滤：防止 Chroma filter 兼容问题导致越权
    return [r for r in results
            if str(r.get("source", "")).startswith("upload/")
            and r.get("owner_id") == owner_id]

```

这段代码解决什么问题

让用户上传自己的代码文件，索引后**只能搜到自己的**——这是数据隔离 / 防越权的核心。安全设计是双层的：

第 1 层：Chroma metadata filter where={"source_type": "user_upload", "owner_id": 你的 id}

第 2 层：应用层兜底过滤 source 必须以 "upload/" 开头 且 owner_id 匹配

关键代码逐句拆解

① add_file — 索引上传文件

```

documents = [{"path": f"upload/{filename}", "content": content}]
chunker = CodeChunker()
chunks = chunker.chunk(documents)

for c in chunks:
    c["metadata"]["source_type"] = "user_upload"
    c["metadata"]["owner_id"] = owner_id

```

- 路径前缀 upload/：这是后续兜底过滤的标记——只有用户上传的内容才有这个前缀，系统语料没有。
- 每个 chunk 打两重标签：source_type="user_upload" + owner_id=用户 id。
- 复用了 DenseRetriever（同一个 Chroma 集合），只是 metadata 不同。

② search — 双重隔离检索

```

where = {**USER_CORPUS, "owner_id": owner_id}
results = self.retriever.search(query, k=k, where=where)

return [r for r in results
        if str(r.get("source", "")).startswith("upload/")
        and r.get("owner_id") == owner_id]

```

第一层：where 过滤——Chroma 只在满足 source_type="user_upload" 且 owner_id=当前用户 的 chunk 里检索。

第二层：应用层兜底——即使 Chroma 的 filter 行为有 bug（比如版本兼容问题导致 filter 失效），返回结果再过滤一遍：source 必须以 upload/ 开头、owner_id 必须等于当前用户。**两重验证，即使某一层失效也越不了权。**

这是“纵深防御”（defense in depth）思想的体现，也是面试官最想听到的安全设计。

面试题要点

Q：用户上传的文件怎么做到互不可见？ 答：索引时每个 chunk 打 owner_id 和 source_type="user_upload" 标签；检索时用 Chroma metadata filter 过滤 owner_id=当前用户，再在应用层做兜底校验（source 前缀 + owner_id 二次确认）。双层防护。

Q：为什么要有应用层兜底？ 答：因为依赖第三方库（Chroma）的 filter 行为有不确定性（版本兼容、边界情况）。安全隔离不能只信一层——即使 filter 失效，第二层校验仍能挡住越权访问。这是纵深防御。

Q：系统语料和用户上传在同一个向量库，会互相污染吗？ 答：不会。检索时用 source_type 过滤：系统检索 where={"source_type": "system"}，用户检索 where={"source_type": "user_upload", "owner_id": ...}。物理共存、逻辑隔离。

4.9 test_set.py — 评测测试集

完整代码

```
"""测试集：代码检索问题与期望结果"""

TEST_SET = [
    {
        "query": "TinyDB 如何初始化和默认表？",
        "expected_sources": ["tinydb/database.py"],
    },
    {
        "query": "TinyDB 支持哪些存储后端，如何配置？",
        "expected_sources": ["tinydb/storages.py"],
    },
    {
        "query": "Query 模块如何实现字段查找和比较？",
        "expected_sources": ["tinydb/queries.py"],
    },
    {
        "query": "TinyDB 中如何创建、缓存和访问表？",
        "expected_sources": ["tinydb/table.py"],
    },
    {
        "query": "TinyDB 的中间件机制是如何处理操作链的？",
        "expected_sources": ["tinydb/middlewares.py"],
    },
    {
        "query": "TinyDB 在存储层如何保证原子写入？",
        "expected_sources": ["tinydb/storages.py"],
    },
    {
        "query": "有哪些查询操作可以检查元素是否在列表中？",
        "expected_sources": ["tinydb/queries.py"],
    },
]
```

```

    "query": "Table.insert 方法如何工作，返回什么？ ",
    "expected_sources": ["tinydb/table.py"],
  },
  {
    "query": "TinyDB 如何处理 JSON 序列化来存储文档？ ",
    "expected_sources": ["tinydb/storages.py", "tinydb/database.py"],
  },
  {
    "query": "Storage 基类的作用是什么，如何继承它实现自定义存储？ ",
    "expected_sources": ["tinydb/storages.py"],
  },
  {
    "query": "TinyDB 如何实现逻辑 OR 和 AND 组合查询？ ",
    "expected_sources": ["tinydb/queries.py"],
  },
  {
    "query": "Table 类如何处理文档 ID 和 ID 计数器？ ",
    "expected_sources": ["tinydb/table.py"],
  },
  {
    "query": "TinyDB 如何保证线程安全或处理并发访问？ ",
    "expected_sources": ["tinydb/database.py", "tinydb/storages.py"],
  },
  {
    "query": "operations 模块如何处理更新数组字段？ ",
    "expected_sources": ["tinydb/operations.py"],
  },
  {
    "query": "Query 如何实现 `one()` 方法只返回一条结果？ ",
    "expected_sources": ["tinydb/queries.py", "tinydb/table.py"],
  },
  {
    "query": "TinyDB 的 Table 和 Database 有什么区别？ ",
    "expected_sources": ["tinydb/table.py", "tinydb/database.py"],
  },
  {
    "query": "TinyDB 的 Query 模块中 `all()` 方法如何使用？ ",
    "expected_sources": ["tinydb/queries.py"],
  },
  {
    "query": "TinyDB 中如何实现自定义存储后端？ ",
    "expected_sources": ["tinydb/storages.py"],
  },
  {
    "query": "TinyDB 中的 JSONStorage 是如何工作的？ ",
    "expected_sources": ["tinydb/storages.py"],
  },
  {
    "query": "如何向 TinyDB 中插入数据并返回 ID？ ",
    "expected_sources": ["tinydb/table.py"],
  },
]

```

这段代码解决什么问题

评估检索效果的“标准答案”。20 条中文问题，每条标注了“这个问题应该命中哪些源文件”。这是所有评估指标（Hit Rate / MRR / NDCG）的基础——没有测试集，就没有评估；没有评估，就没有“我的检索到底好不好”的答案。

设计上几个要点：- 问题覆盖 TinyDB 核心模块：database、storages、queries、table、middlewares、operations——每个模块都有对应问题，保证评估的全面性。 -

expected_sources 是“期望命中的源文件列表”，有些问题有多个期望源（如“JSON 序列化”同时期望 storages 和 database）——因为答案可能跨文件。- 问题是**中文自然语言**，不是代码关键词——这正是考验 RAG 的地方：语义检索能否把中文问题匹配到英文代码。

面试题要点

Q：测试集怎么设计的？ 答：覆盖知识库（TinyDB）的每个核心模块，每个模块配 1-3 个问题；期望源文件标注到具体文件。问题用中文自然语言描述，考察检索的语义理解能力，而不是简单关键词匹配。

Q：指标是什么？ 答：Hit Rate（前 k 个结果里有没有期望文件）、MRR（第一个相关结果的倒数排名）、NDCG（考虑排序质量的折损累计增益）。详见 evaluation.py。

4.10 evaluation.py — 评估引擎与消融实验

完整代码

```
"""评估引擎：测试集评估 + 指标计算 + 消融实验"""

import time
import asyncio
import logging

from app.rag.test_set import TEST_SET
from app.rag.dense_retriever import DenseRetriever
from app.rag.sparse_retriever import SparseRetriever
from app.rag.fusion import hybrid_search, rrf
from app.rag.reranker import Reranker
from app.rag.query_rewriter import rewrite
from app.logger import log
from functools import partial

def _normalize(source: str) -> str:
    """统一路径分隔符：Windows 反斜杠 → 正斜杠"""
    return source.replace("\\", "/")

def hit_rate(results: list[dict], expected_sources: list[str], k: int = 5) -> float:
    """Hit Rate@k: 前 k 个结果中是否包含至少一个期望的源文件"""
    top_k = results[:k]
    actual_sources = {_normalize(r["source"]) for r in top_k}
    for expected in expected_sources:
        if any(_normalize(expected) in src for src in actual_sources):
            return 1.0
    return 0.0

def mrr(results: list[dict], expected_sources: list[str], k: int = 5) -> float:
    """MRR@k: 第一个相关结果的倒数排名"""
    for rank, r in enumerate(results[:k]):
        src = _normalize(r["source"])
        for expected in expected_sources:
            if _normalize(expected) in src:
                return 1.0 / (rank + 1)
    return 0.0
```

```

def ndcg(results: list[dict], expected_sources: list[str], k: int = 5) -> float:
    """NDCG@k: 归一化折损累积增益 (二值相关性) """
    dcg = 0.0
    matched = set()
    for i, r in enumerate(results[:k]):
        src = _normalize(r["source"])
        rel = 0.0
        for e in expected_sources:
            e_norm = _normalize(e)
            if e_norm in src and e_norm not in matched:
                rel = 1.0
                matched.add(e_norm)
                break
        dcg += rel / (i + 1)

    idcg = 0.0
    for i in range(min(k, len(expected_sources))):
        idcg += 1.0 / (i + 1)

    return dcg / idcg if idcg > 0 else 0.0

```

```

async def evaluate_config(
    dense: bool = True,
    sparse: bool = True,
    hyde: bool = False,
    reranker: bool = False,
    top_k: int = 5,
) -> dict:
    """

```

运行一轮评估，返回聚合指标

参数控制消融实验：关掉某个组件就看指标怎么掉

```

    """
    dense_retriever = DenseRetriever()
    sparse_retriever = SparseRetriever()
    reranker_instance = Reranker() if reranker else None

```

```

    total_hit_rate = 0.0
    total_mrr = 0.0
    total_ndcg = 0.0
    total_latency = 0.0
    n = len(TEST_SET)

```

```

    for item in TEST_SET:
        query = item["query"]
        expected = item["expected_sources"]
        start = time.time()

```

```

        # 第一步：查询改写 (HyDE)
        if hyde:
            query = await rewrite(query, strategy="hyde")

```

第二步：检索（直接同步调用，不走线程池）

```

        dense_results = []
        sparse_results = []

```

```

        if dense_retriever:
            dense_results = dense_retriever.search(query, top_k * 2)

```

```

        if sparse_retriever:
            sparse_results = sparse_retriever.search(query, top_k * 2)

```

第三步：融合

```

        if dense_results and sparse_results:
            fused = rrf([dense_results, sparse_results], top_n=top_k * 2)
        elif dense_results:

```

```

        fused = dense_results[:top_k * 2]
    elif sparse_results:
        fused = sparse_results[:top_k * 2]
    else:
        fused = []

    # 第四步：重排序
    if reranker_instance and fused:
        fused = reranker_instance.rerank(query, fused, top_n=top_k)
    else:
        fused = fused[:top_k]

    latency = time.time() - start
    total_latency += latency

    # 单条日志：谁 + 搜到什么 + 得分
    log.info(
        "[%s] Q: %s... → hits=%d | HR=%.2f MRR=%.2f NDCG=%.2f | %.0fms",
        "HYDE" if hyde else "DIRECT",
        query[:40],
        len(fused),
        hit_rate(fused, expected, k=top_k),
        mrr(fused, expected, k=top_k),
        ndcg(fused, expected, k=top_k),
        latency * 1000,
    )

    # 计算指标
    total_hit_rate += hit_rate(fused, expected, k=top_k)
    total_mrr += mrr(fused, expected, k=top_k)
    total_ndcg += ndcg(fused, expected, k=top_k)

    return {
        "hit_rate": round(total_hit_rate / n, 4),
        "mrr": round(total_mrr / n, 4),
        "ndcg": round(total_ndcg / n, 4),
        "avg_latency_ms": round(total_latency / n * 1000, 2),
        "test_set_size": n,
        "config": {
            "dense": dense,
            "sparse": sparse,
            "hyde": hyde,
            "reranker": reranker,
        },
    }
}

```

```

async def run_ablation(experiment_name: str = "ablation_v1"):
    """

```

跑全部消融实验组合，写入 evaluation_runs 表

```

    """
    configs = [
        {"dense": True, "sparse": False, "hyde": False, "reranker": False},
        {"dense": False, "sparse": True, "hyde": False, "reranker": False},
        {"dense": True, "sparse": True, "hyde": False, "reranker": False},
        {"dense": True, "sparse": True, "hyde": True, "reranker": False},
        {"dense": True, "sparse": True, "hyde": True, "reranker": True},
    ]

    names = [
        "dense_only",
        "sparse_only",
        "hybrid_baseline",
        "hybrid_with_hyde",
        "hybrid_with_hyde_and_reranker",
    ]

    results = []

```



```

for cfg, name in zip(configs, names):
    log.info("Running: %s", name)
    print(f"\n--- [{name}] ---")
    result = await evaluate_config(**cfg)
    results.append((name, result))

# 打印对比表
print(f"\n{'='*80}")
print(f"Experiment: {experiment_name}")
print(f"{'='*80}")
print(f"{'Config':<30} {'Hit Rate':<10} {'MRR':<10} {'NDCG':<10} {'Latency(ms)':<12}")
print(f"{'-'*72}")
for name, r in results:
    print(f"{name:<30} {r['hit_rate']:<10.4f} {r['mrr']:<10.4f} {r['ndcg']:<10.4f} {r['avg_latency_ms']:<12.2f}")
print(f"{'='*80}")

# 写入数据库
from app.database import async_session_factory
from app.models.evaluation_run import EvaluationRun

async with async_session_factory() as session:
    for name, r in results:
        session.add(EvaluationRun(
            run_name=f"{experiment_name}_{name}",
            config=r["config"],
            test_set_size=r["test_set_size"],
            hit_rate=r["hit_rate"],
            mrr=r["mrr"],
            ndcg=r["ndcg"],
            avg_latency_ms=r["avg_latency_ms"],
        ))
    await session.commit()

log.info("Ablation results saved to database")
return results

```

这段代码解决什么问题

用数据回答“我的检索到底好不好、哪个组件有用”。它实现了三个评估指标，并跑 5 组“消融实验”——每次关掉一个组件（比如不用 HyDE、不用 reranker），对比指标变化。

关键代码逐句拆解

① `_normalize` — 跨平台路径

```

def _normalize(source: str) -> str:
    return source.replace("\\", "/")

```

Windows 路径分隔符是 \，Linux 是 /。测试集里写的期望源是 `tinydb/database.py`（正斜杠），而 Windows 上索引出来的 source 可能是 `tinydb\\database.py`。归一化后两边才能比对。这是 CI（Linux）和本地（Windows）结果不一致的典型坑。

② `hit_rate` — 命中率

```

def hit_rate(results, expected_sources, k=5):
    top_k = results[:k]
    actual_sources = {_normalize(r["source"]) for r in top_k}
    for expected in expected_sources:
        if any(_normalize(expected) in src for src in actual_sources):

```

```

        return 1.0
    return 0.0

```

前 k 个结果里只要出现至少一个期望源文件，就算命中，返回 1.0；否则 0.0。注意用 `in`（子串匹配）而不是 `==`：因为期望是文件路径，实际 `source` 可能带路径前缀。

③ mrr — 平均倒数排名

```

def mrr(results, expected_sources, k=5):
    for rank, r in enumerate(results[:k]):
        src = _normalize(r["source"])
        for expected in expected_sources:
            if _normalize(expected) in src:
                return 1.0 / (rank + 1)
    return 0.0

```

第一个相关结果排第几？排第 1 $\rightarrow$ 1.0，排第 2 $\rightarrow$ 0.5，排第 3 $\rightarrow$ 0.33.....没找到 $\rightarrow$ 0。MRR 比 Hit Rate 更严格——Hit Rate 只关心“有没有”，MRR 关心“排多靠前”。

④ ndcg — 归一化折损累计增益

```

dcg += rel / (i + 1)    # 位置 i (0 基) 折损因子 1/(i+1)
...
idcg = sum(1/(i+1) for i in range(min(k, len(expected_sources))))
return dcg / idcg if idcg > 0 else 0.0

```

NDCG 是“排序质量”指标：相关文档排在前面 $\rightarrow$ 得分高；排得越靠后，贡献被折损得越多。用 IDCG（理想排序下的最大得分）归一化，让不同长度的查询可比。

细节：matched 集合保证同一个源文件出现多个 `chunk` 只算一次相关性——防止同一个文件刷分。

⑤ evaluate_config — 单组评估

```

async def evaluate_config(dense=True, sparse=True, hyde=False, reranker=False,
top_k=5):
    ...
    for item in TEST_SET:
        if hyde:
            query = await rewrite(query, strategy="hyde")
            ...
            dense_results = dense_retriever.search(query, top_k * 2)
            sparse_results = sparse_retriever.search(query, top_k * 2)
            ...
            fused = rrf([dense_results, sparse_results], top_n=top_k * 2)
            ...
            if reranker_instance and fused:
                fused = reranker_instance.rerank(query, fused, top_n=top_k)

```

参数控制组件开关——这就是“消融”：把每个组件当开关，关掉看指标怎么变。链路复现了在线检索：改写 $\rightarrow$ 双路召回 $\rightarrow$ RRF $\rightarrow$ 重排。

注意：召回 $\text{top}_k * 2$ （留余量），融合后 $\text{top}_k * 2$ ，重排/截断到 top_k 。

⑥ run_ablation — 5 组消融

```

configs = [

```

```

{"dense": True, "sparse": False, ...}, # dense_only
{"dense": False, "sparse": True, ...}, # sparse_only
{"dense": True, "sparse": True, ...}, # hybrid_baseline
{... "hyde": True, ...}, # hybrid + HyDE
{... "hyde": True, "reranker": True}, # hybrid + HyDE + rerank
]

```

跑完打印对比表，并写入 `evaluation_runs` 表。这就是 README 里那张消融表的来源：

配置	Hit Rate	MRR	NDCG	延迟(ms)
dense_only	0.95	0.60	0.60	13
sparse_only	0.95	0.60	0.60	0.3
hybrid_baseline	0.95	0.60	0.60	0.4
hybrid + HyDE	0.95	0.60	0.60	3600
hybrid + HyDE + rerank	0.85	0.49	0.48	3900

面试题要点

Q: Hit Rate、MRR、NDCG 分别衡量什么？ 答：Hit Rate 衡量“有没有找到”（前 k 个里是否含期望结果）；MRR 衡量“找得多靠前”（第一个相关结果的倒数排名）；NDCG 衡量“整体排序质量”（相关结果越靠前分越高，且用理想排序归一化）。

Q: 消融实验的结果说明了什么？ 答：基础混合检索 Hit Rate 已到 0.95，说明双路召回 + RRF 对这个场景已经足够；HyDE 无额外增量（问题太直白）；reranker 反而降分（模型在网页搜索领域训练，对代码不敏感）。结论：**组件必须按领域验证，不能无脑叠加。**

Q: 这个结论对你有何启发？ 答：面试可以主动讲——“我做过消融实验，发现不是组件越多越好。RAG 优化要根据数据和场景验证，这让我对‘加组件’有了更审慎的态度。”这比“我用了所有最先进的技术”更有说服力。

Q: 为什么路径要归一化？ 答：Windows 用反斜杠、Linux 用正斜杠，测试集里期望源是正斜杠。不归一化，同一结果在本地（Windows）和 CI（Linux）会判出不同命中。

第 4 层小结

RAG 管线的 10 个文件，按职能分三组：

- **在线链路**：chunker（分块）→ dense（语义）→ sparse（关键词）→ fusion（RRF 融合）→ rewriter（HyDE 改写）→ reranker（精排）。每个组件都是可开关的。
- **离线编排**：code_indexer（建索引 + 打 system 标签）、user_upload（用户上传 + owner 隔离）。
- **评估**：test_set（20 条标准答案）、evaluation（指标 + 消融）。

贯穿全层的思想：每个组件都有降级路径（模型挂了跳过、缓存挂了降级、改写失败返回原查询）、都有缓存（Redis）、都有日志（可观测）、都有数据支撑（消融实验）。这就是“工程化 RAG”和“demo RAG”的区别。

第 5 层 Agent：让模型自己决策

第 4 层的 RAG 是被动的——“你搜我就返回”。第 5 层的 Agent 是**主动的**：让 LLM 自己决定要不要搜、搜什么、搜完怎么答。这就是 ReAct（Reasoning + Acting）。

项目没有用 LangGraph，而是**手写了一个 ReAct 循环**，5 个文件：

llm.py	LLM 调用封装（DeepSeek / Ollama 可切换，重试 + 退避）
tool_base.py	工具抽象基类（所有工具继承它）
prompt.py	ReAct 系统提示词（工具描述动态注入）
tools.py	三个工具：search / explain / testgen
harness.py	ReAct 循环引擎（解析 Action JSON、执行工具、记日志）

Agent 的调用关系：harness 用 prompt 构造提示词 → 调 llm 拿 LLM 回复 → 从回复里解析出 Action → 在 tools 里找到对应工具执行 → 工具内部调用 RAG 检索（第 4 层）。

5.1 llm.py — LLM 调用封装

完整代码

```
"""LLM 调用封装：支持 Ollama（本地开发） / DeepSeek API（生产）"""

import time
import httpx
from app.config import settings
from app.logger import log

def _parse_response(response: dict) -> str:
    """兼容 Ollama 和 OpenAI 格式的响应解析"""
    # Ollama 格式: {"message": {"content": "..."}}
    if "message" in response:
        return response["message"].get("content", "")
    # OpenAI / DeepSeek 格式: {"choices": [{"message": {"content": "..."}}]}
    if "choices" in response and len(response["choices"]) > 0:
        return response["choices"][0].get("message", {}).get("content", "")
    log.error("Unknown response format: %s", str(response)[:200])
    return ""

def call_llm(prompt: str, system_prompt: str = "", max_retries: int = 2) -> str:
    """
    调用 LLM，返回文本回复

    通过配置 LLM_API_ENDPOINT 切换服务商：
        Ollama   : http://localhost:11434/api/chat
    """
```

```

        DeepSeek: https://api.deepseek.com/v1/chat/completions
"""
messages = []
if system_prompt:
    messages.append({"role": "system", "content": system_prompt})
messages.append({"role": "user", "content": prompt})

headers = {
    "Content-Type": "application/json",
}
if settings.LLM_API_KEY:
    headers["Authorization"] = f"Bearer {settings.LLM_API_KEY}"

for attempt in range(max_retries + 1):
    try:
        response = httpx.post(
            settings.LLM_API_ENDPOINT,
            json={
                "model": settings.LLM_MODEL,
                "messages": messages,
                "stream": False,
                "temperature": 0.3,
                "max_tokens": 512,
            },
            timeout=60,
        )
        response.raise_for_status()
        result = _parse_response(response.json())
        if result:
            return result

    except httpx.TimeoutException:
        log.warning("LLM timeout (attempt %d/%d)", attempt + 1, max_retries + 1)
    except httpx.HTTPStatusError as e:
        log.warning("LLM HTTP %s (attempt %d/%d)", e.response.status_code,
attempt + 1, max_retries + 1)
    except Exception as e:
        log.warning("LLM error: %s (attempt %d/%d)", e, attempt + 1, max_retries
+ 1)

    if attempt < max_retries:
        time.sleep(2 ** attempt)  # 指数退避: 1s → 2s

log.error("LLM call failed after %d retries", max_retries)
return ""

def call_llm_with_messages(messages: list[dict], max_retries: int = 2) -> str:
    """
    调用 LLM，传入完整消息列表（包含 system / user / assistant）

    用于多轮对话场景，保留全部上下文。
    """
    headers = {
        "Content-Type": "application/json",
    }
    if settings.LLM_API_KEY:
        headers["Authorization"] = f"Bearer {settings.LLM_API_KEY}"

    body = {
        "model": settings.LLM_MODEL,
        "messages": messages,
        "stream": False,
        "temperature": 0.3,
        "max_tokens": 1024,
    }

    for attempt in range(max_retries + 1):
        try:

```

```

        response = httpx.post(
            settings.LLM_API_ENDPOINT,
            json = body,
            headers=headers,
            timeout = 60,
        )
        response.raise_for_status()
        result = _parse_response(response.json())
        if result:
            return result

    except httpx.TimeoutException:
        log.warning("LLM timeout (attempt %d/%d)", attempt + 1, max_retries + 1)
    except httpx.HTTPStatusError as e:
        log.warning("LLM HTTP %s (attempt %d/%d)", e.response.status_code,
            attempt + 1, max_retries + 1)
    except Exception as e:
        log.warning("LLM error: %s (attempt %d/%d)", e, attempt + 1, max_retries
            + 1)

    if attempt < max_retries:
        time.sleep(2 ** attempt)

    log.error("LLM call failed after %d retries", max_retries)
    return ""

```

这段代码解决什么问题

统一所有 LLM 调用，解决三个问题：

1. **多服务商兼容**：DeepSeek（OpenAI 兼容格式）和 Ollama（本地格式）响应结构不同，`_parse_response` 统一解析。
2. **稳定性**：LLM 服务不稳定，超时/5xx 自动重试，指数退避。
3. **单轮 / 多轮两种场景**：`call_llm`（单 prompt + system）给 HyDE 改写、工具内部用；`call_llm_with_messages`（完整消息列表）给 ReAct 循环用。

关键代码逐句拆解

① `_parse_response` — 兼容两种格式

```

def _parse_response(response: dict) -> str:
    # Ollama 格式: {"message": {"content": "..."}}
    if "message" in response:
        return response["message"].get("content", "")
    # OpenAI / DeepSeek 格式: {"choices": [{"message": {"content": "..."}}]}
    if "choices" in response and len(response["choices"]) > 0:
        return response["choices"][0].get("message", {}).get("content", "")

```

Ollama 的 `/api/chat` 返回 `{"message": {"content": "..."}};` DeepSeek/OpenAI 返回 `{"choices": [{"message": {"content": "..."}}]}`。先判断 message（Ollama），再判断 choices（OpenAI）。靠配置切换服务商，代码零改动。

② 重试 + 指数退避

```

for attempt in range(max_retries + 1):
    try:
        response = httpx.post(...)
        response.raise_for_status()
        result = _parse_response(response.json())

```

```

        if result:
            return result
    except httpx.TimeoutException:
        log.warning(...)
    ...
    if attempt < max_retries:
        time.sleep(2 ** attempt)  # 1s → 2s

```

- `httpx.post(...)` 同步请求（配合 `asyncio.to_thread` 用）。
- `raise_for_status()`：非 2xx 抛 `HTTPStatusError`。
- 超时、HTTP 错误、其他异常都进重试。
- **指数退避 $2 ** attempt$** ：第 1 次失败等 1s，第 2 次等 2s。避免雪崩式重试打爆下游。
- `max_tokens`：单轮 512，多轮 1024（ReAct 循环上下文更长）。
- `temperature=0.3`：偏低，让输出更确定——**Agent 的工具调用需要稳定的 JSON 输出，不能太“有创意”**。

③ 两个函数的关系

```

def call_llm(prompt, system_prompt="", max_retries=2) -> str:
    messages = []
    if system_prompt:
        messages.append({"role": "system", "content": system_prompt})
    messages.append({"role": "user", "content": prompt})

def call_llm_with_messages(messages, max_retries=2) -> str:
    # 直接把完整 messages 透传

```

`call_llm` 是“组装式”（帮你拼 system + user），`call_llm_with_messages` 是“透传式”（你自己给完整列表）。**ReAct 循环必须用后者**——因为它要传系统提示词 + 历史消息 + 当前问题 + 之前的 Observation 链。

面试题要点

Q：怎么支持多个 LLM 服务商？ 答：通过配置 `LLM_API_ENDPOINT` 切换，代码不改。关键是 `_parse_response` 兼容两种响应格式（Ollama 的 `message`、OpenAI 的 `choices`）。

Q：为什么要有重试和退避？ 答：LLM 服务不稳定（超时、5xx）。重试 + 指数退避（1s → 2s）提高成功率，同时避免并发重试把服务打爆。退避防止“雪崩式重试”。

Q：temperature 为什么设 0.3？ 答：Agent 需要稳定输出 JSON（Action），低 temperature 让输出更确定、更少 JSON 格式错误。创意性任务才需要高 temperature。

Q：call_llm 和 call_llm_with_messages 区别？ 答：前者单轮（自动拼 system+user），用于 HyDE 改写和工具内部；后者透传完整消息列表，用于 ReAct 循环（需要保留系统提示词 + 历史 + 观察结果链）。

5.2 tool_base.py — 工具抽象基类

完整代码

```
"""工具基类：所有 Agent 工具都继承这个"""

from abc import ABC, abstractmethod

class Tool(ABC):
    name: str = ""
    description: str = ""
    parameters: dict = {}

    @abstractmethod
    def execute(self, **kwargs) -> str:
        """执行工具，返回文本结果"""
        pass
```

这段代码解决什么问题

定义“工具长什么样”。所有 Agent 工具（search / explain / testgen）都继承 Tool，必须实现 execute。

ABC + @abstractmethod：Python 抽象基类。实例化 Tool() 会直接报错，强制子类实现 execute。这就保证了“每个工具都能被执行”。

name / description / parameters 是“元数据”：它们不会被执行，但会被注入系统提示词（见 prompt.py），告诉 LLM“有这个工具、它干嘛的、参数是什么”。**工具描述即接口——LLM 靠文本描述学会调用工具。**

面试题要点

Q：为什么工具要有 description 和 parameters？ 答：因为 LLM 看不到 Python 代码，只能看文本。build_tool_descriptions 把 name + description + parameters 渲染成文本注入系统提示词，LLM 才知道什么时候调哪个工具、传什么参数。**工具的“接口”对 LLM 来说就是这段描述文本。**

Q：为什么用抽象基类？ 答：强制统一接口（每个工具都必须实现 execute），同时保证所有工具都带 name/description/parameters 元数据，让工具注册制（新增工具自动注入提示词）成为可能。

5.3 prompt.py — ReAct 系统提示词

完整代码

```
"""ReAct 系统提示词"""

REACT_SYSTEM_PROMPT = """你是 TinyDB 项目的代码分析助手。你可以使用以下工具：

{tool_descriptions}
```


你按照"思考-行动-观察"循环工作：

思考：分析当前情况，决定下一步做什么。

行动：`{{"name": "工具名", "args": {"参数名": "值"}}}`

观察：工具返回的结果（自动显示给你）

... 如果需要，重复以上步骤 ...

思考：我已经获得了足够的信息。

答案：<你的最终回答>

规则：

- 只有在需要调用工具时才输出 Action JSON
- 不要编造工具结果，始终使用 Observation 里的内容
- 如果不需要调用任何工具，直接回答即可
- 最终回答不需要 "答案：" 前缀，直接写就行
- 用中文回答

示例：

用户：TinyDB 怎么存储数据的？

助手：

思考：先搜一下存储相关的代码。

行动：`{{"name": "search", "args": {"query": "TinyDB 存储 JSON 文件"}}}`

观察：`[storages.py]` ...（工具返回的内容）

思考：找到 `JSONStorage` 类了，可以解释一下。

答案：TinyDB 使用 `JSONStorage` 类将数据以 JSON 格式存储在单个文件中。当执行 `insert/update` 等写操作时，整个数据库会被序列化并覆写文件。"""

```
def build_tool_descriptions(tools: list) -> str:
    """从工具列表生成描述文本"""
    lines = []
    for t in tools:
        params_desc = ", ".join(f"{k}: {v}" for k, v in t.parameters.items())
        lines.append(f"- {t.name}: {t.description} | 参数: {{{params_desc}}}")
    return "\n".join(lines)
```

这段代码解决什么问题

这是 Agent 的“操作手册”——告诉 LLM 它是个代码助手、有哪些工具、按什么格式工作、有哪些规矩。提示词写得越清楚，Agent 行为越可控。

关键代码逐句拆解

① 提示词的骨架：思考-行动-观察模板

思考：分析当前情况，决定下一步做什么。

行动：`{"name": "工具名", "args": {"参数名": "值"}}`

观察：工具返回的结果（自动显示给你）

这是 ReAct 的核心格式约定：**LLM 输出 Action JSON → 系统执行工具 → 把 Observation 注入下一轮 → LLM 再思考**。提示词用双花括号 `{{"name": ...}}` 转义——因为后面要 `.format()`，单花括号会被当成格式化占位符。

② 规则部分 — 防幻觉和死循环

- 只有在需要调用工具时才输出 Action JSON
- 不要编造工具结果，始终使用 Observation 里的内容
- 如果不需要调用任何工具，直接回答即可
- 最终回答不需要 "答案:" 前缀，直接写就行
- 用中文回答

这些规则专门针对 LLM 的常见问题：- “不要编造工具结果”——防幻觉，答案必须基于真实检索结果。- “不需要就不调用”——防止 LLM 过度调用工具。- “不用答案前缀”——简化解析，最终回答直接是文本。

③ 示例 (few-shot)

提示词里给了完整的对话示例：用户问 → 思考 → 行动 → 观察 → 答案。**few-shot** 示例能让 LLM 稳定地输出符合格式的 Action JSON——这对后面 harness 的解析器至关重要。

④ build_tool_descriptions — 动态注入

```
def build_tool_descriptions(tools: list) -> str:
    lines = []
    for t in tools:
        params_desc = ", ".join(f"{k}: {v}" for k, v in t.parameters.items())
        lines.append(f"- {t.name}: {t.description} | 参数: {{{params_desc}}}")
    return "\n".join(lines)
```

把工具列表渲染成文本，填充 REACT_SYSTEM_PROMPT 里的 {tool_descriptions} 占位符。
新增一个工具 → 加一个类 → 提示词自动更新。这就是“工具注册制”。

面试题答题要点

Q: 提示词里为什么有 few-shot 示例？ 答：LLM 对格式不敏感，不加示例经常输出不合规的 Action（没 JSON、多了前缀）。给一个完整示例能显著提高格式稳定性，让下游解析器（harness 的括号深度解析）更容易命中。

Q: {{{"name": ...}}} 为什么用双花括号？ 答：提示词模板里用 .format() 注入 {tool_descriptions}，而 Action 示例本身要输出字面量 {"name": ...}——单花括号会被 format 当作占位符，所以必须转义成双花括号。

Q: 规则里“不要编造工具结果”想防什么？ 答：防 LLM 幻觉。LLM 可能在没检索的情况下自己“脑补”代码库内容。规则强制它基于 Observation（真实工具返回）作答，这是 RAG 防幻觉的核心。

Q: 新增一个工具要改什么？ 答：只需在 tools.py 加一个继承 Tool 的类，实现 name/description/parameters/execute。build_tool_descriptions 自动把它注入提示词，harness 的 tool_map 自动注册。这是“开闭原则”——对扩展开放、对修改关闭。

5.4 tools.py — 三个 RAG 工具

完整代码

```
"""Agent 工具: search / explain / testgen"""

import json
from app.agent.tool_base import Tool
from app.rag.dense_retriever import DenseRetriever, SYSTEM_CORPUS
from app.rag.sparse_retriever import SparseRetriever
from app.rag.fusion import rrf
from app.rag.reranker import Reranker
from app.agent.llm import call_llm

class SearchCode(Tool):
    name = "search"
    description = "在 TinyDB 代码库中搜索与查询最相关的代码片段。支持混合检索（向量+关键词），返回 top-k 结果。"
    parameters = {"query": "搜索查询，用英文描述你要找的代码功能"}

    def execute(self, **kwargs) -> str:
        query = kwargs.get("query" , "")
        if not query:
            return "Error: query is required"

        dense = DenseRetriever()
        sparse = SparseRetriever.from_redis()

        dense_results = dense.search(query, k=10, where=SYSTEM_CORPUS)
        sparse_results = sparse.search(query, k=10)

        if dense_results and sparse_results:
            fused = rrf([dense_results, sparse_results], top_n=5)
        elif dense_results:
            fused = dense_results[:5]
        elif sparse_results:
            fused = sparse_results[:5]
        else:
            return "No results Found"

        output = []
        for r in fused:
            text = r["text"][:300]
            output.append(f"[{r['source']}]\\n{text}\\n")
        return "\\n---\\n".join(output)

class ExplainCode(Tool):
    name = "explain"
    description = "解释 TinyDB 中指定函数或类的功能。结合检索到的代码上下文和 LLM 生成自然语言解释。"
    parameters = {"target": "要解释的函数名或类名，如 'TinyDB' 或 'search'"}

    def execute(self, **kwargs) -> str:
        target = kwargs.get("target" , "")
        if not target:
            return "Error: target is required"

        dense = DenseRetriever()
        results = dense.search(target, k=3, where=SYSTEM_CORPUS)
        if not results:
            return f"Could not find code related to '{target}'"

        context = "\\n\\n".join([r["text"][:400] for r in results])
        prompt = f"解释下面代码中 '{target}' 的功能: \\n\\n{context}"
```

```

        explanation = call_llm(prompt, system_prompt="你是一个 Python 代码导师，用中文简洁解释代码功能，突出重点。")
        return explanation

class GenerateTest(Tool):
    name = "testgen"
    description = "为 TinyDB 中的指定函数或类生成单元测试代码。先用 search 检索相关代码，再用 LLM 生成 pytest 测试用例。"
    parameters = {"target": "要生成测试的目标函数名或类名，如 'table.search'"}

    def execute(self, **kwargs) -> str:
        target = kwargs.get("target", "")
        if not target:
            return "Error: target is required"

        dense = DenseRetriever()
        results = dense.search(target, k=3, where=SYSTEM_CORPUS)
        if not results:
            return f"Could not find code related to '{target}'"

        context = "\n\n".join([r["text"][:500] for r in results])
        prompt = f"为以下代码中的 '{target}' 编写 pytest 单元测试：\n\n{context}\n\n只输出测试代码，不要解释。"
        test_code = call_llm(prompt, system_prompt="你是一个 Python 测试工程师，输出可直接运行的 pytest 代码。")
        return test_code

AVAILABLE_TOOLS = [SearchCode(), ExplainCode(), GenerateTest()]

```

这段代码解决什么问题

三个工具，覆盖 Agent 的核心能力：**搜代码（search）**、**解释代码（explain）**、**生成测试（testgen）**。每个工具都是“RAG 检索 + LLM 加工”的组合——检索到相关代码后，让 LLM 干加工活。

关键代码逐句拆解

① SearchCode — 混合检索

```

def execute(self, **kwargs) -> str:
    query = kwargs.get("query", "")
    if not query:
        return "Error: query is required"

    dense = DenseRetriever()
    sparse = SparseRetriever.from_redis()

    dense_results = dense.search(query, k=10, where=SYSTEM_CORPUS)
    sparse_results = sparse.search(query, k=10)

    if dense_results and sparse_results:
        fused = rrf([dense_results, sparse_results], top_n=5)
    elif dense_results:
        fused = dense_results[:5]
    elif sparse_results:
        fused = sparse_results[:5]
    else:
        return "No results Found"

```

- 参数校验：query 为空直接返回错误文本。

- **混合检索**：dense（限定系统语料）+ sparse 各取 top-10，RRF 融合 top-5。
- **分支兜底**：两路都有→RRF；只有一路→直接取；都没有→“No results Found”。
- 输出是给 **LLM** 看的格式化文本：

```
output = []
for r in fused:
    text = r["text"][:300]
    output.append(f"[{r['source']}]\\n{{text}}\\n")
return "\\n---\\n".join(output)
```

每条带源文件路径 [tinydb/storages.py]，内容截断 300 字符——**Observation** 是给 LLM 看的，太长浪费 token。

② ExplainCode — 检索 + 解释

```
results = dense.search(target, k=3, where=SYSTEM_CORPUS)
context = "\\n\\n".join([r["text"][:400] for r in results])
prompt = f"解释下面代码中 '{target}' 的功能：\\n\\n{{context}}"
explanation = call_llm(prompt, system_prompt="你是一个 Python 代码导师，...")
return explanation
```

检索 top-3 代码片段拼成 context，让 LLM 用中文解释。**RAG 的核心模式**：把检索到的上下文喂给 LLM，让回答基于真实代码。

③ GenerateTest — 检索 + 写测试

```
prompt = f"为以下代码中的 '{target}' 编写 pytest 单元测试：\\n\\n{{context}}\\n\\n只输出测试代码，不要解释。"
test_code = call_llm(prompt, system_prompt="你是一个 Python 测试工程师，输出可直接运行的 pytest 代码。")
```

同样的模式，只是 prompt 换成写测试。

④ AVAILABLE_TOOLS = [SearchCode(), ExplainCode(), GenerateTest()]

模块级实例化列表。harness 用它做工具注册：

```
self.tools = tools or AVAILABLE_TOOLS
self.tool_map = {t.name: t for t in self.tools}
```

面试题要点

Q：Agent 工具内部为什么复用 RAG？ 答：工具就是“能力封装”。search 工具内部就是混合检索（dense + BM25 + RRF），explain/testgen 是“检索 + LLM 加工”。Agent 不重复实现检索，而是把 RAG 当作工具来调用——这正是“Agent 编排能力、RAG 提供能力”的分层。

Q：工具返回结果为什么是文本而不是结构化？ 答：因为 Observation 要放回消息列表给 LLM 看，文本是 LLM 最自然的输入。截断 300/400/500 字符控制 token 消耗。

Q：为什么每次 execute 都新建 DenseRetriever？ 答：Chroma 是持久化的，重新加载开销可接受；且避免单例的线程安全问题（Agent 可能并发跑）。简洁但略慢——这也

是改进点之一（见第 10 章）。

5.5 harness.py — ReAct 循环引擎（全项目核心）

完整代码

```
"""Agent Harness: 工具注册 + ReAct 循环 + 执行轨迹记录"""

import json
import asyncio
from typing import Optional
from app.agent.llm import call_llm_with_messages
from app.agent.prompt import REACT_SYSTEM_PROMPT, build_tool_descriptions
from app.agent.tools import AVAILABLE_TOOLS
from app.logger import log

def _extract_json_action(text: str) -> Optional[dict]:
    """从文本中提取第一个包含 'name' 字段的有效 JSON 对象"""
    depth = 0
    start = -1
    for i, ch in enumerate(text):
        if ch == "{":
            if depth == 0:
                start = i
            depth += 1
        elif ch == "}":
            depth -= 1
            if depth == 0 and start >= 0:
                try:
                    obj = json.loads(text[start:i+1])
                    if "name" in obj:
                        return obj
                except json.JSONDecodeError:
                    pass
            start = -1
    return None

class AgentHarness:
    def __init__(self, tools: list = None, session_id: str = "default"):
        self.tools = tools or AVAILABLE_TOOLS
        self.tool_map = {t.name: t for t in self.tools}
        self.history = []
        self.session_id = session_id
        self.conversation_history = [] # 多轮对话: 保存历史 Q&A

    def restore_history(self, history: list[dict]):
        """从数据库恢复对话历史"""
        self.conversation_history = history

    async def _log_step(self, step: int, thought: str,
                       action_name: str = None, action_args: str = None,
                       observation: str = None):
        """异步写 Agent 决策日志到数据库"""
        try:
            from app.database import async_session_factory
            from app.models.agent_log import AgentLog

            async with async_session_factory() as session:
                session.add(AgentLog(
                    session_id=self.session_id,
```

```

        step_number=step,
        thought=thought[:500],
        action_name=action_name,
        action_args=action_args,
        observation=observation[:200] if observation else None,
    ))
    await session.commit()
except Exception as e:
    log.warning("Failed to log agent step: %s", e)

def _schedule_log(self, step: int, thought: str,
                  action_name: str = None, action_args: str = None,
                  observation: str = None):
    """在合适的上下文里调度异步日志：有事件循环用 create_task，否则 asyncio.run"""
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        # 后台线程，没有运行中的事件循环 → 独立跑
        asyncio.run(self._log_step(
            step=step, thought=thought,
            action_name=action_name, action_args=action_args,
            observation=observation,
        ))
    else:
        loop.create_task(self._log_step(
            step=step, thought=thought,
            action_name=action_name, action_args=action_args,
            observation=observation,
        ))

def get_tool_description(self) -> str:
    return build_tool_descriptions(self.tools)

def execute_tool(self, name: str, args: dict) -> str:
    tool = self.tool_map.get(name)
    if not tool:
        return f"Error: unknown tool '{name}' "
    log.info(" Tool call : %s(%s)", name, json.dumps(args))
    result = tool.execute(**args)
    log.info(" Tool result: %s...", result[:80].replace("\n", " "))
    return result

def run(self, user_input: str, max_step: int = 6) -> str:
    system_prompt = REACT_SYSTEM_PROMPT.format(
        tool_descriptions=self.get_tool_description()
    )

    # 构建 messages: 保留历史上下文，追加新问题
    messages = [{"role": "system", "content": system_prompt}]
    for msg in self.conversation_history:
        messages.append(msg)
    messages.append({"role": "user", "content": user_input})

    for step in range(max_step):
        log.info("Step %d: calling LLM...", step + 1)

        response = call_llm_with_messages(messages)

        # 检查 LLM 是否返回空
        if not response:
            return "Error: LLM failed to respond after retries. Please check the
API."

        messages.append({"role": "assistant", "content": response})
        self.history.append({"step": step, "response": response})

    # 检查是否包含 Action JSON (支持嵌套花括号)

```

```

        action = _extract_json_action(response)
        if action:
            try:
                obs = self.execute_tool(action["name"], action.get("args", {}))
                messages.append({"role": "user", "content": f"Observation: {obs}"})

                # 记录决策日志（异步，不阻塞循环）
                self._schedule_log(
                    step=step + 1,
                    thought=response,
                    action_name=action["name"],
                    action_args=json.dumps(action.get("args", {})),
                    observation=obs,
                )

                continue
            except (json.JSONDecodeError, KeyError) as e:
                messages.append({"role": "user", "content": f"Observation: JSON parse error: {e}"})

                self._schedule_log(
                    step=step + 1, thought=response,
                    action_name="parse_error",
                    action_args=str(e),
                    observation=None,
                )

                continue

        # 没有 Action JSON → 认为是最终答案
        self._schedule_log(
            step=step + 1, thought=response,
            action_name="answer", action_args=None,
            observation=None,
        )

        # 保存本轮问答到历史（供下一轮使用）
        self.conversation_history.append({"role": "user", "content": user_input})
        self.conversation_history.append({"role": "assistant", "content":
response.strip()})

        return response.strip()

    return "Max steps reached. Please refine your question."

```

这段代码解决什么问题

这是手写 **ReAct** 的核心。它把“思考-行动-观察”循环串起来：

- ① 把系统提示词 + 历史 + 用户问题组装成 messages
- ② 调 LLM → 得到回复
- ③ 解析回复里有没有 Action JSON
 - ├ 有 → 执行工具 → 把 Observation 注入 messages → 回到②
 - └ 没有 → 认为回复是最终答案 → 返回
- ④ 超过 max_step（6 步）→ 返回“达到步数上限”

关键代码逐句拆解

① `_extract_json_action` — 手写括号深度解析（亮点）

```
def _extract_json_action(text: str) -> Optional[dict]:
```



```

depth = 0
start = -1
for i, ch in enumerate(text):
    if ch == "{":
        if depth == 0:
            start = i
        depth += 1
    elif ch == "}":
        depth -= 1
        if depth == 0 and start >= 0:
            try:
                obj = json.loads(text[start:i+1])
                if "name" in obj:
                    return obj
            except json.JSONDecodeError:
                pass
            start = -1
return None

```

为什么不用简单正则 `\{.*\}` 匹配？ 因为 LLM 输出里的 JSON 可能**嵌套花括号**（比如 args 里又有个 dict），正则的贪婪/非贪婪匹配都会出错。这个手写解析器用深度计数：

- { 加深度、} 减深度
- 深度归零的时刻 = 一个完整 JSON 对象的边界
- 在边界处尝试 `json.loads`，成功且含 name 字段就返回

这样即使 `{"name": "search", "args": {"query": "TinyDB", "nested": {"a": 1}}}` 这种嵌套结构也能完整提取。**优雅、可控、无依赖。**

注意 LLM 可能在回复里混入其他文本（“好的，我来搜索：{...}”），这个函数扫描全文找第一个有效 JSON——**容错**。

② run — 组装消息列表

```

messages = [{"role": "system", "content": system_prompt}]
for msg in self.conversation_history:
    messages.append(msg)
messages.append({"role": "user", "content": user_input})

```

`conversation_history` 只存**上一轮的最终问答**（user + assistant），不含工具调用过程。这样多轮对话时不会把一大堆 Observation 灌给下一轮，省 token。

③ 主循环

```

for step in range(max_step):
    response = call_llm_with_messages(messages)
    if not response:
        return "Error: LLM failed to respond after retries..."
    messages.append({"role": "assistant", "content": response})
    action = _extract_json_action(response)

    if action:
        try:
            obs = self.execute_tool(action["name"], action.get("args", {}))
            messages.append({"role": "user", "content": f"Observation: {obs}"})
            self._schedule_log(...)
            continue
        except (json.JSONDecodeError, KeyError) as e:
            messages.append({"role": "user", "content": f"Observation: JSON parse error: {e}"})

```

continue

...

关键设计：

- **LLM 输出作为 assistant 消息**，工具结果作为 user 消息 Observation: ...——这就是把“观察”注入上下文的标准做法。
- 解析出 Action → 执行工具 → 观察注入 → continue 下一轮。
- **解析异常兜底**：KeyError（缺 name）或 JSON 解析失败时，把错误信息作为 Observation 注入，让 LLM 自己纠正——而不是直接崩溃。
- max_step=6 上限，防死循环。

④ 最终答案判定

```
# 没有 Action JSON → 认为是最终答案
self._schedule_log(step=step+1, thought=response, action_name="answer", ...)
self.conversation_history.append({"role": "user", "content": user_input})
self.conversation_history.append({"role": "assistant", "content": response.strip()})
return response.strip()
```

LLM 的回复里没有 **Action JSON** 就当**作最终答案**。同时把本轮问答存进 conversation_history，供下一轮恢复。

⑤ _schedule_log — 异步日志的“上下文自适应”

```
def _schedule_log(self, step, thought, action_name=None, action_args=None,
observation=None):
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        asyncio.run(self._log_step(...))      # 后台线程：没有事件循环，独立跑
    else:
        loop.create_task(self._log_step(...)) # 有事件循环：create_task
```

Agent 可能在**两个地方**跑：- 通过 API（FastAPI 的异步上下文里，有事件循环）→ create_task - 通过脚本/测试（后台线程，没有事件循环）→ asyncio.run

get_running_loop() 探测当前有没有事件循环，分别处理。这样日志逻辑在两种场景都能工作，不会因“没有事件循环”报错。

_log_step 里 thought[:500]、observation[:200] 截断，防止日志表无限膨胀。

⑥ execute_tool — 工具调度 + 日志

```
def execute_tool(self, name: str, args: dict) -> str:
    tool = self.tool_map.get(name)
    if not tool:
        return f"Error: unknown tool '{name}' "
    result = tool.execute(**args)
    return result
```

tool_map 是 {name: tool} 字典（构造时 {t.name: t for t in self.tools}）。未知工具名返回错误文本（LLM 会看到并纠正）。

面试题要点

Q: ReAct 循环怎么实现的? 答: 循环最多 N 步。每轮把系统提示词 + 历史 + 当前问题组装成 messages 调 LLM; 解析回复里的 Action JSON; 有 Action 就执行工具、把 Observation 作为 user 消息注入、continue; 没有 Action 就当最终答案返回。上限防止死循环。

Q: 怎么从 LLM 输出里安全解析 JSON? 答: 手写括号深度计数器——{ 加深度、} 减深度, 深度归零即完整 JSON 边界, 在边界处 json.loads。比正则可靠, 能处理嵌套花括号, 还能容忍 LLM 输出前后夹杂其他文本。

Q: LLM 输出不合规怎么办? 答: 解析失败或缺 name 字段时, 把 “JSON parse error” 作为 Observation 注入消息列表, 让 LLM 自己看到错误并重新输出。这是容错设计——不崩溃、给模型纠正的机会。

Q: 为什么 conversation_history 只存最终问答、不存工具调用过程? 答: 多轮对话时, 上一轮的工具调用 Observation 链对回答当前问题没有帮助, 还浪费大量 token。只保留最终 Q&A 是 “省 token 的历史压缩”。

Q: Agent 日志怎么做到异步不阻塞? 答: _schedule_log 探测当前有无事件循环: 有则 loop.create_task (不阻塞主循环), 无则 asyncio.run (脚本场景)。日志截断字段长度防膨胀。

动手做

```
# 手动体验 ReAct 循环 (需 .env 配好 LLM_API_KEY)
cd backend
python -c "
from app.agent.harness import AgentHarness
h = AgentHarness()
ans = h.run('TinyDB 怎么把数据存到磁盘? ')
print('ANSWER:', ans)
"
```

第 5 层小结

Agent 层的核心思想: 把 LLM 当作决策者, 工具当作执行器。手写 ReAct 的优势是 “每一行都知道在干嘛” ——提示词格式、JSON 解析、消息注入、日志落库, 全部可控。面试讲 Agent, 不需要背 LangGraph 的 API, 直接说 “我手写了一个 ReAct 循环, 核心是 Thought-Action-Observation 三段式 + 括号深度 JSON 解析器”, 就很有说服力。

5.6 eval_tasks.py — Agent 评测任务集 (预留)

完整代码

```
"""Agent 评测任务集"""
```

```

EVAL_TASKS = [
    {
        "question": "TinyDB 怎么把数据存到磁盘上的?",
        "reference": "TinyDB 使用 JSONStorage 类将数据以 JSON 格式存储在磁盘的单个文件中。每次写操作会序列化整个数据库并覆写文件。支持原子写入（临时文件 + os.replace）。",
        "description": "存储机制",
    },
    {
        "question": "请解释 TinyDB 中的 Query 类是怎么实现字段查询的?",
        "reference": "Query 类通过 __init__ 接收字段名, 重载 __eq__、__lt__、__contains__ 等运算符生成条件表达式。多个条件通过 & 和 | 组合。每个 Query 实例表示一个查询条件。",
        "description": "查询机制",
    },
    {
        "question": "TinyDB 支持哪几种存储后端? 分别有什么特点?",
        "reference": "TinyDB 支持 JSONStorage (JSON 文件存储, 持久化)、MemoryStorage (内存存储, 不持久化)。JSONStorage 是默认后端。",
        "description": "存储后端",
    },
    {
        "question": "TinyDB 的 insert 和 update 方法有什么区别? 分别怎么用?",
        "reference": "insert 插入新文档并自动分配 _id。update 根据条件更新已有文档。insert 返回文档 ID, update 返回修改的文档数。",
        "description": "插入与更新",
    },
    {
        "question": "请帮我生成一段测试代码, 测试 Table 类的 search 方法。",
        "reference": "生成的测试代码应该包含 pytest 函数, 创建 TinyDB 实例, 插入测试数据, 调用 search 方法并断言结果符合预期。",
        "description": "测试生成",
    },
    {
        "question": "TinyDB 的中间件是怎么工作的? CachingMiddleware 有什么用?",
        "reference": "中间件包装存储层, 在读写操作前后执行额外逻辑。CachingMiddleware 缓存写入请求, 减少磁盘 I/O, 需要手动 flush() 或 close() 才会实际写入。",
        "description": "中间件机制",
    },
    {
        "question": "解释一下 TinyDB 中 operations 模块里的 Increment 和 Delete 操作。",
        "reference": "Increment(field, value) 将指定字段增加 value。Delete(field) 删除文档中的指定字段。它们作为 update 方法的参数使用。",
        "description": "数据操作",
    },
    {
        "question": "TinyDB 是怎么保证并发安全的? 多个进程同时写一个文件会怎样?",
        "reference": "TinyDB 默认不保证并发安全。多个进程同时写入可能导致数据损坏。JSONStorage 的原子写入只能防止单进程写入中断, 不能防止多进程竞争。建议单进程使用或加外部锁。",
        "description": "并发安全",
    },
]

```

这段代码解决什么问题

这是 **Agent 评测任务集**——8 条“问题 + 参考答案 + 分类”，用来评估 Agent 的回答质量。它和 RAG 层的 test_set.py（评测检索）是一对：

- test_set.py → 评测**检索**好不好（答案是否命中正确文件）

- `eval_tasks.py` → 评测 **Agent** 回答好不好（回答是否接近参考答案）

关键设计逐个讲

① 结构：question + reference + description

```
{  
    "question": "TinyDB 怎么把数据存到磁盘上的？",  
    "reference": "TinyDB 使用 JSONStorage 类...",  
    "description": "存储机制",  
}
```

- **question**: 问 Agent 的问题。
- **reference**: **参考答案**——评估时用某种文本相似度（如 ROUGE / LLM 打分）对比 Agent 的回答。
- **description**: 问题分类，方便按主题统计。

② 覆盖 Agent 的典型能力

8 条任务覆盖了 Agent 应该会的几类事：存储机制、查询机制、API 使用（insert/update）、**测试生成**、中间件原理、并发安全——正好对应 `search/explain/testgen` 三个工具的场景。

重要说明：当前是“预留”代码

`eval_tasks.py` 目前没有任何地方 **import** 它（在 `backend/app/` 里全局搜索，只有它自己被定义）。也就是说：

- 它定义好了任务集，但**评估 Agent 回答的脚本还没写**（不像 `test_set.py` 有配套的 `evaluation.py`）。
- 它和 AGENT 的评测引擎 之间的关系是“数据有了，跑分代码待补”。

为什么文档仍要讲它？ 因为它是项目结构的一部分，也是很好的面试素材——“我设计了 Agent 评测任务集（8 条问答，覆盖存储/查询/测试生成等场景），配套的自动化评分脚本是下一步工作”。这比让面试官发现一个“没讲的死文件”要主动得多。

面试答题要点

Q：怎么评估 Agent 的回答质量？ 答：项目里设计了 `eval_tasks.py` 任务集，每条有参考回答。评估思路是用文本相似度（如 ROUGE）或 LLM 打分，把 Agent 的回答和参考回答对比。目前任务集已就绪，自动化评分脚本是后续工作——这样既展示了设计，又诚实说明了进度。

Q：Agent 评测和 RAG 检索评测有什么区别？ 答：检索评测（`test_set.py`）看“检出来的文件对不对”，指标是 Hit Rate/MRR/NDCG；Agent 评测（`eval_tasks.py`）看“最终回答像不像参考答案”，指标是文本相似度。一个是中间环节，一个是最终效果。

第 6 层 服务层

第 4、5 层的 RAG 和 Agent 是“能力”，但用户怎么用这些能力？需要一个**业务服务层**把它们串成具体的业务流程。这一层先看会话持久化服务，下一层看路由。

6.1 conversation_service.py — 会话持久化服务

完整代码

```
"""会话持久化服务：Agent 对话历史的保存与恢复"""

from sqlalchemy import select

from app.database import async_session_factory
from app.models.conversation import Conversation, Message


async def get_or_create_conversation(user_id: int, session_id: str) -> Conversation:
    """获取或创建会话记录（按 user_id + session_id 隔离）"""
    async with async_session_factory() as session:
        result = await session.execute(
            select(Conversation).where(
                Conversation.user_id == user_id,
                Conversation.session_id == session_id,
            )
        )
        conv = result.scalar_one_or_none()
        if conv is None:
            conv = Conversation(user_id=user_id, session_id=session_id)
            session.add(conv)
            await session.commit()
            await session.refresh(conv)
        return conv


async def save_message(user_id: int, session_id: str, role: str, content: str,
                      tool_calls=None):
    """保存一条消息（归属当前用户）"""
    async with async_session_factory() as session:
        session.add(Message(
            user_id=user_id,
            session_id=session_id,
            role=role,
            content=content,
            tool_calls=tool_calls,
        ))
        await session.commit()


async def load_history(user_id: int, session_id: str, limit: int = 20) -> list[dict]:
    """加载会话历史消息（按 user_id 过滤），返回 [{role, content}, ...]"""
    async with async_session_factory() as session:
        result = await session.execute(
            select(Message)
            .where(
                Message.user_id == user_id,
                Message.session_id == session_id,
            )
            .order_by(Message.created_at.asc())
        )
```

```

        .limit(limit)
    )
    messages = result.scalars().all()
    return [
        {"role": m.role, "content": m.content}
        for m in messages
    ]

```

这段代码解决什么问题

让 Agent 对话跨请求持久化。每次调用 `/api/agent/chat`，服务层负责：会话不存在就创建、消息存起来、下次对话把历史捞出来还给 Agent。三个函数，职责单一。

注意：每个函数都带 `user_id` 过滤——这是数据隔离在服务层的体现，用户 A 永远读不到用户 B 的消息。

关键代码逐句拆解

① `get_or_create_conversation` — 先查后建

```

result = await session.execute(
    select(Conversation).where(
        Conversation.user_id == user_id,
        Conversation.session_id == session_id,
    )
)
conv = result.scalar_one_or_none()
if conv is None:
    conv = Conversation(user_id=user_id, session_id=session_id)
    session.add(conv)
    await session.commit()
    await session.refresh(conv)
return conv

```

- `scalar_one_or_none()`：正好一条返回它，没有返回 `None`（多于一条会报错——但前面模型层有 `UniqueConstraint(user_id, session_id)` 兜底，保证最多一条）。
- 不存在就创建，`commit` 后 `refresh` 拿回数据库生成的 `id`。
- **注意这里没有并发防护**：两个并发请求同时发现“不存在”，都走创建——靠数据库的唯一约束兜底，第二个会报错。这是“应用层乐观 + 数据库层兜底”的取舍。

② `save_message` — 追加一条消息

```

session.add(Message(
    user_id=user_id, session_id=session_id,
    role=role, content=content, tool_calls=tool_calls,
))
await session.commit()

```

`tool_calls` 参数预留（当前路由没传，字段留空）。

③ `load_history` — 按时间顺序加载最近 N 条

```

select(Message)
.where(Message.user_id == user_id, Message.session_id == session_id)
.order_by(Message.created_at.asc())
.limit(limit)

```

- `order_by(created_at.asc())`: 时间正序，保证对话顺序。
- `.limit(limit)`: 最多 20 条——历史太长会撑爆上下文窗口，截断是保护机制。
- 返回 `[{role, content}, ...]` 结构，正好是 Agent `restore_history` 需要的格式（`harness.conversation_history` 就是 `[{role, content}]` 列表）。

面试题要点

Q: 会话历史为什么存 MySQL 而不是内存？ 答：服务重启、多实例部署都需要共享状态。存数据库可持久化、可扩展。内存只适合单进程 demo。

Q: 为什么加载历史要 limit？ 答：上下文窗口有限。20 条历史 + 当前问题太长会超出 LLM 的 `max_tokens`，或让响应变慢。截断历史是 RAG/Agent 系统的常见保护。

Q: get_or_create 并发会怎样？ 答：应用层“先查后建”存在竞态，但数据库唯一约束（`user_id, session_id`）兜底，重复创建会被拒。这是“应用层尽量优化 + 数据库层保证正确”的经典组合。

第 7 层 API 路由层

服务层串好了业务逻辑，路由层把它暴露成 HTTP 接口。FastAPI 的路由层就是“门口的接待员”：接住请求、做参数校验、调服务、返回响应。4 个路由文件对应 4 组接口。

7.1 auth_router.py — 认证接口

完整代码

"""认证路由：注册、登录、刷新、登出"""

```
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from app.auth import blacklist_token, is_token_blacklisted

import datetime
from datetime import UTC

from app.database import get_db
from app.models.user import User
from app.auth import (
    hash_password, verify_password,
    create_access_token, create_refresh_token, decode_token,
    get_current_user, security
)

router = APIRouter(prefix="/auth" , tags=["auth"])

class RegisterRequest(BaseModel):
```



```

    username:str
    password:str

class LoginRequest(BaseModel):
    username:str
    password:str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"

@router.post("/register" , response_model=TokenResponse)
async def register(req: RegisterRequest , db: AsyncSession = Depends(get_db)):
    existing = await db.execute(select(User).where(User.username == req.username))
    if existing.scalar_one_or_none():
        raise HTTPException(status_code=400 , detail="Username already exists")

    user = User(
        username = req.username,
        hashed_password = hash_password(req.password),
    )
    db.add(user)
    await db.commit()
    await db.refresh(user)

    return TokenResponse(
        access_token=create_access_token({"sub": user.username}),
        refresh_token=create_refresh_token({"sub": user.username}),
    )

@router.post("/login" , response_model=TokenResponse)
async def login(req: LoginRequest, db: AsyncSession = Depends(get_db)):

    result = await db.execute(select(User).where(User.username == req.username))
    user = result.scalar_one_or_none()
    if not user or not verify_password(req.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    return TokenResponse(
        access_token=create_access_token({"sub": user.username}),
        refresh_token=create_refresh_token({"sub": user.username}),
    )

@router.post("/refresh", response_model=TokenResponse)
async def refresh(refresh_token: str, db: AsyncSession = Depends(get_db)):
    payload = decode_token(refresh_token)
    if payload.get("type") != "refresh":
        raise HTTPException(status_code=401, detail="Invalid token type")

    username = payload.get("sub")
    result = await db.execute(select(User).where(User.username == username))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=401, detail="User not found")

    return TokenResponse(
        access_token=create_access_token({"sub": user.username}),
        refresh_token=create_refresh_token({"sub": user.username}),
    )

@router.post("/logout")
async def logout(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    current_user: User = Depends(get_current_user),
):

```

```
# 把 access token 加入黑名单, 剩余有效期
payload = decode_token(credentials.credentials)
exp = payload.get("exp", 0)
now = int(datetime.datetime.now(UTC).timestamp())
remain = max(exp - now, 1)

blacklist_token(credentials.credentials, remain)
return {"message": "Logged out successfully"}
```

这段代码解决什么问题

注册、登录、刷新、登出四个接口，对应 auth.py 的能力。所有请求体 / 响应都用 Pydantic 模型校验。

关键代码逐句拆解

① Pydantic 模型 — 请求/响应契约

```
class RegisterRequest(BaseModel):
    username: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"
```

- 请求模型定义接口“要什么”；响应模型（`response_model=TokenResponse`）定义“返回什么”。FastAPI 自动做校验、序列化、生成 OpenAPI 文档。
- 前端传多了字段会被忽略，传少了类型不对会 422。

② register — 注册

```
existing = await db.execute(select(User).where(User.username == req.username))
if existing.scalar_one_or_none():
    raise HTTPException(status_code=400, detail="Username already exists")

user = User(username=req.username, hashed_password=hash_password(req.password))
db.add(user)
await db.commit()
await db.refresh(user)

return TokenResponse(
    access_token=create_access_token({"sub": user.username}),
    refresh_token=create_refresh_token({"sub": user.username}),
)
```

- 先查重：用户名存在返回 400。
- **hash_password** 只存哈希，永不存明文。
- 注册成功直接返回 **token**（免登录），sub 存用户名。

③ login — 登录

```
if not user or not verify_password(req.password, user.hashed_password):
    raise HTTPException(status_code=401, detail="Invalid credentials")
```

用户不存在和密码错误返回同一个错误（401 “Invalid credentials”）——防账号枚举（否则攻击者

能探测“这个用户名是否存在”）。这是安全细节，面试常考。

④ refresh — 刷新 token

```
payload = decode_token(refresh_token)
if payload.get("type") != "refresh":
    raise HTTPException(status_code=401, detail="Invalid token type")
```

校验 `type == "refresh"`：防止拿 access token 来换新 token。因为两种 token 都用同一个 SECRET_KEY 签的，都能 decode_token 成功，必须靠 type 区分。

⑤ logout — 登出

```
payload = decode_token(credentials.credentials)
exp = payload.get("exp", 0)
now = int(datetime.datetime.now(UTC).timestamp())
remain = max(exp - now, 1)
blacklist_token(credentials.credentials, remain)
```

- 从 token 里读出过期时间 exp（Unix 秒）。
- remain = exp - now：剩余有效秒数，作为 Redis 黑名单的 TTL。
- max(remain, 1)：兜底，至少 1 秒（防止刚签发就登出导致 TTL 为 0）。
- blacklist_token 把 token 写进 Redis，剩余寿命到期自动清除。

面试答题要点

Q：为什么登录失败返回统一的“Invalid credentials”？ 答：防止账号枚举。如果“用户不存在”和“密码错误”返回不同信息，攻击者就能探测系统里有哪些用户名。统一错误信息是安全最佳实践。

Q：refresh 接口怎么防止 access token 冒充？ 答：两种 token 用同一密钥签发，都能验签。所以 payload 里加了 type 字段区分，refresh 接口校验 `type == "refresh"`，access token 不通过。

Q：登出是怎么实现的？ 答：JWT 无状态，无法主动作废。登出时把 access token 加入 Redis 黑名单，TTL = token 剩余有效期。每次鉴权先查黑名单。到期自动清除，不占空间。

Q：为什么注册/登录都返回 token？ 答：注册成功直接发 token，用户免去“注册完再登录”的步骤，体验更好。sub 字段存用户名，是 JWT 的标准声明。

7.2 agent_router.py — Agent 对话接口

完整代码

```
"""Agent 对话接口"""

import asyncio
from fastapi import APIRouter, Depends
from pydantic import BaseModel
```

```

from app.agent.harness import AgentHarness
from app.models.user import User
from app.auth import get_current_user
from app.services.conversation_service import (
    get_or_create_conversation, save_message, load_history,
)

router = APIRouter(prefix="/agent", tags=["agent"])

class ChatRequest(BaseModel):
    message: str
    session_id: str = "default"

class ChatResponse(BaseModel):
    answer: str
    session_id: str

@router.post("/chat", response_model=ChatResponse)
async def chat(
    req: ChatRequest,
    current_user: User = Depends(get_current_user),
):
    user_id = current_user.id

    # 1. 确保会话记录存在（按用户隔离）
    await get_or_create_conversation(user_id, req.session_id)

    # 2. 从 MySQL 恢复历史（只查当前用户的）
    history = await load_history(user_id, req.session_id)
    harness = AgentHarness(session_id=req.session_id)
    harness.restore_history(history)

    # 3. 运行 Agent
    answer = await asyncio.to_thread(harness.run, req.message)

    # 4. 保存本轮消息（归属当前用户）
    await save_message(user_id, req.session_id, "user", req.message)
    await save_message(user_id, req.session_id, "assistant", answer)

    return ChatResponse(answer=answer, session_id=req.session_id)

```

这段代码解决什么问题

这是用户问 **Agent** 问题的入口。它把第 5、6 层串起来：会话管理 → 历史恢复 → Agent 运行 → 结果落库。

关键代码逐句拆解

① `current_user: User = Depends(get_current_user)`

路由的第一个依赖就是鉴权。没登录直接 **401**，登录了 `current_user` 就是数据库里的 `User` 对象。

② 四步编排

```

# 1. 确保会话存在
await get_or_create_conversation(user_id, req.session_id)

```

```
# 2. 恢复历史（只查当前用户的）
history = await load_history(user_id, req.session_id)
harness = AgentHarness(session_id=req.session_id)
harness.restore_history(history)

# 3. 运行 Agent
answer = await asyncio.to_thread(harness.run, req.message)

# 4. 保存本轮消息
await save_message(user_id, req.session_id, "user", req.message)
await save_message(user_id, req.session_id, "assistant", answer)
```

每个查询都带 `user_id`——数据隔离贯穿到服务层。`load_history` 只查当前用户的会话历史。

③ `asyncio.to_thread(harness.run, ...)` — 异步框架里跑同步 Agent

```
answer = await asyncio.to_thread(harness.run, req.message)
```

`harness.run` 内部是同步的 LLM 调用（`httpx.post` 阻塞）。如果直接 `await harness.run(...)`，整个事件循环会卡住几十秒（等 LLM 回复），其他请求全部排队。`asyncio.to_thread` 把它扔进线程池，让出事件循环处理其他请求。

这是贯穿全项目的模式：同步阻塞库（`httpx`、`Chroma`、`BM25`）嵌进异步框架时，用 `to_thread` 隔离。

④ 先保存 user 消息再保存 assistant 消息

先存用户问题，再存回答。注意顺序：如果 Agent 报错（`harness.run` 抛异常），`save_message("assistant")` 不会执行——但用户消息已存。这是“至少保留用户输入”的容错。

面试题要点

Q: Agent 是同步的，怎么放进异步 FastAPI？ 答：

`asyncio.to_thread(harness.run, ...)` 把同步的 Agent 循环（内部是阻塞的 `httpx` LLM 调用）扔进线程池，让出事件循环。否则一个请求等 LLM 回复几十秒，整个服务就卡死了。

Q: 多轮对话怎么实现？ 答：每个 `session_id` 对应一个会话。请求进来先

`load_history` 从 MySQL 恢复最近 20 条，`restore_history` 给 Agent，Agent 基于历史 + 新问题作答，再 `save_message` 存两轮。

Q: 数据隔离怎么贯穿？ 答：所有查询都带 `user_id`。

`get_or_create_conversation`、`load_history`、`save_message` 都以 `user_id + session_id` 为键。用户 A 传用户 B 的 `session_id` 也查不到 B 的历史。

7.3 search_router.py — 代码检索接口

完整代码

```
"""代码检索接口"""

import asyncio
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from app.rag.dense_retriever import DenseRetriever, SYSTEM_CORPUS
from app.rag.sparse_retriever import SparseRetriever
from app.rag.fusion import rrf, async_hybrid_search
from app.rag.reranker import Reranker
from app.rag.query_rewriter import rewrite
from app.models.user import User
from app.auth import get_current_user

router = APIRouter(prefix="/search", tags=["search"])

class SearchRequest(BaseModel):
    query: str
    top_k: int = 5
    use_hybrid: bool = True
    use_hyde: bool = False
    use_rerank: bool = False

class SearchResult(BaseModel):
    text: str
    source: str

class SearchResponse(BaseModel):
    query: str
    results: list[SearchResult]
    latency_ms: float

@router.post("", response_model=SearchResponse)
async def search(
    req: SearchRequest,
    current_user: User = Depends(get_current_user),
):
    query = req.query.strip()
    if not query:
        raise HTTPException(status_code=400, detail="Query is empty")

    dense = DenseRetriever()
    sparse = SparseRetriever.from_redis()

    if req.use_hyde:
        query = await rewrite(query, strategy="hyde")

    if req.use_hybrid:
        # 复用已有的 async_hybrid_search (含日志打点), 默认只搜系统语料
        fused = await async_hybrid_search(
            query, dense, sparse, top_n=req.top_k,
            dense_where=SYSTEM_CORPUS,
        )
    else:
        dense_results = await asyncio.to_thread(
            dense.search, query, req.top_k, SYSTEM_CORPUS
        )
        fused = dense_results

    if req.use_rerank and fused:
        reranker = Reranker()
```

```

        fused = await asyncio.to_thread(
            reranker.rerank, query, fused, req.top_k
        )

    return SearchResponse(
        query=req.query,
        results=[
            SearchResult(text=r["text"], source=r["source"])
            for r in fused
        ],
        latency_ms=0.0, # 简化：不测延迟，需要的话再补
    )

```

这段代码解决什么问题

暴露 RAG 检索能力，每个组件都做成开关：

参数	默认	作用
<code>use_hybrid</code>	<code>True</code>	是否走双路混合检索（否则只走稠密）
<code>use_hyde</code>	<code>False</code>	是否先 HyDE 改写
<code>use_rerank</code>	<code>False</code>	是否 cross-encoder 重排

这样调用方可以自由组合，对比不同配置的检索效果。

关键代码逐句拆解

① 默认只搜系统语料

```

fused = await async_hybrid_search(
    query, dense, sparse, top_n=req.top_k,
    dense_where=SYSTEM_CORPUS, # 只搜系统语料，不碰用户上传
)

```

`dense_where=SYSTEM_CORPUS`：检索范围限定为系统语料（`{"source_type": "system"}`）。防止系统检索把用户上传的私有内容搜出来——这是数据隔离在检索接口的落地。

② 组件开关的流水线

```

if req.use_hyde:
    query = await rewrite(query, strategy="hyde")

if req.use_hybrid:
    fused = await async_hybrid_search(...)
else:
    fused = dense # 只走稠密

if req.use_rerank and fused:
    fused = reranker.rerank(...)

```

HyDE 改写 → 检索 → 重排，每步可开关。这就是“把 RAG 链路做成可插拔”。

③ 同步调用用 `to_thread` 包裹

```
dense_results = await asyncio.to_thread(dense.search, query, req.top_k,
SYSTEM_CORPUS)
fused = await asyncio.to_thread(reranker.rerank, query, fused, req.top_k)
```

Chroma 检索和重排模型推理都是同步阻塞的，扔线程池。只有 `async_hybrid_search` 是异步的（它内部已用 `to_thread` 了）。

④ 响应模型过滤字段

```
results=[
    SearchResult(text=r["text"], source=r["source"])
    for r in fused
],
```

只返回 `text` 和 `source`，不暴露内部字段（`score`、`owner_id` 等）——接口最小化，也避免泄露不该给前端的信息。

面试题要点

Q：为什么检索接口要把组件做成开关？ 答：方便对比验证。前端/测试可以分别开 `hybrid`、`HyDE`、`rerank`，直观看到哪个配置结果更好。这和技术栈里的消融实验是一套思路。

Q：检索接口怎么保证不搜到用户私有内容？ 答：`dense_where=SYSTEM_CORPUS` 把检索范围限定在系统语料（`source_type=system`）。用户上传内容在另一个逻辑空间，检索接口根本不碰。

Q：latency_ms 为什么是 0.0？ 答：这个字段当前是简化实现（TODO）。真正统计延迟可以复用 `fusion.py` 里的 `latency_ms`，或加 `timing` 装饰器。这是可以主动提到的改进点。

7.4 upload_router.py — 文件上传接口

完整代码

```
"""文件上传接口"""

from fastapi import APIRouter, Depends, UploadFile, File, HTTPException
from pydantic import BaseModel
from app.rag.user_upload import UserUploadIndex
from app.models.user import User
from app.auth import get_current_user
from pathlib import Path

router = APIRouter(prefix="/upload", tags=["upload"])

ALLOWED_EXTENSIONS = {".py", ".js", ".md", ".txt"}

class UploadSearchRequest(BaseModel):
    query: str
    top_k: int = 5
```



```

@router.post("")
async def upload_file(
    file: UploadFile = File(...),
    current_user: User = Depends(get_current_user),
):
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(status_code=400, detail=f"Unsupported file type: {ext}")

    content_bytes = await file.read()
    try:
        content = content_bytes.decode("utf-8")
    except UnicodeDecodeError:
        raise HTTPException(status_code=400, detail="File must be UTF-8 encoded")

    uploader = UserUploadIndex()
    chunk_count = uploader.add_file(file.filename, content, owner_id=current_user.id)

    return {
        "filename": file.filename,
        "chunk_count": chunk_count,
        "message": "File indexed successfully",
    }

@router.post("/search")
async def search_upload(
    req: UploadSearchRequest,
    current_user: User = Depends(get_current_user),
):
    """只检索当前用户自己上传的内容 (owner_id 隔离) """
    query = req.query.strip()
    if not query:
        raise HTTPException(status_code=400, detail="Query is empty")

    uploader = UserUploadIndex()
    results = uploader.search(query, owner_id=current_user.id, k=req.top_k)

    return {
        "query": req.query,
        "results": [
            {"text": r["text"], "source": r["source"]}
            for r in results
        ],
    }

```

这段代码解决什么问题

用户上传自己的代码文件 → 索引进 Chroma → 只能搜自己的。这是“个人知识库”能力，也是数据隔离的完整闭环。

关键代码逐句拆解

① 文件类型白名单

```

ALLOWED_EXTENSIONS = {".py", ".js", ".md", ".txt"}

ext = Path(file.filename).suffix.lower()
if ext not in ALLOWED_EXTENSIONS:
    raise HTTPException(status_code=400, detail=f"Unsupported file type: {ext}")

```

用白名单（允许什么）而不是黑名单（禁止什么）——安全实践。`.suffix.lower()` 取扩展名并小写，防 `Test.PY` 绕过。

② 编码校验

```
content_bytes = await file.read()
try:
    content = content_bytes.decode("utf-8")
except UnicodeDecodeError:
    raise HTTPException(status_code=400, detail="File must be UTF-8 encoded")
```

索引前先解码，失败直接 400——防止二进制文件/乱码文件进入索引库。

③ 上传即索引（owner 隔离）

```
uploader = UserUploadIndex()
chunk_count = uploader.add_file(file.filename, content, owner_id=current_user.id)
```

`add_file` 内部给每个 chunk 打 `source_type="user_upload" + owner_id=current_user.id`（见 4.8 节）。

④ 检索自己的上传

```
results = uploader.search(query, owner_id=current_user.id, k=req.top_k)
```

`owner_id=current_user.id` 从当前登录用户取，**不信任前端传的 owner**——防止越权。
`UserUploadIndex.search` 内部还有应用层兜底过滤（双重隔离）。

面试题要点

Q：上传文件怎么防止越权检索？ 答：三处配合。① 索引时 chunk 打 `owner_id`；② 检索时 `owner_id` 从当前登录用户取（不信任前端参数）；③ `UserUploadIndex.search` 内部再做应用层兜底过滤（source 前缀 + `owner_id` 二次校验）。纵深防御。

Q：为什么用扩展名白名单？ 答：白名单比黑名单安全——只允许明确支持的类型，未知类型一律拒绝，避免恶意文件混入。`.suffix.lower()` 防大小写绕过。

第 6、7 层小结

服务层（`conversation_service`）把“会话”这个业务概念落地，路由层把能力暴露成 HTTP。四个路由各司其职：`auth`（身份）、`agent`（对话）、`search`（检索）、`upload`（上传）。贯穿所有路由的三件事：鉴权依赖、数据隔离（`user_id/owner_id`）、异步不阻塞（`to_thread`）。

第 8 层 入口与部署

前面 7 层把“能力”都做好了，这一层把它们**组装成可运行的系统**，并解决“怎么跑起来”的问题：

- `main.py` —— 应用入口，组装路由 + 生命周期

- streamlit_app.py —— 前端，人机交互
 - docker-compose.yml —— 一键部署 4 个服务
 - Dockerfile —— 后端/前端镜像构建
 - ci.yml —— GitHub Actions 持续集成
-

8.1 main.py — 应用入口组装

完整代码

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.config import settings
from app.database import engine, Base
from app.logger import log
from app.routers.auth_router import router as auth_router
from app.routers.agent_router import router as agent_router
from app.routers.search_router import router as search_router
from app.routers.upload_router import router as upload_router
from app.middleware import RateLimitMiddleware

@asynccontextmanager
async def lifespan(app: FastAPI):
    log.info("%s Starting up...", settings.APP_NAME)

    try:
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
            log.info("Tables created successfully")
    except Exception as e:
        log.warning("Failed to create tables (may already exist): %s", e)

    yield

    try:
        await engine.dispose()
        log.info("Connection closed")
    except Exception as e:
        log.warning("Error closing connection: %s", e)

app = FastAPI(title=settings.APP_NAME, lifespan=lifespan)
app.add_middleware(RateLimitMiddleware, rate_limit=60, window_seconds=60)
app.include_router(auth_router, prefix="/api")
app.include_router(agent_router, prefix="/api")
app.include_router(search_router, prefix="/api")
app.include_router(upload_router, prefix="/api")

@app.get("/")
async def root():
    return {"message": f"{settings.APP_NAME} is running!"}

@app.get("/health")
async def health():
    return {"status": "ok"}
```

这段代码解决什么问题

组装一切：创建 FastAPI 实例、注册中间件、挂载路由、管理生命周期（启动建表 / 关闭释放连接）。

关键代码逐句拆解

① lifespan — 生命周期管理（FastAPI 现代写法）

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 启动时执行
    try:
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
            log.info("Tables created successfully")
    except Exception as e:
        log.warning("Failed to create tables (may already exist): %s", e)

    yield # ← 应用运行期间停在这里

    # 关闭时执行
    try:
        await engine.dispose()
        log.info("Connection closed")
    except Exception as e:
        log.warning("Error closing connection: %s", e)
```

`lifespan` 是异步上下文管理器，替代旧的 `@app.on_event("startup")`。三段结构：

- **yield 之前：**启动钩子——建表。
- **yield：**应用运行期间（一直停在这）。
- **yield 之后：**关闭钩子——释放连接池。

建表的细节：

```
async with engine.begin() as conn:
    await conn.run_sync(Base.metadata.create_all)
```

- `engine.begin()` 开启事务。
- `conn.run_sync(...)`：因为 SQLAlchemy 的 DDL（建表）是同步 API，`run_sync` 把它包到异步连接里执行。
- `Base.metadata.create_all`：基于所有继承 `Base` 的模型建表。**为什么不会重复建？**
`create_all` 默认 `checkfirst=True`，表已存在就跳过。`except` 捕获“表已存在”等错误，启动不崩溃。

② 中间件注册

```
app.add_middleware(RateLimitMiddleware, rate_limit=60, window_seconds=60)
```

挂载限流中间件：**每个客户端 60 秒内最多 60 次 API 请求**。注意中间件在路由之前执行，所以所有请求先过限流。

③ 路由挂载

```
app.include_router(auth_router, prefix="/api")
app.include_router(agent_router, prefix="/api")
app.include_router(search_router, prefix="/api")
app.include_router(upload_router, prefix="/api")
```

- 统一 prefix="/api", 加上各 router 自己的 prefix (/auth、/agent、/search、/upload), 得到最终路径:
 - /api/auth/register、/api/auth/login、/api/auth/refresh、/api/auth/logout
 - /api/agent/chat
 - /api/search
 - /api/upload、/api/upload/search

④ 健康检查接口

```
@app.get("/health")
async def health():
    return {"status": "ok"}
```

给 Docker / 负载均衡 / CI 探活用的。/ 返回应用名, /health 返回 {"status": "ok"}。

面试题要点

Q: FastAPI 的 lifespan 是什么? 答: 异步上下文管理器, yield 前是启动钩子、yield 后是关闭钩子。替代旧的 on_event("startup"/"shutdown")。这里启动时建表、关闭时释放连接池。

Q: Base.metadata.create_all 为什么不重复建表? 答: create_all 默认 checkfirst=True, 检查表是否存在, 存在就跳过。所以多次启动不会报错。

Q: 中间件和路由的执行顺序? 答: 中间件在请求进入路由前执行。RateLimitMiddleware 的 dispatch 在 call_next 之前做限流检查, 超限直接 429, 否则 call_next 放行进入路由。

8.2 streamlit_app.py — Streamlit 前端

完整代码

```
"""Streamlit 前端: 登录 + Agent 对话 + 检索 + 文件上传"""

import os
import json
import time
import uuid
import requests
import streamlit as st

API_BASE = os.environ.get("API_BASE", "http://127.0.0.1:8000/api")

# ----- 会话状态初始化 -----
if "access_token" not in st.session_state:
    st.session_state.access_token = None
```

```

if "refresh_token" not in st.session_state:
    st.session_state.refresh_token = None
if "messages" not in st.session_state:
    st.session_state.messages = []
if "session_id" not in st.session_state:
    st.session_state.session_id = None

def auth_headers():
    """构造带 token 的请求头"""
    return {"Authorization": f"Bearer {st.session_state.access_token}"}

# ----- 侧边栏: 登录 / 注册 -----
st.sidebar.title("📁 登录")
auth_mode = st.sidebar.radio("模式", ["登录", "注册"])

if st.session_state.access_token:
    st.sidebar.success("已登录")
    if st.sidebar.button("退出登录"):
        try:
            requests.post(f"{API_BASE}/auth/logout", headers=auth_headers())
        except Exception:
            pass
        st.session_state.access_token = None
        st.session_state.refresh_token = None
        st.session_state.messages = []
        st.session_state.session_id = None
        st.rerun()
else:
    username = st.sidebar.text_input("用户名")
    password = st.sidebar.text_input("密码", type="password")
    if st.sidebar.button("提交"):
        url = f"{API_BASE}/auth/login" if auth_mode == "登录" else
f"{API_BASE}/auth/register"
        resp = requests.post(url, json={"username": username, "password": password})
        if resp.status_code == 200:
            data = resp.json()
            st.session_state.access_token = data["access_token"]
            st.session_state.refresh_token = data["refresh_token"]
            # 每个用户/每次登录生成独立会话, 避免跨用户共享历史
            st.session_state.session_id = uuid.uuid4().hex[:16]
            st.session_state.messages = []
            st.sidebar.success("登录成功!")
            st.rerun()
        else:
            st.sidebar.error(resp.json().get("detail", "失败"))

# ----- 主界面 -----
st.title("📁 Code Assistant Agent")
st.caption("基于 TinyDB 代码库的 AI 助手")

# 未登录就显示提示
if not st.session_state.access_token:
    st.info("请先在左侧登录或注册")
    st.stop()

# ----- Tab: 对话 / 检索 / 上传 -----
tab_chat, tab_search, tab_upload = st.tabs(["📁 对话", "📁 检索", "📁 上传"])

# --- 对话 Tab ---
with tab_chat:
    for msg in st.session_state.messages:

```

```

        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    if prompt := st.chat_input("问点 TinyDB 的问题..."):
        st.session_state.messages.append({"role": "user", "content": prompt})
        with st.chat_message("user"):
            st.markdown(prompt)

        with st.chat_message("assistant"):
            with st.spinner("Agent 思考中..."):
                resp = requests.post(
                    f"{API_BASE}/agent/chat",
                    json={"message": prompt, "session_id":
st.session_state.session_id or "default"},
                    headers=auth_headers(),
                    timeout=120,
                )
            if resp.status_code == 200:
                answer = resp.json()["answer"]
            else:
                answer = f"错误: {resp.text}"
            st.markdown(answer)
            st.session_state.messages.append({"role": "assistant", "content":
answer})

# --- 检索 Tab ---
with tab_search:
    query = st.text_input("输入检索查询 (英文效果更好) ")
    top_k = st.slider("返回条数", 1, 10, 5)
    if st.button("检索") and query:
        resp = requests.post(
            f"{API_BASE}/search",
            json={"query": query, "top_k": top_k},
            headers=auth_headers(),
            timeout=60,
        )
        if resp.status_code == 200:
            results = resp.json()["results"]
            if not results:
                st.info("没有找到结果")
            for r in results:
                st.markdown(f"***{r['source']}***")
                st.code(r["text"][:300], language="python")
        else:
            st.error(resp.json().get("detail", "检索失败"))

# --- 上传 Tab ---
with tab_upload:
    uploaded = st.file_uploader("上传代码文件", type=["py", "js", "md", "txt"])
    if uploaded and st.button("上传并索引"):
        resp = requests.post(
            f"{API_BASE}/upload",
            files={"file": (uploaded.name, uploaded.getvalue(), "text/plain")},
            headers=auth_headers(),
            timeout=60,
        )
        if resp.status_code == 200:
            st.success(f"索引成功: {resp.json()['chunk_count']} chunks")
        else:
            st.error(resp.json().get("detail", "上传失败"))

```

这段代码解决什么问题

一个纯前端页面（Streamlit），用 requests 调后端 API。用户在这里登录、和 Agent 对话、检索代码、上传文件。不涉及任何后端逻辑——它只是“后端的遥控器”。

关键代码逐句拆解

① st.session_state — 页面级状态

```
if "access_token" not in st.session_state:
    st.session_state.access_token = None
...
```

Streamlit 每次交互会重跑整个脚本，所以状态必须存 st.session_state（否则刷新就丢）。token、消息、session_id 都存这里。

② 登录/注册 — 调用 auth 接口

```
url = f"{API_BASE}/auth/login" if auth_mode == "登录" else
f"{API_BASE}/auth/register"
resp = requests.post(url, json={"username": username, "password": password})
if resp.status_code == 200:
    data = resp.json()
    st.session_state.access_token = data["access_token"]
    ...
    st.session_state.session_id = uuid.uuid4().hex[:16]    # 每次登录新会话
```

关键设计：每次登录生成新 session_id (uuid.uuid4().hex[:16]) ——防止一个用户退出后，下一个用户登录还带着上一个用户的会话历史。数据隔离在前端也做了一层。

③ 对话 Tab — 调 agent/chat

```
resp = requests.post(
    f"{API_BASE}/agent/chat",
    json={"message": prompt, "session_id": st.session_state.session_id or "default"},
    headers=auth_headers(),
    timeout=120,
)
```

- headers=auth_headers(): 带 Authorization: Bearer <token>。
- timeout=120: Agent 循环可能多步调 LLM，给足超时。

④ 检索 Tab

```
st.code(r["text"][:300], language="python")
```

结果用 st.code 以 Python 代码块展示，截断 300 字符。**检索接口只搜系统语料**（后端 dense_where=SYSTEM_CORPUS），前端不用传任何过滤参数。

⑤ 上传 Tab

```
resp = requests.post(
    f"{API_BASE}/upload",
    files={"file": (uploaded.name, uploaded.getvalue(), "text/plain")},
    headers=auth_headers(),
    timeout=60,
```


)

requests 的 multipart 上传格式: {"file": (文件名, 内容, MIME 类型)}。

面试题要点

Q: 前端怎么和后端通信? 答: Streamlit 页面用 requests 调后端 REST API, token 放 Authorization 头。前后端通过 HTTP + JWT 解耦。

Q: 为什么每次登录生成新 session_id? 答: 防止跨用户历史泄漏。如果 session_id 固定, 用户 B 登录后会看到用户 A 的会话历史。每次登录随机生成, 配合后端 user_id + session_id 隔离, 双保险。

Q: 为什么对话请求超时设 120 秒? 答: Agent 最多 6 步, 每步可能调 LLM (耗时数秒), 一轮对话可能几十秒。普通 HTTP 默认超时太短, 会提前断开。

8.3 docker-compose.yml — 一键部署

完整代码

```
# version: "3.8"

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: 123456
      MYSQL_DATABASE: code_assistant
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 10

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      MYSQL_HOST: mysql
      MYSQL_PORT: 3306
      MYSQL_USER: root
      MYSQL_PASSWORD: 123456
      MYSQL_DATABASE: code_assistant
```

```

    REDIS_HOST: redis
    REDIS_PORT: 6379
    LLM_API_KEY: ${LLM_API_KEY}
    JWT_SECRET: ${JWT_SECRET}
  depends_on:
    mysql:
      condition: service_healthy
    redis:
      condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "8501:8501"
    environment:
      API_BASE: http://backend:8000/api
    depends_on:
      - backend

volumes:
  mysql_data:

```

这段代码解决什么问题

一个命令跑起整个系统：`docker compose up --build` 启动 4 个服务（MySQL、Redis、backend、frontend）。网络、依赖、健康检查都在一个文件里声明。

关键代码逐句拆解

① MySQL / Redis — 基础设施

```

mysql:
  image: mysql:8.0
  environment:
    MYSQL_ROOT_PASSWORD: 123456
    MYSQL_DATABASE: code_assistant
  volumes:
    - mysql_data:/var/lib/mysql

```

- 声明式配置：root 密码、默认数据库名。
- `volumes` 挂载命名卷，**数据持久化**——容器重启不丢数据。

② healthcheck — 健康检查（关键设计）

```

healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 5s
  timeout: 3s
  retries: 10

```

MySQL 和 Redis 都加了健康检查。**为什么必须有？** 因为 backend 依赖数据库，如果没等 MySQL 就绪就启动后端，连接会失败。

③ depends_on + condition: service_healthy — 依赖就绪才启动

```

backend:
  depends_on:
    mysql:
      condition: service_healthy
    redis:

```

```
condition: service_healthy
```

- **depends_on**: 指定依赖关系。
- **condition: service_healthy**: 只有依赖服务通过健康检查后才启动 backend。这比“sleep 10 秒”靠谱——等的是“真正就绪”而不是“时间到了”。

④ 环境变量透传 — 密钥不进文件

```
backend:
  environment:
    ...
    LLM_API_KEY: ${LLM_API_KEY}
    JWT_SECRET: ${JWT_SECRET}
```

`${LLM_API_KEY}` 从宿主机的 `.env` 文件读入（compose 自动加载同目录 `.env`），密钥不进 `docker-compose.yml` 仓库。这是密钥管理的延续。

⑤ 服务间通信用服务名

```
frontend:
  environment:
    API_BASE: http://backend:8000/api
```

Compose 网络里，backend 是主机名。前端容器访问后端直接 `http://backend:8000`，不用 IP。

面试题要点

Q: 为什么用 `condition: service_healthy`? 答: `depends_on` 默认只保证“容器启动”，不保证“服务就绪”。MySQL 容器起来但没初始化完，后端连数据库会失败。
`healthcheck + condition: service_healthy` 保证依赖真正可用才启动。

Q: 密钥怎么不写进 `compose` 文件? 答: `${LLM_API_KEY}` 引用宿主机的环境变量（compose 自动读同目录 `.env`）。密钥只存在于宿主机 `.env`，不进版本库。

Q: MySQL 数据怎么持久化? 答: 命名卷 `mysql_data` 挂到 `/var/lib/mysql`。容器删了重建，数据还在。

8.4 Dockerfile — 后端与前端镜像

完整代码

```
# backend/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# 安装系统依赖 (Chromadb 需要的底层库)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*
```

```

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

# frontend/Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "streamlit_app.py", "--server.address=0.0.0.0", "--server.port=8501"]

```

这段代码解决什么问题

定义镜像构建：**环境隔离 + 依赖打包 + 启动命令**。把 Python 应用变成可部署的容器。

关键代码逐句拆解

① 多阶段的思想（这里是简版）

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

```

标准流程：基础镜像 → 工作目录 → 先拷依赖清单 → 装依赖 → 拷代码 → 启动。

细节：先 COPY requirements.txt 再 pip install，最后才 COPY .——利用 Docker 层缓存。改代码不重装依赖，构建飞快。

② backend 额外装系统依赖

```

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

```

Chromadb 等库需要编译工具链。--no-install-recommends 和 rm -rf 控制镜像体积。

③ CMD — 启动命令

```

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

- --host 0.0.0.0：容器内所有网络接口，不写这个外部访问不到。
- frontend 用 streamlit run 同理。

面试题要点

Q: Dockerfile 为什么先拷 requirements 再拷代码? 答: 利用 Docker 构建缓存。依赖变更频率远低于代码, 先把依赖层缓存住, 改代码只需重新拷贝代码层, 构建快得多。

Q: 为什么 -host 0.0.0.0? 答: 容器内默认只监听 localhost, 外部/其他容器访问不到。0.0.0.0 监听所有接口, 配合 EXPOSE 和 compose 端口映射才能被访问。

8.5 ci.yml — GitHub Actions 持续集成

完整代码

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: 123456
          MYSQL_DATABASE: code_assistant_test
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h localhost"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=10
      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd="redis-cli ping"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=10

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          cd backend
          pip install fastapi uvicorn sqlalchemy aiomysql pymysql redis \
            python-jose[cryptography] bcrypt pydantic pydantic-settings python-dotenv \
            httpx pytest
```

```

- name: Wait for services
  run: |
    python -c "
      import time, socket
      for host, port in [('127.0.0.1', 3306), ('127.0.0.1', 6379)]:
        for _ in range(30):
          try:
            socket.create_connection((host, port), timeout=1)
            print(f'{host}:{port} ready')
            break
          except OSError:
            time.sleep(1)
        else:
          raise SystemExit(f'{host}:{port} not ready')
    "

- name: Run unit tests
  run: |
    cd backend
    python -m pytest tests/test_auth.py tests/test_auth_api.py
    tests/test_fusion.py -v

```

这段代码解决什么问题

每次 **push** 到 **main** 自动跑测试，防止坏代码进主分支。CI 里启动 MySQL/Redis 服务容器、装依赖、跑 pytest。

关键代码逐句拆解

① 触发条件

```

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

```

push 到 main、或向 main 提 PR 时触发。

② services — 服务容器

```

services:
  mysql:
    image: mysql:8.0
    env:
      MYSQL_ROOT_PASSWORD: 123456
      MYSQL_DATABASE: code_assistant_test
    options: >-
      --health-cmd="mysqladmin ping -h localhost"
    ...

```

GitHub Actions 的 **services** 直接在测试机起 MySQL/Redis 容器（不是 Docker Compose，是 Actions 原生的 service container）。数据库名用 `code_assistant_test`，测试数据库与开发库隔离。

③ Wait for services — 等端口就绪

```

import time, socket
for host, port in [('127.0.0.1', 3306), ('127.0.0.1', 6379)]:
    for _ in range(30):

```

```
try:
    socket.create_connection((host, port), timeout=1)
    print(f'{host}:{port} ready')
    break
except OSError:
    time.sleep(1)
else:
    raise SystemExit(f'{host}:{port} not ready')
```

30 次重试、每次 1 秒，直到端口能连上。没等就绪就跑测试会连不上数据库导致误报失败。

④ 跑单元测试

```
python -m pytest tests/test_auth.py tests/test_auth_api.py tests/test_fusion.py -v
```

只跑不依赖重型组件（嵌入模型、Chroma）的纯逻辑测试：鉴权、鉴权 API、RRF 融合。**CI 保持轻量快速**——不加载 embedding 模型（几百 MB），只验证核心逻辑。

面试题答题要点

Q: CI 里为什么单独起 MySQL/Redis 服务容器？ 答：测试需要真实的 MySQL/Redis 才能跑集成测试（如鉴权 API 要连数据库）。GitHub Actions 的 `services` 在测试机上临时起容器，测完自动销毁，环境干净。

Q: 为什么 CI 只跑部分测试，不跑 RAG 相关测试？ 答：RAG 测试要加载 embedding 模型（几百 MB）和 Chroma，CI 上慢且易因网络失败。纯逻辑测试（JWT、RRF、指标）快速可靠，能在几分钟内反馈问题。这是“CI 分层”——快速层先跑。

Q: 为什么测试数据库用 `code_assistant_test`？ 答：和开发库 `code_assistant` 隔离，测试产生的脏数据不污染开发环境。

第 8 层小结

组装与部署：`main.py` 把能力拼成应用，Streamlit 给人机交互，Docker Compose 一键起 4 个服务，Dockerfile 定义镜像，CI 守护代码质量。到这里，整个项目从代码到部署的完整链路就讲完了。

8.6 tests/ — 单元测试与集成测试

CI 里跑的那几个测试文件，也是项目工程化的一部分。它们验证了核心逻辑的正确性，且**刻意保持轻量**（不加载嵌入模型、不依赖 Chroma），保证 CI 快速稳定。

8.6.1 conftest.py — 测试环境配置

```
"""Shared test configuration that keeps tests out of the development database."""

import os
```

```
os.environ["SKIP_SECRET_VALIDATION"] = "1"
os.environ["JWT_SECRET"] = "test-secret-only-for-ci-0123456789abcdef"
os.environ["LLM_API_KEY"] = "sk-test-only-for-ci-not-real"
os.environ["MYSQL_HOST"] = "127.0.0.1"
os.environ["MYSQL_PORT"] = "3306"
os.environ["MYSQL_USER"] = "root"
os.environ["MYSQL_PASSWORD"] = "123456"
os.environ["MYSQL_DATABASE"] = "code_assistant_test"
os.environ["REDIS_HOST"] = "127.0.0.1"
os.environ["REDIS_PORT"] = "6379"
```

它解决什么问题：pytest 会自动加载 `conftest.py`，在任何测试执行前设置好环境变量。作用有两个：

1. **跳过密钥校验：**`SKIP_SECRET_VALIDATION=1` 让 `config._validate_secrets()` 不拦截（测试没有真密钥）。
2. **连测试数据库：**`MYSQL_DATABASE=code_assistant_test`——测试绝不碰开发库，脏数据不污染开发环境。
3. **用假的 LLM 密钥：**`LLM_API_KEY=sk-test-only...` 让配置能通过校验，但测试里不会真的调 LLM。

`MYSQL_HOST=127.0.0.1` 对应 CI 里的 service 容器映射到宿主机的地址。**conftest** 是“测试环境隔离”的中央开关。

8.6.2 test_auth.py — 密码哈希单元测试

```
"""密码哈希单元测试"""
```

```
import sys
import os

# 必须在 import app 之前设置：纯逻辑测试不需要真实密钥
os.environ.setdefault("SKIP_SECRET_VALIDATION", "1")
os.environ.setdefault("JWT_SECRET", "test-secret-only-for-ci-0123456789abcdef")

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app.auth import hash_password, verify_password

class TestPasswordHashing:
    def test_hash_then_verify_success(self):
        hashed = hash_password("mysecret")
        assert verify_password("mysecret", hashed) is True

    def test_wrong_password_fails(self):
        hashed = hash_password("mysecret")
        assert verify_password("wrong", hashed) is False

    def test_hash_is_different_each_time(self):
        h1 = hash_password("same")
        h2 = hash_password("same")
        assert h1 != h2 # bcrypt 每次加随机盐
```

它验证什么：bcrypt 密码哈希的核心性质——

- 正确密码能验证通过（`test_hash_then_verify_success`）。
- 错误密码验证失败（`test_wrong_password_fails`）。
- 相同密码两次哈希结果不同（`test_hash_is_different_each_time`）——因为 bcrypt 每次加随机

盐。这正是 bcrypt 比 MD5 安全的关键，用测试把它钉死。

两个 setup 细节： - `sys.path.insert(0, ...)`: 手动把 `backend/` 加进模块搜索路径，让 `from app.auth import ...` 能工作（pytest 从项目根跑时不一定在 `sys.path` 里）。 - `os.environ.setdefault(...)`: 在 `import app` 之前设置，因为 `config` 加载时就要读这些环境变量。**import 顺序是这类测试的经典坑。**

8.6.3 test_auth_api.py — 鉴权 API 集成测试

"""鉴权 API 集成测试：注册 / 登录 / 刷新 / 登出 / 未授权访问

用一个只挂载 `auth_router` 的独立 FastAPI app 测试，避免触发 `chromadb` 等重型依赖加载，保证 CI 快速运行。

依赖 CI 提供的 MySQL + Redis services (见 `.github/workflows/ci.yml`)。

```
"""
import pytest
from fastapi import FastAPI
from fastapi.testclient import TestClient

from app.database import engine, Base
from app.routers.auth_router import router

# 独立 app, 只挂 auth_router, 不加载 chromadb 相关模块
app = FastAPI()
app.include_router(router, prefix="/api")

async def _create_tables():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

async def _drop_tables():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

async def _dispose_engine():
    await engine.dispose()

@pytest.fixture(scope="module")
def client():
    with TestClient(app) as test_client:
        test_client.portal.call(_create_tables)
        yield test_client
        test_client.portal.call(_drop_tables)
        test_client.portal.call(_dispose_engine)

class TestRegister:
    def test_register_success(self, client):
        resp = client.post("/api/auth/register",
                           json={"username": "alice", "password": "secret123"})
        assert resp.status_code == 200
        data = resp.json()
        assert "access_token" in data
        assert "refresh_token" in data

    def test_register_duplicate(self, client):
        client.post("/api/auth/register",
```

```

        json={"username": "bob", "password": "secret123"})
resp = client.post("/api/auth/register",
                  json={"username": "bob", "password": "secret123"})
assert resp.status_code == 400

```

```

class TestLogin:

```

```

    def test_login_success(self, client):
        client.post("/api/auth/register",
                    json={"username": "carol", "password": "secret123"})
        resp = client.post("/api/auth/login",
                            json={"username": "carol", "password": "secret123"})
        assert resp.status_code == 200
        assert "access_token" in resp.json()

    def test_login_wrong_password(self, client):
        client.post("/api/auth/register",
                    json={"username": "dave", "password": "secret123"})
        resp = client.post("/api/auth/login",
                            json={"username": "dave", "password": "wrong"})
        assert resp.status_code == 401

```

```

class TestProtected:

```

```

    def test_no_token_returns_401(self, client):
        resp = client.post("/api/auth/logout")
        assert resp.status_code == 401

    def test_invalid_token_returns_401(self, client):
        resp = client.post("/api/auth/logout",
                            headers={"Authorization": "Bearer not-a-real-token"})
        assert resp.status_code == 401

```

```

class TestLogoutBlacklist:

```

```

    def test_logout_revokes_token(self, client):
        # 注册登录拿 token
        client.post("/api/auth/register",
                    json={"username": "erin", "password": "secret123"})
        login = client.post("/api/auth/login",
                            json={"username": "erin", "password": "secret123"})
        token = login.json()["access_token"]
        headers = {"Authorization": f"Bearer {token}"}

        # 登出成功
        resp = client.post("/api/auth/logout", headers=headers)
        assert resp.status_code == 200

        # token 已进黑名单, 再访问受保护接口应 401
        resp = client.post("/api/auth/logout", headers=headers)
        assert resp.status_code == 401

```

```

class TestRefresh:

```

```

    def test_refresh_returns_new_tokens(self, client):
        client.post("/api/auth/register",
                    json={"username": "frank", "password": "secret123"})
        login = client.post("/api/auth/login",
                            json={"username": "frank", "password": "secret123"})
        refresh_token = login.json()["refresh_token"]

        resp = client.post("/api/auth/refresh", params={"refresh_token":
refresh_token})
        assert resp.status_code == 200
        data = resp.json()
        assert "access_token" in data
        assert "refresh_token" in data

```

```
def test_refresh_rejects_access_token(self, client):
    client.post("/api/auth/register",
                json={"username": "grace", "password": "secret123"})
    login = client.post("/api/auth/login",
                        json={"username": "grace", "password": "secret123"})
    access_token = login.json()["access_token"]

    # 用 access token 当 refresh token 用, 应该被拒绝
    resp = client.post("/api/auth/refresh", params={"refresh_token":
access_token})
    assert resp.status_code == 401
```

它验证什么：把 auth_router 挂到**独立 FastAPI app** 上，用 TestClient（FastAPI 自带测试客户端）跑真实 HTTP 流程。覆盖了：

- **注册**：成功返回双 token；重复用户名返回 400。
- **登录**：成功返回 token；密码错误返回 401。
- **未授权访问**：无 token / 假 token → 401。
- **登出黑名单**：登出后再用同一 token 访问 → 401（验证 Redis 黑名单真的生效）。
- **刷新**：refresh token 能换新 token；用 **access token 冒充 refresh token** → 401（验证 type == "refresh" 校验）。

关键设计——独立 app：

```
app = FastAPI()
app.include_router(router, prefix="/api")
```

只挂 auth_router，不 **import 整个 main.py**。这样 agent_router / search_router 里 import 的 chromadb 等重型模块不会被加载，CI 跑得快。这是“最小化测试范围”的技巧。

fixture 生命周期：

```
@pytest.fixture(scope="module")
def client():
    with TestClient(app) as test_client:
        test_client.portal.call(_create_tables)    # 测前建表
        yield test_client
        test_client.portal.call(_drop_tables)      # 测后删表
        test_client.portal.call(_dispose_engine)  # 释放连接
```

scope="module"：整个模块共用一次连接（不是每个测试都建删表）。建表 → 测试 → 删表 → 释放，**用完清理，不留脏数据**。

注意：这个测试依赖真实 MySQL + Redis（conftest 里配的 127.0.0.1）。本地跑需要先 docker compose up mysql redis，CI 里由 service 容器提供。

8.6.4 test_fusion.py — RRF 融合单元测试

```
"""RRF 融合单元测试"""

import sys
import os

# 必须在 import app 之前设置：避免触发 config 密钥校验
os.environ.setdefault("SKIP_SECRET_VALIDATION", "1")
```

```

os.environ.setdefault("JWT_SECRET", "test-secret-only-for-ci-0123456789abcdef")

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app.rag.fusion import rrf

def _item(source: str, idx: int):
    return {"text": f"chunk {idx}", "source": source, "chunk_index": idx, "score":
0.0}

class TestRRF:
    def test_single_rank(self):
        results_list = [
            [_item("a.py", 0), _item("b.py", 1)],
        ]
        fused = rrf(results_list, top_n=5)
        assert len(fused) == 2
        assert fused[0]["source"] == "a.py"
        assert fused[1]["source"] == "b.py"

    def test_merge_duplicates(self):
        results_list = [
            [_item("a.py", 0)],
            [_item("a.py", 0)],
        ]
        fused = rrf(results_list, top_n=5)
        assert len(fused) == 1 # 同一个 chunk 只出现一次

    def test_rank_boost_from_both_lists(self):
        results_list = [
            [_item("a.py", 0), _item("b.py", 1)],
            [_item("b.py", 1), _item("a.py", 0)],
        ]
        fused = rrf(results_list, top_n=5)
        assert fused[0]["source"] in ("a.py", "b.py")
        assert len(fused) == 2

    def test_top_n_limit(self):
        results_list = [
            [_item(f"f{i}.py", i) for i in range(10)],
        ]
        fused = rrf(results_list, top_n=3)
        assert len(fused) == 3

```

它验证什么：RRF 融合算法的核心性质——纯逻辑、无依赖，是最理想的单元测试对象。

- **test_single_rank**：单路结果按原序融合。
- **test_merge_duplicates**：同一个 chunk 在两路都出现时，只保留一条（(source, chunk_index) 去重键生效）。
- **test_rank_boost_from_both_lists**：两路都召回的文档融合后排在前面。
- **test_top_n_limit**：top_n 截断生效。

_item() 辅助函数：构造最小测试数据（{text, source, chunk_index, score}），不用真实 Chroma 就能测融合逻辑——这就是“依赖注入/接口清晰”带来的可测试性。

8.6.5 test_metrics.py — 评估指标单元测试

```

"""评估指标单元测试"""

import sys

```

```

import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app.rag.evaluation import hit_rate, mrr, ndcg

def _make_result(source: str):
    return {"text": "foo", "source": source, "chunk_index": 0, "score": 0.0}

class TestHitRate:
    def test_hit_when_expected_in_top_k(self):
        results = [_make_result("tinydb/database.py")]
        assert hit_rate(results, ["tinydb/database.py"], k=5) == 1.0

    def test_miss_when_expected_not_found(self):
        results = [_make_result("tinydb/table.py")]
        assert hit_rate(results, ["tinydb/database.py"], k=5) == 0.0

    def test_hit_any_expected(self):
        results = [_make_result("tinydb/storages.py")]
        assert hit_rate(results, ["tinydb/database.py", "tinydb/storages.py"], k=5)
== 1.0

    def test_windows_backslash_normalized(self):
        results = [_make_result("tinydb\\database.py")] # Windows 路径
        assert hit_rate(results, ["tinydb/database.py"], k=5) == 1.0

class TestMRR:
    def test_first_rank(self):
        results = [_make_result("tinydb/database.py")]
        assert mrr(results, ["tinydb/database.py"], k=5) == 1.0

    def test_second_rank(self):
        results = [
            _make_result("tinydb/table.py"),
            _make_result("tinydb/database.py"),
        ]
        assert mrr(results, ["tinydb/database.py"], k=5) == 0.5

    def test_no_match(self):
        results = [_make_result("tinydb/table.py")]
        assert mrr(results, ["tinydb/database.py"], k=5) == 0.0

class TestNDCG:
    def test_perfect_ranking(self):
        results = [
            _make_result("tinydb/database.py"),
            _make_result("tinydb/database.py"),
        ]
        assert abs(ndcg(results, ["tinydb/database.py"], k=5) - 1.0) < 0.001

    def test_relevance_at_second(self):
        results = [
            _make_result("tinydb/table.py"),
            _make_result("tinydb/database.py"),
        ]
        assert abs(ndcg(results, ["tinydb/database.py"], k=5) - 0.5) < 0.001

    def test_duplicate_source_credited_once(self):
        results = [
            _make_result("tinydb/database.py"),
            _make_result("tinydb/database.py"),
            _make_result("tinydb/database.py"),
        ]
        # 同一源文件多个 chunk，只算一次相关性，NDCG 应为 1.0（而非大于 1）

```

```
assert abs(ndcg(results, ["tinydb/database.py"], k=5) - 1.0) < 0.001
```

它验证什么：三个检索评估指标的行为。有几个测试特别值得注意：

- **test_windows_backslash_normalized**：验证 `_normalize` 把 Windows 反斜杠路径转成斜杠后命中——这个测试直接守护了“跨平台结果一致”。
- **test_duplicate_source_credited_once**：验证 NDCG 对同一源文件的多个 chunk 只算一次相关性（matched 集合生效），防止同一个文件刷分。
- MRR 的 `test_second_rank` 验证“排第 2 $\rightarrow$ 0.5”的倒数排名逻辑。

为什么 CI 没跑它？ `ci.yml` 的 `pytest` 命令只跑了三个文件（`test_auth`、`test_auth_api`、`test_fusion`），`test_metrics.py` 没进 CI。但它同样是纯逻辑测试，很容易加进 CI——这正是第 10 章“改进建议 10.8”里提到的点。

tests/ 小结

`tests/` 目录体现了两个工程原则：

1. **测试分层**：纯逻辑测试（`test_fusion`、`test_metrics`、`test_auth` 的哈希）快而稳定；集成测试（`test_auth_api`）依赖 MySQL/Redis 但覆盖真实 HTTP 流程。CI 跑轻量层，保证快速反馈。
2. **环境隔离**：`confest.py` 把测试导向 `code_assistant_test` 库、跳过密钥校验、用假 LLM 密钥——测试环境绝不污染开发环境。

第 9 章 全链路数据流总结

把前面 8 层的所有文件串起来，看一次真实交互的完整数据流。

9.1 全链路流程图

用户 (Streamlit 前端)

| 输入问题



POST /api/agent/chat $\rightarrow$ `middleware.py` 限流检查 (Redis ZSET 滑动窗口)



$\rightarrow$ `auth.py get_current_user` (JWT 验签 + Redis 黑名单 + 查库)



`agent_router.chat`



① `get_or_create_conversation` (`conversation_service` $\rightarrow$ MySQL)



② `load_history` (`conversation_service` $\rightarrow$ MySQL $\rightarrow$ 最近 20 条)



③ `harness.run` (`asyncio.to_thread` $\rightarrow$ 线程池)



`AgentHarness.run` (ReAct 循环, 最多 6 步)



组装 messages: `REACT_SYSTEM_PROMPT` (`prompt.py`, 工具描述动态注入)



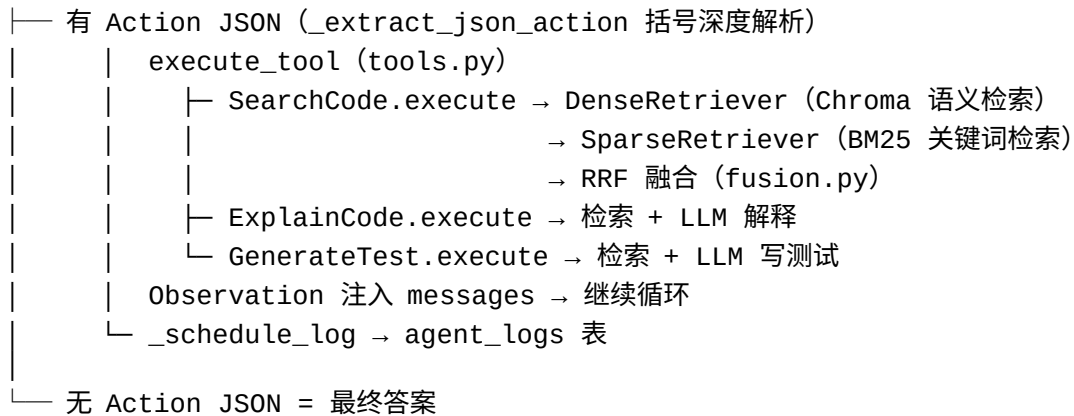
+ 历史 + 用户问题



`call_llm_with_messages` (`llm.py` $\rightarrow$ DeepSeek API, 重试+退避)



▼ 循环判断



conversation_history 更新 → 返回 answer



save_message (user + assistant → messages 表)



ChatResponse 返回给前端

9.2 三种典型交互

① 登录（首次使用）

前端 POST /api/auth/register → 查重 (users 表)

→ bcrypt 哈希密码

→ 建用户 → 签发 access + refresh token

前端存 token → 生成随机 session_id → 进入对话页

涉及: auth_router.register → auth.hash_password / create_access_token → models/user.py。

② Agent 对话（核心场景）

用户: TinyDB 怎么存数据?

Agent 第1步: 思考 → 输出 {"name": "search", "args": {"query": "storage JSON"}}

→ 执行 SearchCode → Chroma + BM25 → RRF → top5 代码片段

→ Observation 注入 → 继续

Agent 第2步: LLM 基于检索到的 storages.py 代码, 组织中文回答

→ 无 Action JSON → 返回答案

→ 保存 user/assistant 消息 → 返回前端

涉及: agent_router.chat → harness.run → llm.call_llm_with_messages → tools.SearchCode → dense_retriever / sparse_retriever / fusion.rtf → agent_logs / conversation_service。

③ 检索（不走 Agent）

前端 POST /api/search {query, use_hybrid: true}

→ rewrite (如果 use_hyde) → query_rewrites 表

→ async_hybrid_search → dense + sparse → RRF → retrieval_logs 表

→ 返回 top-k 代码片段

涉及：search_router.search → fusion.async_hybrid_search → query_rewriter → retrieval_logs。

9.3 数据在哪些表里流转

环节	写哪张表	谁写的
注册/登录	users	auth_router
会话创建	conversations	conversation_service
对话消息	messages	conversation_service
Agent 每步决策	agent_logs	harness._schedule_log
每次检索链路	retrieval_logs	fusion.async_hybrid_search
查询改写	query_rewrites	query_rewriter
消融实验	evaluation_runs	evaluation.run_ablation
索引重建	index_versions	code_indexer
点赞/点踩	feedbacks	预留（前端未接入）

9.4 可观测性：出了问题怎么看

三层可观测是这个项目的亮点：

- 日志（logger.py）：代码里的 log.info/warning/error，带模块名和函数名。
- 检索黑匣子（retrieval_logs）：一次检索的稠密路、稀疏路、融合结果、延迟全记录。
- Agent 黑匣子（agent_logs）：Agent 每一步的思考、行动、观察全记录。

面试可以讲：如果某个用户反馈“Agent 答得不对”，我可以查 agent_logs 回放它每一步做了什么——第 2 步搜了什么、Observation 是什么、为什么第 4 步给出结论。这种“可回放的决策链路”是 Agent 应用生产化的关键。

第 10 章 潜在改进建议

以下改进点按“性价比”排序。前几个是“低成本高收益”，最后一个可以做成“你主动说出的架构演进”。

10.1 单例化 DenseRetriever（成本低、收益高）

现状：tools.py、search_router.py 里每次请求都 DenseRetriever()，而它的构造函数要加载 **embedding 模型（几百 MB）+ 初始化 Chroma**。

问题：每次请求重新加载模型，内存和耗时都浪费。

改法：模块级单例 + 懒加载。

```
# dense_retriever.py 末尾
_dense_retriever = None

def get_dense_retriever() -> DenseRetriever:
    global _dense_retriever
    if _dense_retriever is None:
        _dense_retriever = DenseRetriever()
    return _dense_retriever
```

注意：单例要考虑线程安全（加锁或依赖 FastAPI 的依赖注入缓存）。面试可以说“我意识到每次重建开销大，用懒加载单例优化，注意了并发下的线程安全”。

10.2 Agent 流式输出（成本中、展示效果好）

现状：/api/agent/chat 同步等 Agent 跑完才返回，前端转圈 10-30 秒。

改法：SSE（Server-Sent Events）或 WebSocket，把 Agent 的思考过程**流式推送**给前端，用户实时看到“正在搜索…正在分析…”。

价值：体验提升 + 展示“我的 Agent 有实时可观测性”。这也是你记忆里舆情系统规划过的 WebSocket 方案，可以复用。

10.3 补齐 search_router 的 latency_ms

现状：SearchResponse.latency_ms 写死 0.0（代码注释也标了“简化”）。

改法：async_hybrid_search 已经返回 latency_ms，直接在响应里带上。

价值：接口完整性 + 前端可以展示延迟，和消融实验的延迟数据呼应。

10.4 Redis 不可用时的降级策略统一

现状：auth 黑名单、限流、检索缓存各自处理 Redis 降级（有的 except: pass，有的置 None）。

改法：抽一个 RedisClient 工具类统一管理连接 + 降级策略，get_redis() 返回 None 或统一的降级包装。

价值：消除重复代码，面试展示“我注意到重复模式，做了抽象”。

10.5 为 feedbacks 表接入前端

- 现状：

feedbacks 表存在，但前端没有点赞/点踩按钮，路由也没有对应接口。
- 改法：

加 /api/feedback 接口 + 前端按钮，收集用户对回答的反馈，为后续微调/评测提供真实数据。
- 价值：

闭环数据飞轮——真实反馈是评估的“金标准”。

10.6 向量化检索与关键词检索的并发（成本中、收益中）

- 现状：

hybrid_search 串行执行 dense 和 sparse。
- 改法：

dense.search 和 sparse.search 并行（asyncio.gather + to_thread），两路无依赖，可以同时跑。
- 价值：

延迟减半（两路中最慢的那个），体现对并发的理解。

10.7 索引版本升级链路（架构演进）

- 现状：

index_versions 记录了每次索引配置，但没有“索引过期”机制。如果改了分块策略，旧的 Chroma 索引不会自动重建。
- 改法：

启动时对比 index_versions 最新记录的 strategy/chunk_size 和当前配置，不一致则自动触发 rebuild_index()。
- 价值：

配置即真相——索引自动跟随配置，面试展示“我考虑过索引的生命周期管理”。

10.8 单元测试覆盖扩展

- 现状：

CI 只跑 test_auth.py、test_auth_api.py、test_fusion.py。RAG 的指标（test_metrics.py）和会话服务没有进 CI。
- 改法：

把 test_metrics.py 加进 CI（纯逻辑、无重型依赖）；为 conversation_service、user_upload 的隔离逻辑补测试。
- 价值：

测试覆盖是工程质量的门面。README 里写的安全修复（隔离、限流）最好都有测试背书。

附录 A 环境变量参考

变量	必填	默认值	说明
LLM_API_KEY	-	DeepSeek API 密钥（sk- 开头）	

变量	必填	默认值	说明
LLM_API_ENDPOINT	否	DeepSeek	LLM 服务地址（Ollama 本地可切换）
LLM_MODEL	否	deepseek-v4-flash	模型名
JWT_SECRET		-	JWT 签名密钥，至少 32 字符
MYSQL_HOST	否	localhost	MySQL 地址
MYSQL_PORT	否	3306	MySQL 端口
MYSQL_USER	否	root	MySQL 用户
MYSQL_PASSWORD	否	123456	MySQL 密码
MYSQL_DATABASE	否	code_assistant	数据库名
REDIS_HOST	否	localhost	Redis 地址
REDIS_PORT	否	6379	Redis 端口
REDIS_CACHE_TTL	否	300	检索缓存过期（秒）
CHROMA_PERSIST_DIR	否	./data/chroma	Chroma 持久化目录
REPO_PATH	否	./data/target_repo	要索引的代码仓库
CHUNK_STRATEGY	否	recursive	分块策略

```
密钥生成命令: python -c "import secrets;
print(secrets.token_hex(32))"
```

文档完。按「地基 → 数据 → 安全 → RAG → Agent → 服务 → 路由 → 部署 → 测试」的顺序，项目 46 个代码文件全部讲解完毕。祝你面试顺利！